

C.8 DEZASAMBLARE ȘI PATCHING CU X96DBG

PAUL A. GAGNIUC



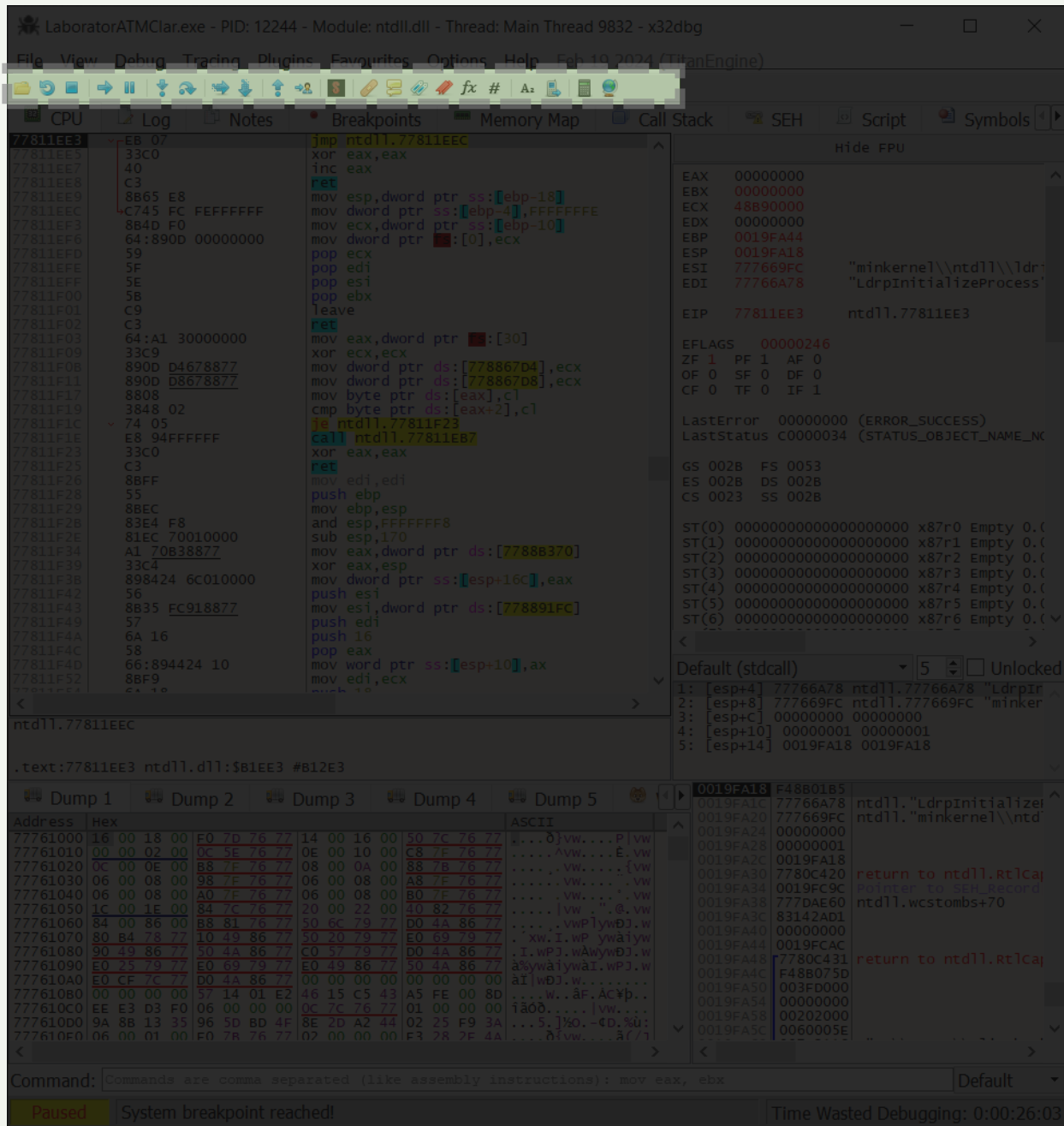
Academia Tehnică Militară „Ferdinand I”

PRINCIPALELE PĂRȚI ALE PREZENTĂRII

C.8 Dezasamblare și Patching cu X32dbg / X64dbg:

- C.8.1 IDENTIFICAREA PAROLELOR ASCUNSE IN EXECUTABILE
- C.8.2 ADĂUGAREA DE COD MAȘINĂ
- C.8.3 MODIFICARE ENTRY POINT ȘI INSERȚIE DE COD MAȘINĂ
- C.8.4 ADAUGAREA DE STRUCTURI DE DATE IN EXECUTABILE
- C.8.5 MODIFICĂRI DE ADRESE CĂTRE ALTE ARGUMENTE

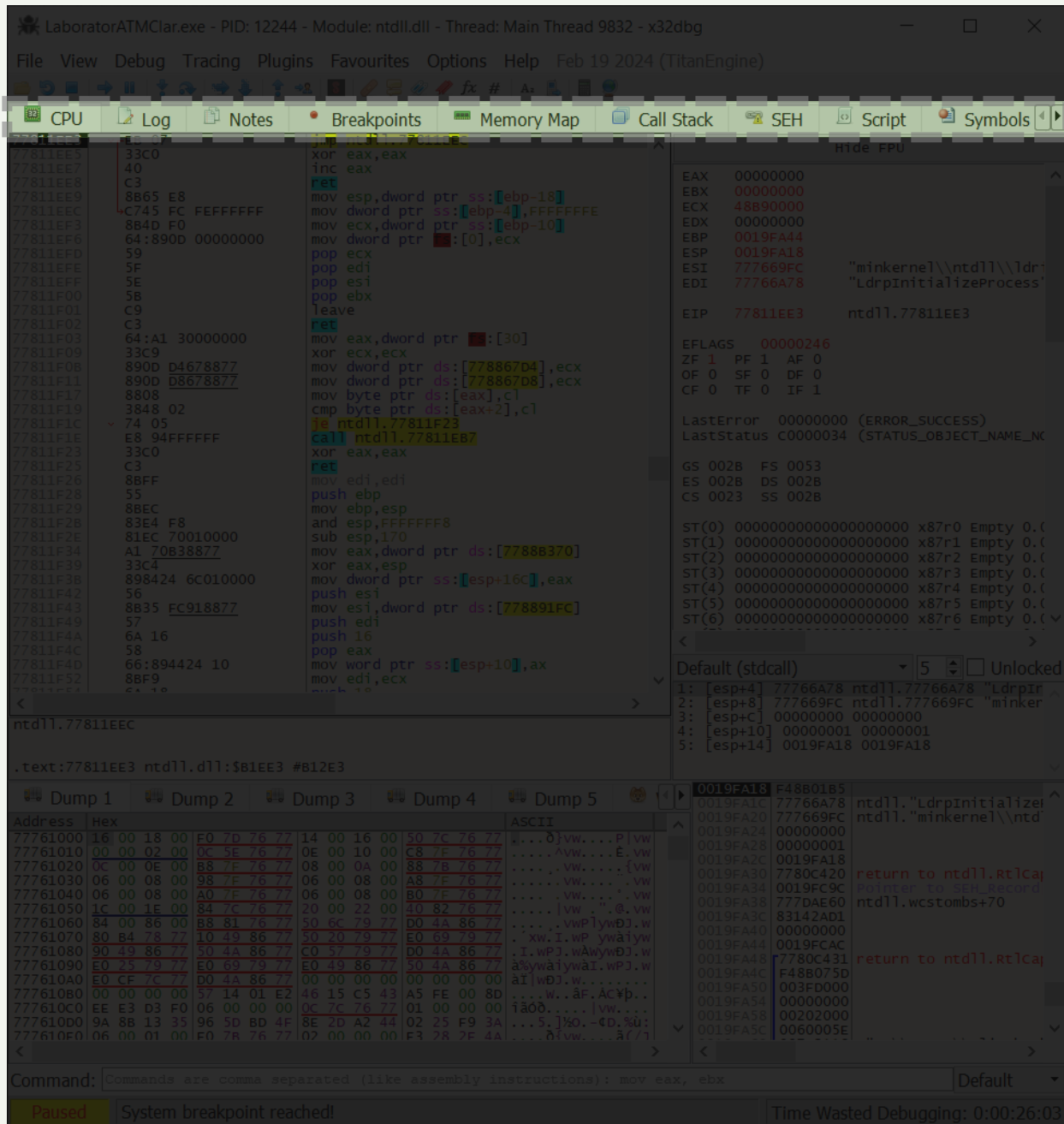




X64dbg User Interface

Play Controls

Bara de meniu oferă acces la opțiuni avansate de configurare, controlul execuției, încărcarea sau salvarea de sesiuni, integrarea de pluginuri și personalizarea mediului de depanare. Rolul ei este de a centraliza și structura toate comenzile disponibile, servind drept punct de plecare pentru orice acțiune pe care utilizatorul dorește să o efectueze asupra executabilului analizat.



X64dbg User Interface

View Tabs

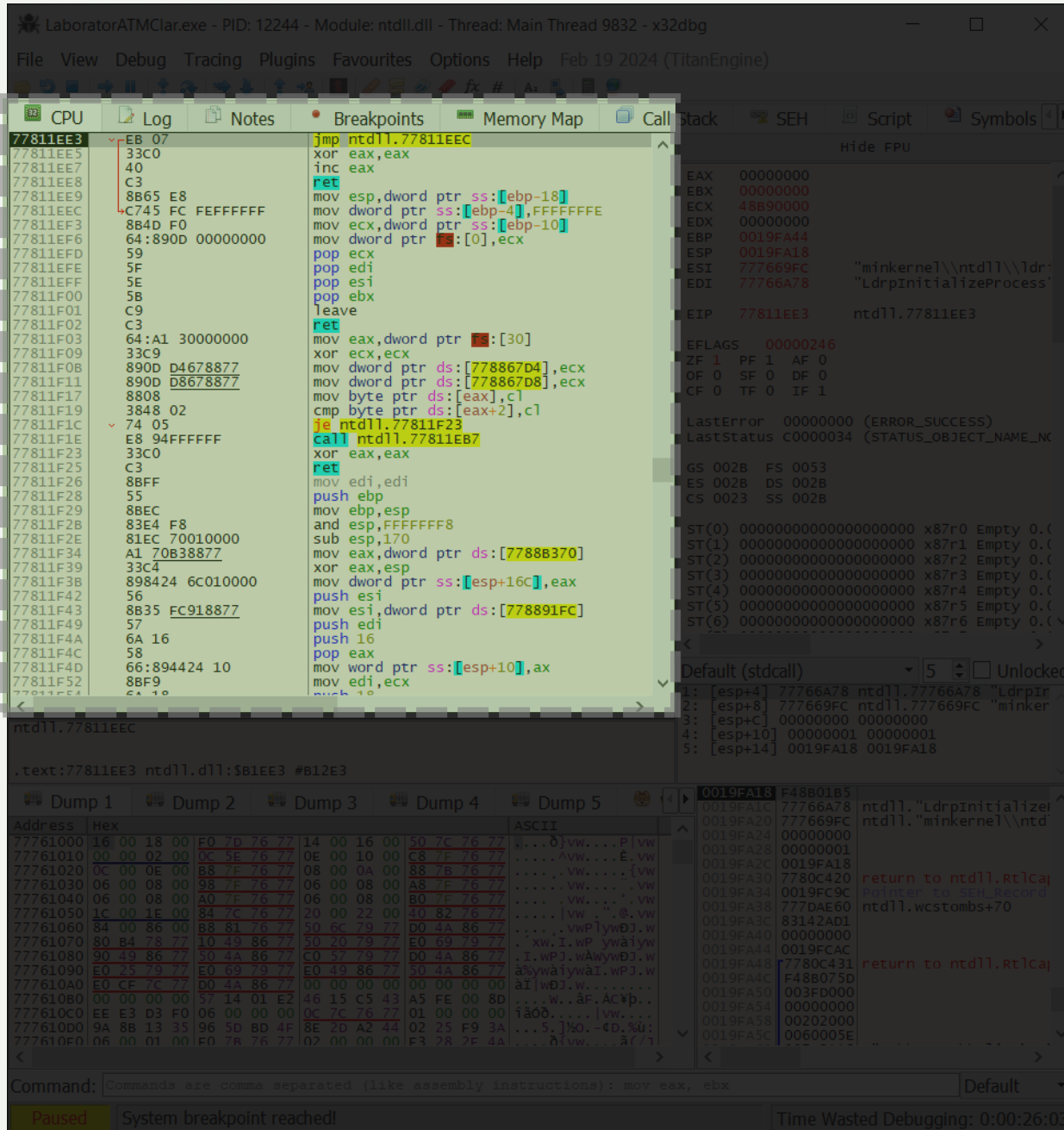
Taburile ofera acces rapid la funcțiile esențiale ale depanatorului, precum vizualizarea CPU, log-urile de execuție, notițele, gestionarea breakpoints, harta de memorie, stiva de apeluri (Call Stack), gestionarea excepțiilor (SEH), rularea scripturilor și afișarea simbolurilor.

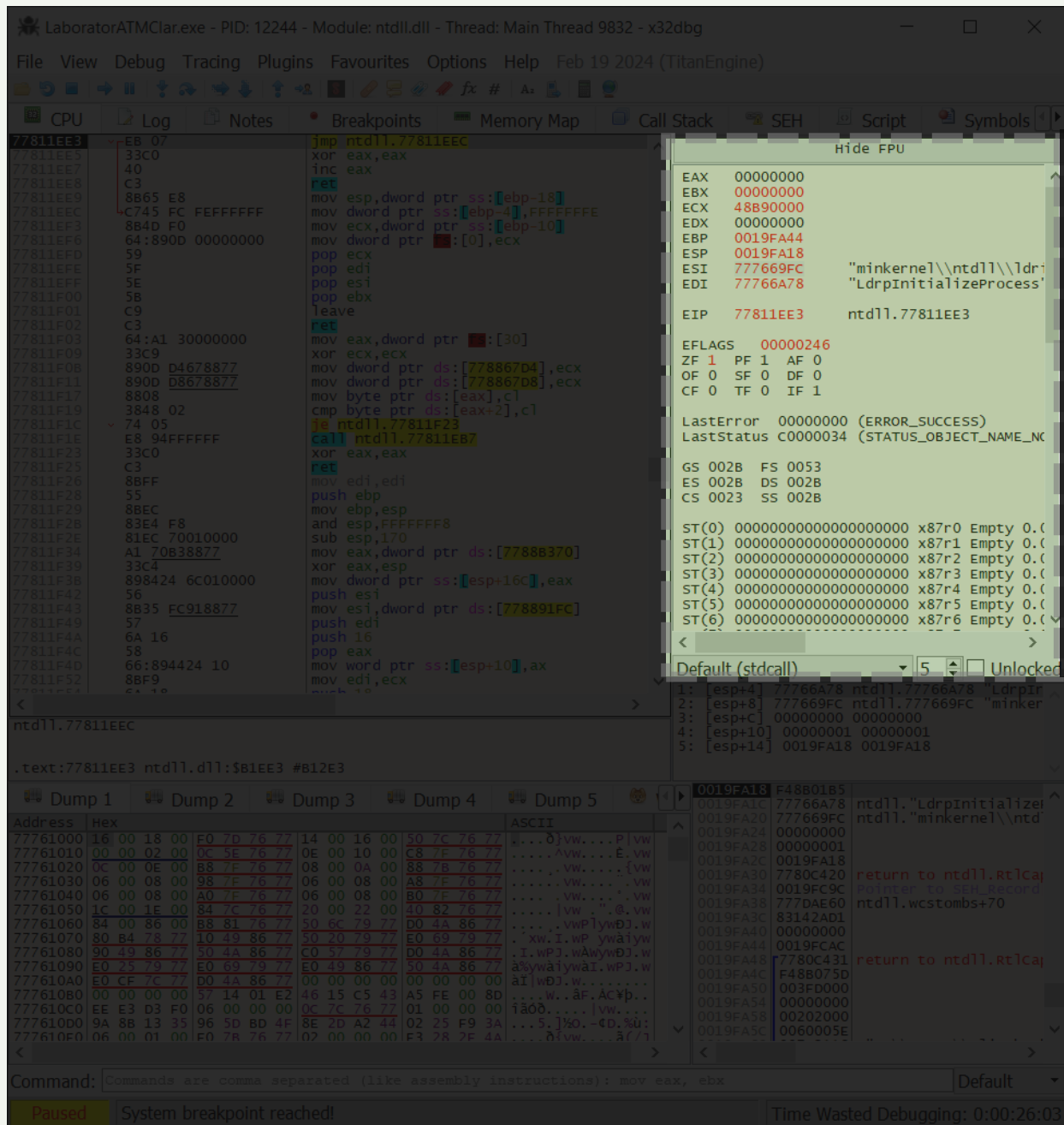
Practic, aceste taburi controlează ce tip de informații sunt afișate în fereastra de lucru, iar utilizatorul poate comuta între ele pentru a analiza diferite aspecte ale executabilului. Bara de instrumente are rolul de a centraliza principalele unelte de analiză, oferind o navigare rapidă și structurată prin componentele procesului aflat în execuție.

X64dbg User Interface

CPU Assembly

Aici se afla fereastra principală de disassembly. Rolul ei este să arate instrucțiunile pe care procesorul urmează să le execute, traduse din cod mașină în limbaj de asamblare. Utilizatorul poate urmări pas cu pas cum se execută programul, poate seta breakpoints, poate vedea fluxul de execuție și apelurile de funcții. Practic, această fereastră este „inima” analizării unui executabil, deoarece afișează direct logica internă a programului așa cum este interpretată de procesor.





X64dbg User Interface

Registers

Rolul acestei zone este să ofere o imagine completă asupra execuției programului la nivel de procesor. Aici se văd valorile regiștrilor și ale flag-urilor procesorului, care se actualizează după fiecare instrucțiune executată. În partea de jos sunt afișate informații despre stivă, inclusiv adresele de retur și parametrii funcțiilor. Practic, aceasta zonă este nucleul analizei: combină instrucțiunile, starea regiștrilor și conținutul stivei pentru a oferi utilizatorului control și vizibilitate totală asupra execuției unui fișier executabil.

Legenda:

- **EAX:** Math and Return Values
- **EBX:** Base index (for use with arrays)
- **ECX:** Counter
- **EDX:** Math
- **EBP:** Base Pointer
- **ESP:** Stack Pointer
- **ESI/EDI:** Memory Transfer

Valorile precum **GS**, **ES**, **FS**, **DS**, **CS**, **SS** sunt registre de segment din arhitectura x86/x86-64.

- **CS (Code Segment)** indică segmentul unde se află codul executabil.
- **DS (Data Segment)** segmentul implicit pentru date.
- **SS (Stack Segment)** segmentul folosit pentru stivă.
- **ES (Extra Segment)** segment suplimentar pentru operații pe date.
- **FS / GS** registrii de segment speciali, folosite de sistemul de operare pentru structuri interne (în Windows, FS și GS sunt utilizate pentru Thread Environment Block (TEB) și alte date legate de thread/proces).

ZF (Zero Flag) setat la 1 dacă rezultatul unei operații este zero.

PF (Parity Flag) indică dacă rezultatul are un număr par de biți setați la 1.

AF (Adjust Flag) folosit pentru operații aritmetice pe 4 biți (ex. BCD).

OF (Overflow Flag) semnalează depășirea limitei la operații pe numere semnate.

SF (Sign Flag) reflectă semnul rezultatului (1 = negativ).

DF (Direction Flag) determină direcția operațiilor pe șiruri (0 = înainte, 1 = înapoi).

CF (Carry Flag) semnalează un transport sau un împrumut la operații aritmetice.

TF (Trap Flag) dacă e setat, activează execuția pas-cu-pas (debugging).

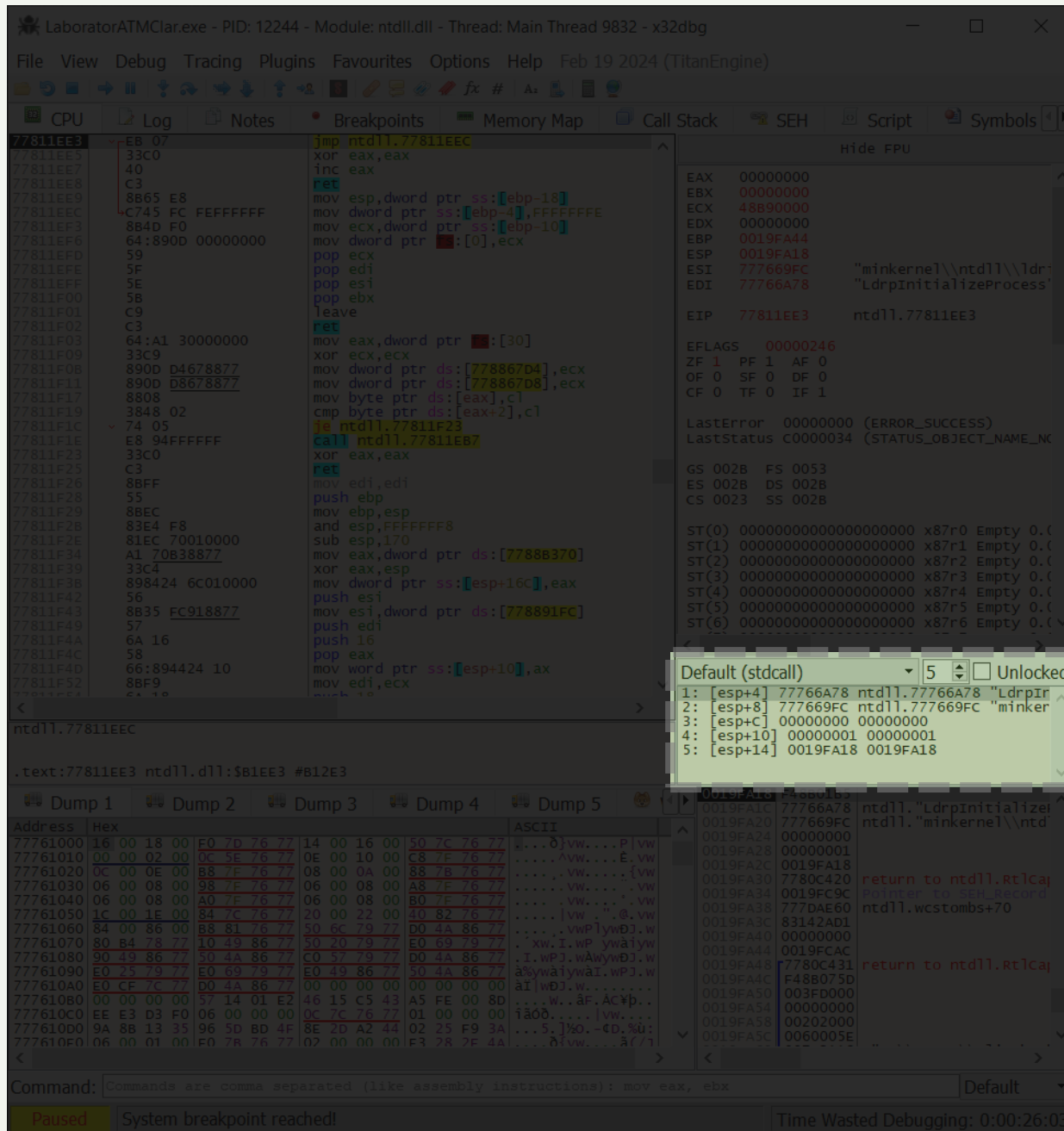
IF (Interrupt Flag) controlează dacă întreruperile hardware sunt permise (1 = activat).

X64dbg User Interface

Stack Window

Conținut brut al stivei (argumente, valori locale)

Această fereastră afișează conținutul stivei procesului, adică zona de memorie unde sunt plasate adrese de retur, variabile locale și parametri funcțiilor apelate. Fiecare linie corespunde unei adrese din stivă, iar valorile afișate permit urmărirea succesiunii apelurilor și a modului în care programul gestionează execuția funcțiilor. În analiza unui executabil, fereastra Stack este esențială pentru a înțelege cum se transmit parametrii, cum se realizează salturile în cod și pentru a identifica eventuale suprascrieri de memorie sau exploatari de tip buffer overflow.

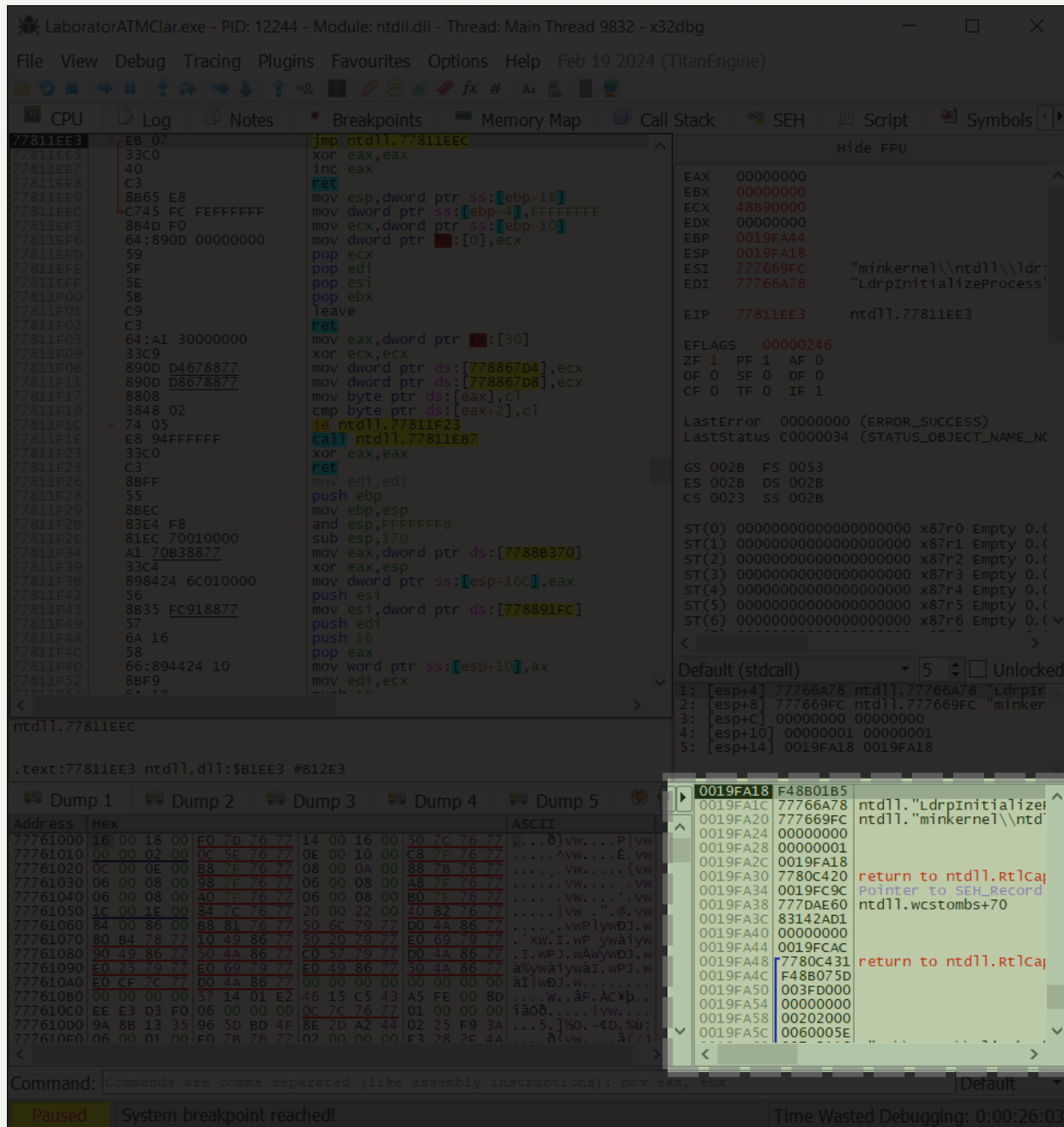


X64dbg User Interface

Call Stack

Lista apelurilor de funcții și adresele de return.

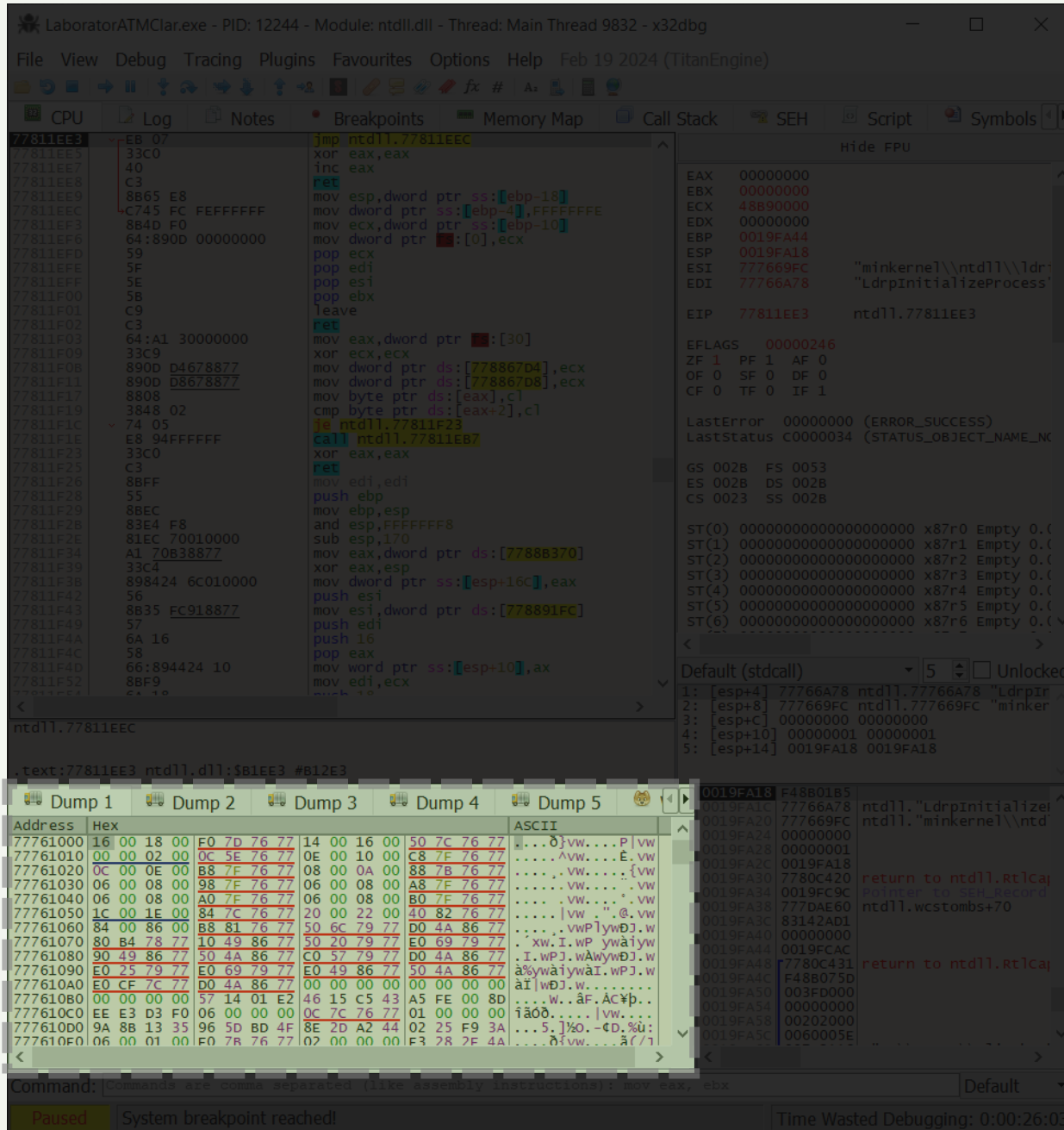
Această zonă arată succesiunea apelurilor de funcții efectuate până la punctul curent de execuție. Fiecare linie indică o adresă de retur și, atunci când este posibil, numele funcției sau modulului asociat. Call Stack-ul este util pentru a înțelege contextul în care se află programul, ordinea apelurilor și pentru a reconstrui traseul execuției. În analiza executabilelor, această vizualizare este esențială pentru depistarea funcțiilor critice, urmărirea fluxului logic și identificarea eventualelor vulnerabilități ce țin de gestionarea apelurilor și a memoriei.



X64dbg User Interface

Dumps (Heap)

Zona de Dump, este cea mai importantă zonă pentru inginerie inversă! Aici se afișează conținutul memoriei procesului sub două forme: în stânga valorile hexazecimale, iar în dreapta echivalentul ASCII acolo unde este posibil. Fereastra permite utilizatorului să vizualizeze și să inspecteze datele încărcate în anumite adrese, cum ar fi șiruri de caractere, pointeri, structuri sau instrucțiuni codificate. În analiza unui executabil, Dump-ul este esențial pentru a observa direct cum sunt stocate și manipulate datele în memorie, fiind un instrument indispensabil în ingineria inversă și detectarea tehnicilor de obfuscare sau criptare.



C.8.1

IDENTIFICAREA PAROLELOR ASCUNSE IN EXECUTABILE



APLICAȚIE PROTEJATĂ PRIN PAROLĂ

CE VEDEM CÂND EXECUTĂM FIȘIERUL ȘI CUM FUNCȚIONEAZĂ?



1

Visual Basic 6.0 este RAD, adică Rapid Application Development)

Introduceți parola:

Verifica

Introduceți parola:

Verifica

Introduceți parola:

incercare

Verifica

Ce se află în spatele aplicației compilate?

LaboratorATM

Parola Incorecta ! Mai incearca

OK

Sursa executabilului:

```
Private secret As String

Private Sub Buton_Click()

    If (InputParola.Text = secret) Then
        MsgBox "Parola corecta !"
    Else
        MsgBox "Parola Incorecta ! Mai incearca"
    End If

End Sub

Private Sub Form_Load()
    secret = "atm2024"
End Sub
```

6

X32dbg ne setează punctul de intrare inițial (nu cel din .text):

LaboratorATMClar.exe - PID: 12244 - Module: ntdll.dll - Thread: Main Thread 9832 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

77811EE3 EB 07 jmp ntdll.77811EEC
77811EE5 33C0 xor eax,eax
77811EE7 40 inc eax
77811EE8 C3 ret
77811EE9 8B65 E8 mov esp,dword ptr ss:[ebp-18]
77811EEC C745 FC FE mov dword ptr ss:[ebp-4],FFFFFFFE
77811EF3 8B4D F0 mov ecx,dword ptr ss:[ebp-10]
77811EF6 64:890D 00000000 mov dword ptr [0],ecx
77811EFD 59 pop ecx
77811EFE 5F pop esi
77811EFF 5E pop edi
77811F00 5B pop ebx
77811F01 C9 leave
77811F02 C3 ret
77811F03 64:A1 30000000 mov eax,dword ptr [30]
77811F09 33C9 xor ecx,ecx
77811F0B 890D D4678877 mov dword ptr ds:[778867D4],ecx
77811F11 890D D8678877 mov dword ptr ds:[778867D8],ecx
77811F17 8808 mov byte ptr ds:[eax],cl
77811F19 3848 02 cmp byte ptr ds:[eax+2],cl
77811F1C 74 05 je ntdll.77811F23
77811F1E E8 94FFFFFF call ntdll.77811EB7
77811F23 33C0 xor eax,eax
77811F25 C3 ret
77811F26 8BFF mov edi,edi
77811F28 55 push ebp
77811F29 8BEC mov ebp,esp
77811F2B 83E4 F8 and esp,FFFFFFF8
77811F2E 81EC 70010000 sub esp,170
77811F34 A1 70B38877 mov eax,dword ptr ds:[7788B370]
77811F39 33C4 xor eax,esp
77811F3B 898424 6C010000 mov dword ptr ss:[esp+16c],eax
77811F42 56 push esi
77811F43 8B35 FC918877 mov esi,dword ptr ds:[778891FC]
77811F49 57 push edi
77811F4A 6A 16 push 16
77811F4C 58 pop eax
77811F4D 66:894424 10 mov word ptr ss:[esp+10],ax
77811F52 8BF9 mov edi,ecx
77811F54 6A 16 push 16

Hide FPU

EAX 00000000
EBX 00000000
ECX 48890000
EDX 00000000
EBP 0019FA44
ESP 0019FA18
ESI 777669FC
EDI 77766A78
EIP 77811EE3 ntdll.77811EE3

EF 00000000
PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C0000034 (STATUS_OBJECT_NAME_NOT_FOUND)

GS 002B FS 0053
ES 002B DS 002B
CS 0023 SS 002B

ST(0) 00000000000000000000000000000000 x87r0 Empty 0.0
ST(1) 00000000000000000000000000000000 x87r1 Empty 0.0
ST(2) 00000000000000000000000000000000 x87r2 Empty 0.0
ST(3) 00000000000000000000000000000000 x87r3 Empty 0.0
ST(4) 00000000000000000000000000000000 x87r4 Empty 0.0
ST(5) 00000000000000000000000000000000 x87r5 Empty 0.0
ST(6) 00000000000000000000000000000000 x87r6 Empty 0.0

Default (stdcall) 5 Unlocked

1: [esp+4] 77766A78 ntdll.77766A78 "LdrpIr
2: [esp+8] 777669FC ntdll.777669FC "minker
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 0019FA18 0019FA18

ntdll.77811EEC

.text:77811EE3 ntdll.dll:\$B1EE3 #B12E3

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
77761000	16 00 18 00 F0 7D 76 77	...0}vw...P vw
77761010	00 00 02 00 0C 5E 76 77	...vw...E.vw
77761020	0C 00 0E 00 88 7F 76 77	...vw...{vw
77761030	06 00 08 00 98 7F 76 77	...vw...vw
77761040	06 00 08 00 A0 7F 76 77	...vw...vw
77761050	1C 00 1E 00 84 7C 76 77	...vw...@.vw
77761060	84 00 86 00 88 81 76 77	...vwPlywDJ.w
77761070	80 B4 78 77 10 49 86 77	...xw.I.wP ywaiy
77761080	90 49 86 77 50 4A 86 77	...I.wPJ.wAiywDJ.w
77761090	E0 25 79 77 E0 69 79 77	...aiywaiy.wPJ.w
777610A0	E0 CF 7C 77 D0 4A 86 77	...ai wDJ.w...
777610B0	00 00 00 00 57 14 01 E2	...w..äF.ACyp..
777610C0	EA E3 D3 F0 06 00 00 00	...iäö... vw...
777610D0	9E 88 13 35 96 5D BD 4F	...5.}%o.-d,%u:
777610E0	06 00 01 00 F0 78 76 77	...ä vw...ä(7/

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused System breakpoint reached! Time Wasted Debugging: 0:00:26:03

Căutăm șiruri în executabil:

LaboratorATMClar.exe - PID: 12244 - Module: ntdll.dll - Thread: Main Thread 9832 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

77811EE3 EB 07 jmp ntdll.77811EEC
77811EE5 33C0 xor eax,eax
77811EE7 40 inc eax
77811EE8 C3 ret
77811EE9 8B65 E8 mov esp,dword ptr ss:[ebp-18]
77811EEC C745 FC FEFF mov dword ptr ss:[ebp-4],FFFFFFF
77811EF3 8B4D F0 mov ecx,dword ptr ss:[ebp-10]
77811EF6 64:890D 00000000 mov dword ptr [0],ecx
77811EFD 59 pop ecx
77811EFE 5F pop esi
77811EFF 5E pop edi
77811F00 5B pop ebx
77811F01 C9 leave
77811F02 C3 ret
77811F03 64:A1 30000000 mov eax,dword ptr [30]
77811F09 33C9 xor ecx,ecx
77811F0B 890D D4678877 mov dword ptr ds:[778867D4],ecx
77811F11 890D D8678877 mov dword ptr ds:[778867D8],ecx
77811F17 8808 mov byte ptr ds:[eax],cl
77811F19 3848 02 cmp byte ptr ds:[eax+2],cl
77811F1C 74 05 je ntdll.77811F23
77811F1E E8 94FFFFFF call ntdll.77811EB7
77811F23 33C0 xor eax,eax
77811F25 C3 ret
77811F26 8BFF mov edi,edi
77811F28 55 push ebp
77811F29 8BEC mov ebp,esp
77811F2B 83E4 F8 and esp,FFFFFFF8
77811F2E 81EC 70010000 sub esp,170
77811F34 A1 70B38877 mov eax,dword ptr ds:[7788B370]
77811F39 33C4 xor eax,esp
77811F3B 898424 6C010000 mov dword ptr ss:[esp+16c],eax
77811F42 56 push esi
77811F43 8B35 FC918877 mov esi,dword ptr ds:[778891FC]
77811F49 57 push edi
77811F4A 6A 16 push 16
77811F4C 58 pop eax
77811F4D 66:894424 10 mov word ptr ss:[esp+10],ax
77811F52 8BF9 mov edi,ecx
77811F54 6A 16 push 16

Binary
Copy
Breakpoint
Follow in Dump
Follow in Disassembler
Follow in Memory Map
Graph
Help on mnemonic
Show mnemonic brief
Highlighting mode
Edit columns...
Label
Comment
Toggle Bookmark
Trace coverage
Analysis
Download Symbols for This Module
Assemble
Patches
Set EIP Here
Create New Thread Here
Go to
Search for
Find references to
Command
Constant
String references
Intermodule
Pattern
GUID

ntdll.77811EEC

.text:77811EE3 ntdll.dll:\$B1EE3 #B12E3

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
77761000	16 00 18 00 F0 7D 76 77	...0}vw...P vw
77761010	00 00 02 00 0C 5E 76 77	...vw...E.vw
77761020	0C 00 0E 00 88 7F 76 77	...vw...{vw
77761030	06 00 08 00 98 7F 76 77	...vw...vw
77761040	06 00 08 00 A0 7F 76 77	...vw...vw
77761050	1C 00 1E 00 84 7C 76 77	...vw...@.vw
77761060	84 00 86 00 88 81 76 77	...vwPlywDJ.w
77761070	80 B4 78 77 10 49 86 77	...xw.I.wP ywaiy
77761080	90 49 86 77 50 4A 86 77	...I.wPJ.wAiywDJ.w
77761090	E0 25 79 77 E0 69 79 77	...aiywaiy.wPJ.w
777610A0	E0 CF 7C 77 D0 4A 86 77	...ai wDJ.w...
777610B0	00 00 00 00 57 14 01 E2	...w..äF.ACyp..
777610C0	EA E3 D3 F0 06 00 00 00	...iäö... vw...
777610D0	9E 88 13 35 96 5D BD 4F	...5.}%o.-d,%u:
777610E0	06 00 01 00 F0 78 76 77	...ä vw...ä(7/

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused System breakpoint reached! Time Wasted Debugging: 0:00:26:23

Prin forța brută putem testa toate șirurile, dar unele sunt evidente:

Breakpoints Memory Map Call Stack SEH Script Symbols Source References			
User Modules (Strings)			
Address	Disassembly	String A	String
0041EFEE	cmp eax,575D40	00575D40	&L"nsemanager-11-1-0"
0043F86F	push advapi32.7567FF95	7567FF95	"0\$br"
00444D7E	and eax,594D30	00594D30	"<\t"
00466205	mov dword ptr ss:[ebp-64],laboratoratmclar.465C54	00465C54	L"Parola corecta !"
00466248	mov dword ptr ss:[ebp-64],laboratoratmclar.465C7C	00465C7C	L"Parola Incorecta ! Mai incearca"
00466330	mov edx,laboratoratmclar.465CC0	00465CC0	L"atm2024"
00484755	push advapi32.7567FF95	7567FF95	"0\$br"
00489C64	and eax,594D30	00594D30	"<\t"
0048B724	cmp eax,575D40	00575D40	&L"nsemanager-11-1-0"

1

Facem dublu clic pe șirul de interes care ne-a permis accesul la program.



3

Putem verifica direct string-urile pe care le găsim. Parola este ușor de găsit atâta timp cât este în text clar. Întrebarea este: Cum putem schimba parola din fisierul executabil?

LaboratorATMClar.exe - PID: 12244 - Module: laboratoratmclar.exe - Thread: Main Thread 9832 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00466330 BA C05C4600 mov edx,laboratoratmclar.465CC0
00466335 8D4E 34 lea ecx,dword ptr ds:[esi+4]
00466338 FF15 68104000 call dword ptr ds:[<esi+4>] ; <copy>
0046633E C745 FC 00000000 mov dword ptr ss:[ebp-4],ecx ; <copy>
00466345 8B45 08 mov eax,dword ptr ss:[ebp-4]
00466348 50 push eax
00466349 8B10 mov edx,dword ptr ds:[esi+4]
0046634B FF52 08 call dword ptr ds:[<esi+4>] ; <copy>
0046634E 8B45 FC mov eax,dword ptr ss:[ebp-4]
00466351 8B4D EC mov ecx,dword ptr ss:[ebp-14]
00466354 5F pop edi
00466355 5E pop esi
00466356 64:890D 00000000 mov dword ptr [0],ecx
0046635D 5B pop ebx
0046635E 8BE5 mov esp,ebp
00466360 5D pop ebp
00466361 C2 0400 ret 4
00466364 90 nop
00466365 90 nop
00466366 90 nop
00466367 90 nop
00466368 90 nop
00466369 90 nop
0046636A 90 nop
0046636B 90 nop
0046636C 90 nop
0046636D 90 nop
0046636E 90 nop
0046636F 90 nop
00466370 9E sahf
00466371 9E sahf
00466372 9E sahf
00466373 9E sahf
00466374 9C sahf
00466375 6306 pushfd
00466377 00FF arpl word ptr ds:[esi],ax
00466379 FF add bh,bh
0046637A FF
0046637B FF
0046637C FF

edx=0
00465CC0 L"atm2024"

.text:00466330 laboratoratmclar.exe:\$66330 #66330

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII	
77761000	16 00 18 00 F0 7D 76 77	14 00 16 00 50 7C 76 77	00000000
77761010	00 00 02 00 0C 5E 76 77	0E 00 10 00 C8 7F 76 77	00000001
77761020	0C 00 0E 00 B8 7F 76 77	08 00 0A 00 88 7F 76 77	00000002
77761030	06 00 08 00 98 7F 76 77	06 00 08 00 A8 7F 76 77	00000003
77761040	06 00 08 00 80 7F 76 77	06 00 08 00 80 7F 76 77	00000004
77761050	1C 00 1E 00 84 7C 76 77	20 00 22 00 40 82 76 77	00000005
77761060	84 00 86 00 B8 81 76 77	50 6C 79 77 D0 4A 86 77	00000006
77761070	80 B4 78 77 10 49 86 77	50 20 79 77 E0 69 79 77	00000007
77761080	90 49 86 77 50 4A 86 77	C0 57 79 77 D0 4A 86 77	00000008
77761090	E0 25 79 77 E0 69 79 77	E0 49 86 77 50 4A 86 77	00000009
777610A0	E0 CF 7C 77 D0 4A 86 77	00 00 00 00 00 00 00 00	0000000A
777610B0	00 00 00 00 57 14 01 E2	46 15 C5 43 A5 FE 00 8D	0000000B
777610C0	EE E3 D3 F0 06 00 00 00	0C 7C 76 77 01 00 00 00	0000000C
777610D0	9A 88 13 35 50 5D BD 4F	8E 2D A2 44 02 25 F9 3A	0000000D
777610E0	06 00 01 00 F0 78 76 77	02 00 00 00 F3 28 7F 4A	0000000E

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused 9 string(s) in 312ms Time Wasted Debugging: 0:00:29:54

Apoi X32dbg ne arată linia care indică o anumită adresă în dump (memorie)

Accesăm adresa 00465CC0 din memoria dump:

LaboratorATMClar.exe - PID: 12244 - Module: laboratoratmclar.exe - Thread: Main Thread 9832 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00466330 BA C05C4600 mov edx,laboratoratmclar.465CC0
00466335 8D4E 34 lea ecx,dword ptr ds:[esi+34]
00466338 FF15 68104000 call dword ptr ds:[<vbaStrCopy>]
0046633E C745 FC 00000000 mov dword ptr ss:[ebp-4],0
00466345 8B45 08 mov eax,dword ptr ss:[ebp+8]
00466348 50 push eax
00466349 mov edx,dword ptr ds:[eax]
0046634B FF52 08 call dword ptr ds:[edx+8]
0046634E 8B45 FC mov ecx,dword ptr ss:[ebp-4]
00466351 8B4D EC mov ecx,dword ptr ss:[ebp-14]
00466354 5F pop edi
00466355 5E pop esi
00466356 64:890D 00000000 mov dword ptr [0],ecx
0046635D 5B mov ebx,esp
0046635E 8BE5 esp,ebp
00466360 5D pop ebp
00466361 C2 0400 ret 4
00466364 90 nop
00466365 90 nop
00466366 90 nop
00466367 90 nop
00466368 90 nop
00466369 90 nop
0046636A 90 nop
0046636B 90 nop
0046636C 90 nop
0046636D 90 nop
0046636E 90 nop
0046636F 90 nop
00466370 9E sahf
00466371 9E sahf
00466372 9E sahf
00466373 9E sahf
00466374 9C pushfd
00466375 arpl word ptr ds:[esi],ax
00466377 00FF add bh,bh
00466379 FF
0046637A FF
0046637B FF

Selected Address
laboratoratmclar.00465CC0

Binary
Copy
Breakpoint
Follow in Dump
Follow in Disassembler
Follow in Memory Map
Graph
Help on mnemonic Ctrl+F1
Show mnemonic brief Ctrl+Shift+F1
Highlighting mode H
Edit columns...
Label
Comment ;
Toggle Bookmark Ctrl+D
Trace coverage
Analysis
Assemble Space
Patches Ctrl+P
Set EIP Here Ctrl+*
Create New Thread Here
Go to
Search for
Find references to

Address Hex
77761000 16 00 18 00 F0 7D 76 77 14 00 16 00 50 7C 76 77
77761010 00 00 02 00 0C 5E 76 77 0E 00 10 00 C8 7F 76 77
77761020 0C 00 0E 00 B8 7E 76 77 08 00 0A 00 88 7B 76 77
77761030 06 00 08 00 08 7E 76 77 06 00 08 00 A8 7E 76 77
77761040 06 00 08 00 A0 7E 76 77 06 00 08 00 80 7E 76 77
77761050 1C 00 1E 00 84 7C 76 77 20 00 22 00 40 82 76 77
77761060 84 00 86 00 B8 81 76 77 50 6C 79 77 D0 4A 86 77
77761070 80 B4 78 77 10 49 86 77 50 20 79 77 E0 69 79 77
77761080 90 49 86 77 50 4A 86 77 C0 57 79 77 D0 4A 86 77
77761090 E0 25 79 77 E0 69 79 77 E0 49 86 77 50 4A 86 77
777610A0 E0 CF 7C 77 D0 4A 86 77 00 00 00 00 00 00 00 00
777610B0 00 00 00 00 57 14 01 E2 46 15 C5 43 A5 FE 06 8D
777610C0 EE E3 D3 F0 06 00 00 00 0C 7C 76 77 01 00 00 00
777610D0 9A 8B 13 35 96 5D BD 4F 8E 2D A2 44 02 25 F9 3A
777610E0 06 00 01 00 F0 78 76 77 02 00 00 00 F3 28 2F 4A

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused 9 string(s) in 312ms Time Wasted Debugging: 0:00:31:06

În memoria dump putem vedea clar parola:

LaboratorATMClar.exe - PID: 12244 - Module: laboratoratmclar.exe - Thread: Main Thread 9832 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00466330 BA C05C4600 mov edx,laboratoratmclar.465CC0
00466335 8D4E 34 lea ecx,dword ptr ds:[esi+34]
00466338 FF15 68104000 call dword ptr ds:[<vbaStrCopy>]
0046633E C745 FC 00000000 mov dword ptr ss:[ebp-4],0
00466345 8B45 08 mov eax,dword ptr ss:[ebp+8]
00466348 50 push eax
00466349 mov edx,dword ptr ds:[eax]
0046634B FF52 08 call dword ptr ds:[edx+8]
0046634E 8B45 FC mov ecx,dword ptr ss:[ebp-4]
00466351 8B4D EC mov ecx,dword ptr ss:[ebp-14]
00466354 5F pop edi
00466355 5E pop esi
00466356 64:890D 00000000 mov dword ptr [0],ecx
0046635D 5B mov ebx,esp
0046635E 8BE5 esp,ebp
00466360 5D pop ebp
00466361 C2 0400 ret 4
00466364 90 nop
00466365 90 nop
00466366 90 nop
00466367 90 nop
00466368 90 nop
00466369 90 nop
0046636A 90 nop
0046636B 90 nop
0046636C 90 nop
0046636D 90 nop
0046636E 90 nop
0046636F 90 nop
00466370 9E sahf
00466371 9E sahf
00466372 9E sahf
00466373 9E sahf
00466374 9C pushfd
00466375 arpl word ptr ds:[esi],ax
00466377 00FF add bh,bh
00466379 FF
0046637A FF
0046637B FF

Hide FPU

EAX 00000000
EBX 00000000
ECX 48890000
EDX 00000000
EBP 0019FA44
ESI 0019FA18
EDI 77766A78
EIP 77811EE3 ntdll.77811EE3

EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError: 00000000 (ERROR_SUCCESS)
LastStatus: C0000034 (STATUS_OBJECT_NAME_NOT_FOUND)

GS 002B FS 0053
ES 002B DS 002B
CS 0023 SS 002B

ST(0) 00000000000000000000000000000000 x87r0 Empty 0.0
ST(1) 00000000000000000000000000000000 x87r1 Empty 0.0
ST(2) 00000000000000000000000000000000 x87r2 Empty 0.0
ST(3) 00000000000000000000000000000000 x87r3 Empty 0.0
ST(4) 00000000000000000000000000000000 x87r4 Empty 0.0
ST(5) 00000000000000000000000000000000 x87r5 Empty 0.0
ST(6) 00000000000000000000000000000000 x87r6 Empty 0.0

Default (stdcall) 5 Unlocked

1: [esp+4] 77766A78 ntdll.77766A78 "Ldrp
2: [esp+8] 777669FC ntdll.777669FC "minker
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 0019FA18 0019FA18

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address Hex
00465CC0 61 00 74 00 6D 00 32 00 30 00 32 00 34 00 00 00
00465CD0 16 42 41 36 2E 44 4C 4C 00 00 00 00 5F 5F 76 62
00465CE0 51 53 74 72 43 6F 70 79 00 00 00 00 5F 5F 76 62
00465CF0 51 46 72 65 65 56 61 72 4C 69 73 74 00 00 00 00
00465D00 51 46 72 65 61 56 61 72 44 75 70 00 5F 5F 76 62
00465D10 51 46 72 65 65 53 74 72 00 00 00 00 5F 5F 76 62
00465D20 51 46 72 65 65 53 74 72 43 68 65 63 68 65 74 00
00465D30 00 00 00 00 5F 5F 76 62 61 4F 62 6A 53 65 74 00
00465D40 5F 5F 76 62 61 53 74 72 43 6D 70 00 01 00 00 00
00465D50 64 5A 46 00 00 00 00 00 78 60 46 00 FF FF FF FF
00465D60 00 00 00 00 B8 5A 46 00 08 70 46 00 00 00 00 00
00465D70 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00465D80 08 27 D9 06 00 00 00 00 00 00 00 00 00 00 00 00
00465D90 04 5D 46 00 01 00 00 00 28 5B 46 00 00 00 00 00
00465DA0 04 5D 46 00 01 00 00 00 04 5D 46 00 00 00 00 00

return to ntdll.RtlCa
Pointer to SEH_Record
ntdll.wcstombs+70

return to ntdll.RtlCa
Pointer to SEH_Record
ntdll.wcstombs+70

return to ntdll.RtlCa

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused laboratoratmclar.exe: 00465CC0 -> 00465CC0 (0x00000001 bytes) Time Wasted Debugging: 0:00:32:04

Selectăm parola în *dump* pentru editare:

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	
Address	Hex			ASCII	
00465C90	6F 00 72 00	65 00 63 00	74 00 61 00	20 00 21 00	o.r.e.c.t.a. !.
00465CA0	20 00 4D 00	61 00 69 00	20 00 69 00	6E 00 63 00	.M.a.i.i.n.c.
00465CB0	65 00 61 00	72 00 63 00	61 00 00 00	0E 00 00 00	e.a.r.c.a.....
00465CC0	61 00 74 00	6D 00 32 00	30 00 32 00	34 00 00 00	a.t.m.2.0.2.4..
00465CD0	56 42 41 36	2E 44 4C 4C	00 00 00 00	5F 5F 76 62	BA6.DLL....__vb
00465CE0	61 53 74 72	43 6F 70 79	00 00 00 00	5F 5F 76 62	FreeVarList....__vb
00465CF0	61 46 72 65	65 56 61 72	4C 69 73 74	00 00 00 00	FreeVarDup....__vb
00465D00	5F 5F 76 62	61 56 61 72	44 75 70 00	5F 5F 76 62	FreeVarDup....__vb
00465D10	61 46 72 65	65 4F 62 6A	00 00 00 00	5F 5F 76 62	aFreeObj....__vb
00465D20	61 46 72 65	65 53 74 72	00 00 00 00	5F 5F 76 62	aFreeStr....__vb
00465D30	61 48 72 65	73 75 6C 74	43 68 65 63	6B 4F 62 6A	aHresultcheckObj
00465D40	00 00 00 00	5F 5F 76 62	61 4F 62 6A	53 65 74 00	__vbaObjSet.
00465D50	5F 5F 76 62	61 53 74 72	43 6D 70 00	01 00 00 00	__vbaStrCmp....
00465D60	64 5A 46 00	00 00 00 00	78 60 46 00	FF FF FF FF	dZF....x'F.yyyy
00465D70	00 00 00 00	R8 5A 46 00	08 70 46 00	00 00 00 00	ZF..nF....

Edit data at 00465CC0

Hex String Copy data

ASCII

a t m 2 0 2 4

UNICODE:

atm2024

UTF-8

a t m 2 0 2 4

Codepage...

Hex:

61 00 74 00 6D 00 32 00 30 00 32 00 34 00

Keep Size

OK Cancel

Edităm secțiunea selectată în *dump* în modul binar:

LaboratorATMClar.exe - PID: 12244 - Module: laboratoratmclar.exe - Thread: Main Thread 9832 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

Hide FPU

EAX 00000000
EBX 00000000
ECX 48890000
EDX 00000000
EBP 0019FA44
ESP 0019FA18
ESI 777669FC
EDI 77766A78
EIP 77811EE3
FPU AGS 00000246

00466330 BA C05C4600 mov edx, laboratoratmclar.465CC0
00466335 8D4E 34 lea ecx, dword ptr ds:[esi+34]
00466338 FF15 68104000 call dword ptr ds:[_vbaStrCopy>]
0046633E C745 FC 00000000 mov dword ptr ss:[ebp-4], 0
00466345 8845 08 mov eax, dword ptr ss:[ebp+8]
00466348 50 push eax
00466349 8B10 mov edx, dword ptr ds:[eax]
0046634B FF52 08 call dword ptr ds:[edx+8]
0046634E 8B45 FC mov eax, dword ptr ss:[ebp-4]
00466351 884D EC mov ecx, dword ptr ss:[ebp-14]
00466354 5F pop edi
00466355 5E pop esi
00466356 64:890D 00000000 mov dword ptr [0], ecx
0046635D 5B pop ebx
0046635E 8BE5 mov esp, ebp
00466360 5D pop edi
00466361 C2 0400 ret 4
00466364 90
00466365 90
00466366 90
00466367 90
00466368 90
00466369 90
0046636A 90
0046636B 90
0046636C 90
0046636D 90
0046636E 90
0046636F 90
00466370 9E
00466371 9E
00466372 9E
00466373 9E
00466374 9C
00466375 6306
00466376 00FF
00466377 FF
00466378 FF
00466379 FF
0046637A FF
0046637B FF
0046637C FF

00465CC0 L"atm2024"

.text:00466330 laboratoratmclar.exe:\$66330 #66330

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	Hex	Hex	Hex	ASCII
00465C90	6F 00 72 00	65 00 63 00	74 00 61 00	20 00 21 00	o.r.e.
00465CA0	20 00 4D 00	61 00 69 00	20 00 69 00	6E 00 63 00	.M.a.i.
00465CB0	65 00 61 00	72 00 63 00	61 00 00 00	0E 00 00 00	e.a.r.c.
00465CC0	61 00 74 00	6D 00 32 00	30 00 32 00	34 00 00 00	a.t.m.2.0.2.4..
00465CD0	56 42 41 36	2E 44 4C 4C	00 00 00 00	5F 5F 76 62	BA6.DLL.....vb
00465CE0	61 53 74 72	43 6F 70 79	00 00 00 00	5F 5F 76 62	FreeVarList....vb
00465CF0	61 46 72 65	65 56 61 72	4C 69 73 74	00 00 00 00	FreeVarDup....vb
00465D00	5F 5F 76 62	61 56 61 72	44 75 70 00	5F 5F 76 62	FreeVarDup....vb
00465D10	61 46 72 65	65 4F 62 6A	00 00 00 00	5F 5F 76 62	aFreeObj.....vb
00465D20	61 46 72 65	65 53 74 72	00 00 00 00	5F 5F 76 62	aFreeStr.....vb
00465D30	61 48 72 65	73 75 6C 74	43 68 65 63	6B 4F 62 6A	aHresultcheckObj
00465D40	00 00 00 00	5F 5F 76 62	61 4F 62 6A	53 65 74 00	vbaObjSet.
00465D50	5F 5F 76 62	61 53 74 72	43 6D 70 00	01 00 00 00	vbaStrCmp.
00465D60	64 5A 46 00	00 00 00 00	78 60 46 00	FF FF FF FF	dZF.....x'F.yyyy
00465D70	00 00 00 00	R8 5A 46 00	08 70 46 00	00 00 00 00	ZF..nF.....

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused laboratoratmclar.exe: 00465CC0 -> 00465CCD (0x0000000E bytes)

Time Wasted Debugging: 0:00:34:36

UTF-16 codifică fiecare caracter pe cel puțin 2 octeți (bytes), chiar și atunci când caracterul este ASCII pur ("a.t.m.2.0.2.4").

EDITARE PAROLĂ

MENȚINEREA DIMENSIUNII ESTE IMPORTANTĂ

Edit data at 00465CC0

Hex String Copy data

❌ ASCII

atm2024

UNICODE:

atm2024

UTF-8 Codepage...

atm2024

Hex:

61 00 74 00 6D 00 32 00 30 00 32 00 34 00

☒ Keep Size

OK Cancel

1

Edit data at 00465CC0

Hex String Copy data

❌ ASCII

savantl

UNICODE:

savantl

UTF-8 Codepage...

savantl

Hex:

73 00 61 00 76 00 61 00 6E 00 74 00 31 00

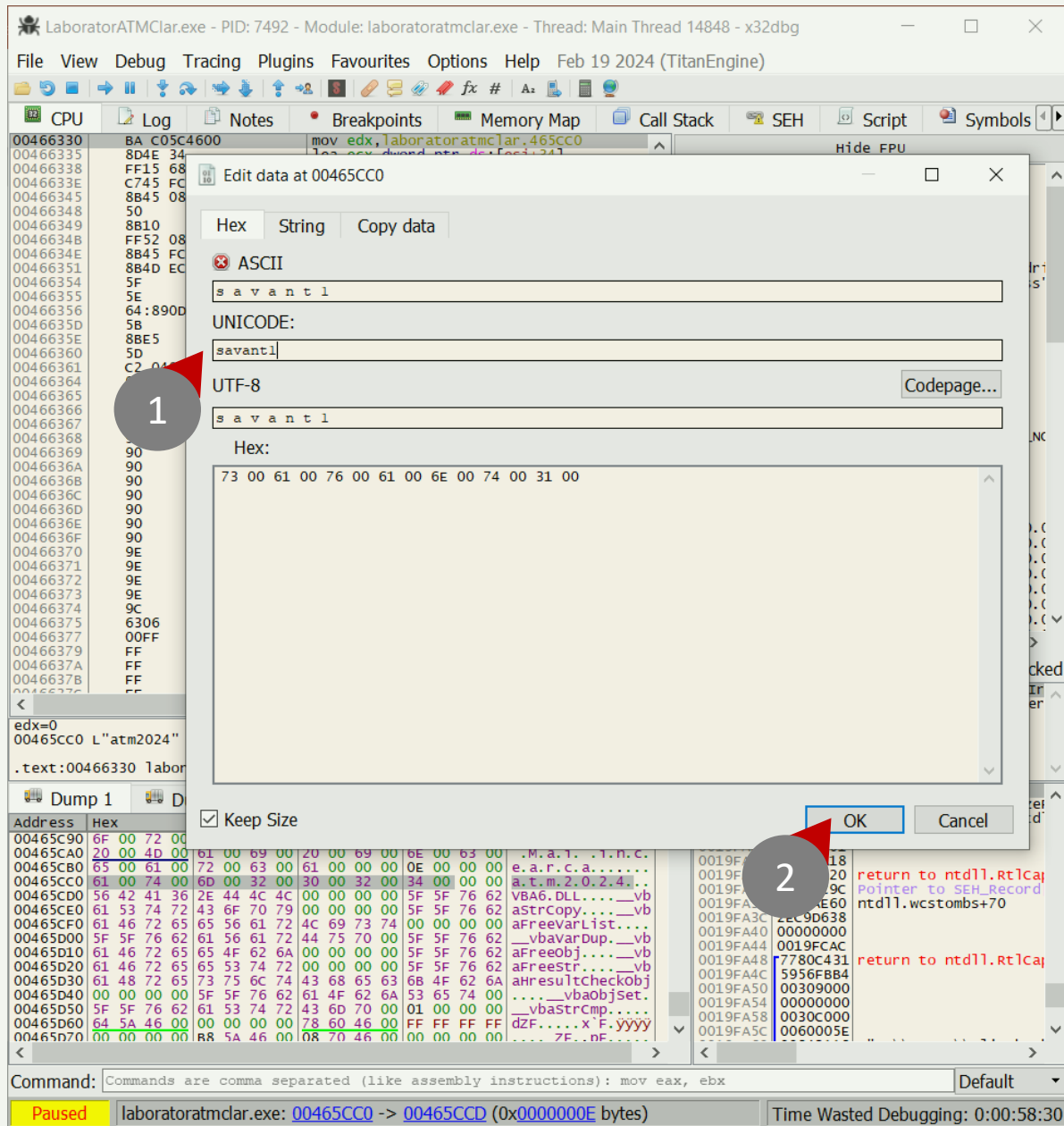
☒ Keep Size

OK Cancel

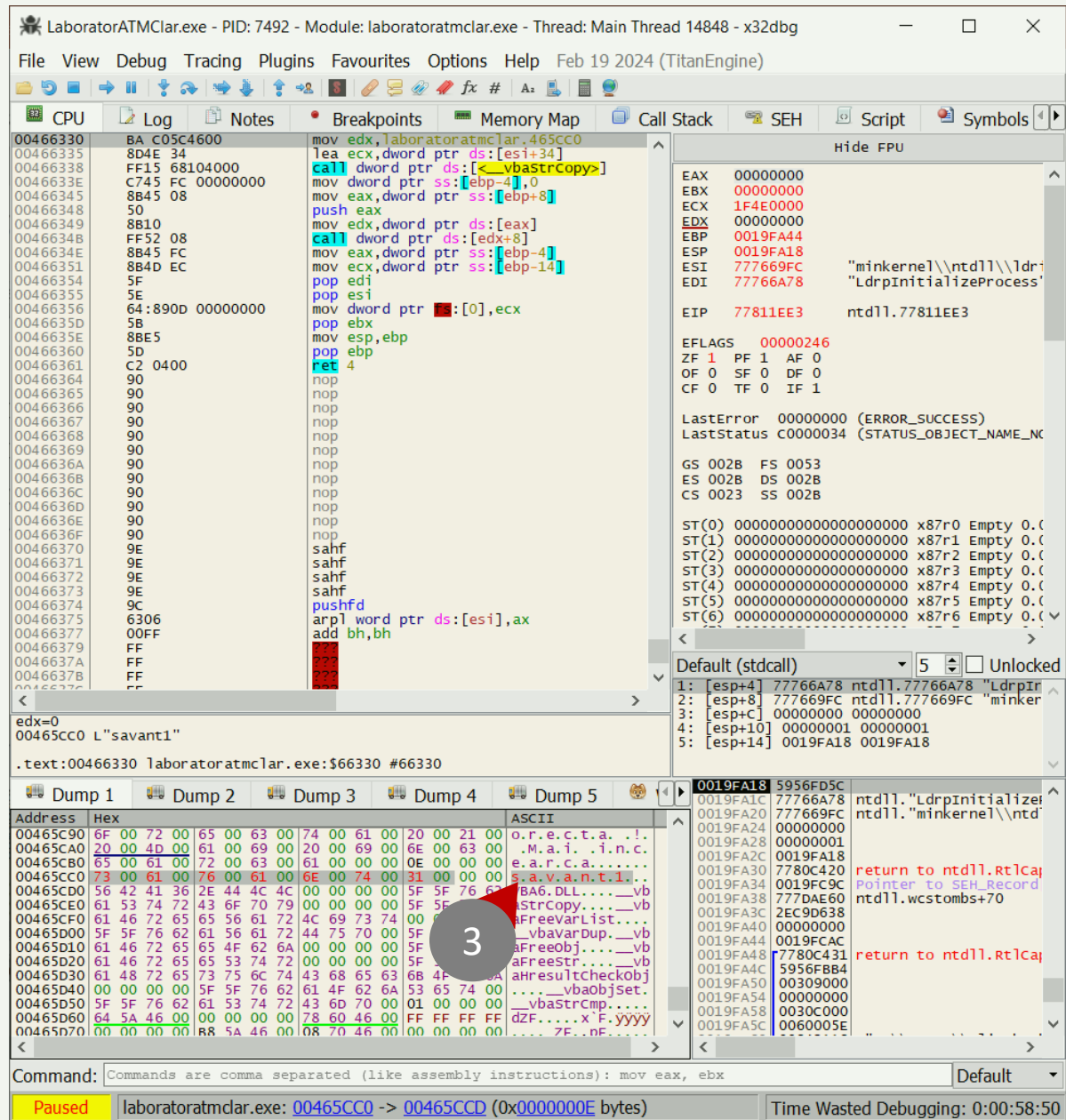
2

3

Schimbăm parola:



Observăm modificările salvate în *dump*:



Scriem noua versiune a executabilului:

LaboratorATMClar.exe - PID: 7492 - Module: laboratoratmclar.exe - Thread: Main Thread 14848 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

Open F3
Recent Files
Attach Alt+A
Detach Ctrl+Alt+F2
Database
Patch file... Ctrl+P
Change End Line
Restore
Exit Alt+X

1

0046635E 8BE5 mov esp,ebp
00466360 5D pop ebp
00466361 C2 0400 ret 4
00466364 90 nop
00466365 90 nop
00466366 90 nop
00466367 90 nop
00466368 90 nop
00466369 90 nop
0046636A 90 nop
0046636B 90 nop
0046636C 90 nop
0046636D 90 nop
0046636E 90 nop
0046636F 90 nop
00466370 9E sahf
00466371 9E sahf
00466372 9E sahf
00466373 9E sahf
00466374 9C pushfd
00466375 6306 arpl word ptr ds:[esi],ax
00466377 00FF add bh,bh
00466379 FF
0046637A FF
0046637B FF

EDX=0
00465CC0 L"savant1"

.text:00466330 laboratoratmclar.exe:\$66330 #66330

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
00465C90	6F 00 72 00 65 00 63 00 74 00 61 00 20 00 21 00	o.r.e.c.t.a. !.
00465CA0	20 00 4D 00 61 00 69 00 20 00 69 00 6E 00 63 00	.M.a.i. !.n.c.
00465CB0	65 00 61 00 72 00 63 00 61 00 00 00 0E 00 00 00	e.a.r.c.a.....
00465CC0	73 00 61 00 76 00 61 00 6E 00 74 00 31 00 00 00	S.a.v.a.n.t.i..
00465CD0	56 42 41 36 2E 44 4C 4C 00 00 00 00 5F 5F 76 62	VBA6.DLL....vb
00465CE0	61 53 74 72 43 6F 70 79 00 00 00 00 5F 5F 76 62	astrCopy....vb
00465CF0	61 46 72 65 65 56 61 72 4C 69 73 74 00 00 00 00	aFreeVarList....vb
00465D00	5F 5F 76 62 61 56 61 72 44 75 70 00 5F 5F 76 62	aFreeVarDup....vb
00465D10	61 46 72 65 65 4F 62 6A 00 00 00 00 5F 5F 76 62	aFreeObj....vb
00465D20	61 46 72 65 65 53 74 72 00 00 00 00 5F 5F 76 62	aFreeStr....vb
00465D30	61 48 72 65 73 75 6C 74 43 68 65 63 6B 4F 62 6A	ahResultCheckObj
00465D40	00 00 00 00 5F 5F 76 62 61 4F 62 6A 53 65 74 00	vbaObjSet.
00465D50	5F 5F 76 62 61 53 74 72 43 6D 70 01 00 00 00	vbaStrcmp....
00465D60	64 5A 46 00 00 00 00 78 60 46 00 FF FF FF FF	dZF.....x'F.yyyy
00465D70	00 00 00 00 RR 5A 46 00 08 70 46 00 00 00 00 00	ZF..nF.....

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused Open the patch dialog. Time Wasted Debugging: 0:00:59:32

LaboratorATMClar.exe - PID: 7492 - Module: laboratoratmclar.exe - Thread: Main Thread 14848 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00466330 BA C05C4600 mov edx,laboratoratmclar.465CC0
00466338 804E 34 lea ecx,dword ptr ds:[esi+34]
00466338 FF15 68104000 call dword ptr ds:[<vbaStrCopy>]
0046633E C745 FC 00000000 mov dword ptr ss:[ebp-4],0
00466345 8B45 08 mov eax,dword ptr ss:[ebp+8]
00466348 50 push eax
00466349 mov edx,dword ptr ds:[eax]
0046634B FF52 08 call dword ptr ds:[edx+8]
0046634E 8B45 FC mov eax,dword ptr ss:[ebp-4]
00466351 8B4D EC mov ecx,dword ptr ss:[ebp-14]
00466354 5F pop edi
00466355 5E pop esi
00466356 64:890D 00000000
0046635D 5B
0046635E 8BE5
00466360 5D
00466361 C2 0400
00466364 90
00466365 90
00466366 90
00466367 90
00466368 90
00466369 90
0046636A 90
0046636B 90
0046636C 90
0046636D 90
0046636E 90
0046636F 90
00466370 9E
00466371 9E
00466372 9E
00466373 9E
00466374 9C
00466375 6306
00466377 00FF
00466379 FF
0046637A FF
0046637B FF

EDX=0
00465CC0 L"savant1"

.text:00466330 laboratoratmclar.exe:\$66330 #66330

Dump 1 Dump 2

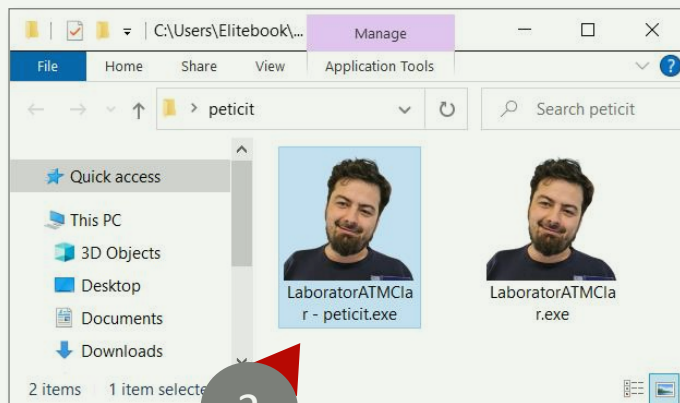
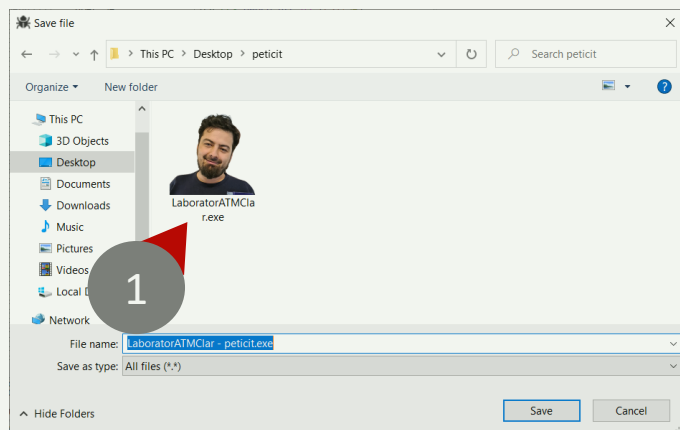
Address	Hex	ASCII
00465C90	6F 00 72 00 65 00 63 00 74 00 61 00 20 00 21 00	o.r.e.c.t.a. !.
00465CA0	20 00 4D 00 61 00 69 00 20 00 69 00 6E 00 63 00	.M.a.i. !.n.c.
00465CB0	65 00 61 00 72 00 63 00 61 00 00 00 0E 00 00 00	e.a.r.c.a.....
00465CC0	73 00 61 00 76 00 61 00 6E 00 74 00 31 00 00 00	S.a.v.a.n.t.i..
00465CD0	56 42 41 36 2E 44 4C 4C 00 00 00 00 5F 5F 76 62	VBA6.DLL....vb
00465CE0	61 53 74 72 43 6F 70 79 00 00 00 00 5F 5F 76 62	astrCopy....vb
00465CF0	61 46 72 65 65 56 61 72 4C 69 73 74 00 00 00 00	aFreeVarList....vb
00465D00	5F 5F 76 62 61 56 61 72 44 75 70 00 5F 5F 76 62	aFreeVarDup....vb
00465D10	61 46 72 65 65 4F 62 6A 00 00 00 00 5F 5F 76 62	aFreeObj....vb
00465D20	61 46 72 65 65 53 74 72 00 00 00 00 5F 5F 76 62	aFreeStr....vb
00465D30	61 48 72 65 73 75 6C 74 43 68 65 63 6B 4F 62 6A	ahResultCheckObj
00465D40	00 00 00 00 5F 5F 76 62 61 4F 62 6A 53 65 74 00	vbaObjSet.
00465D50	5F 5F 76 62 61 53 74 72 43 6D 70 01 00 00 00	vbaStrcmp....
00465D60	64 5A 46 00 00 00 00 78 60 46 00 FF FF FF FF	dZF.....x'F.yyyy
00465D70	00 00 00 00 RR 5A 46 00 08 70 46 00 00 00 00 00	ZF..nF.....

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

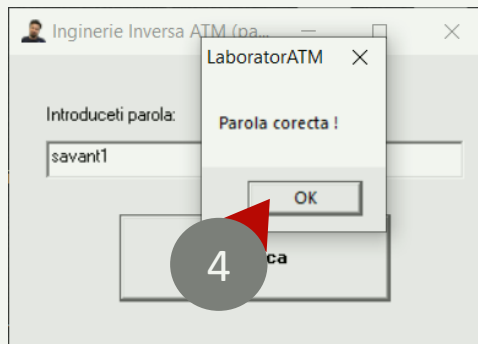
Paused laboratoratmclar.exe: 00465CC0 -> 00465CCD (0x0000000E bytes) Time Wasted Debugging: 0:01:00:19

PAROLĂ SCHIMBATĂ

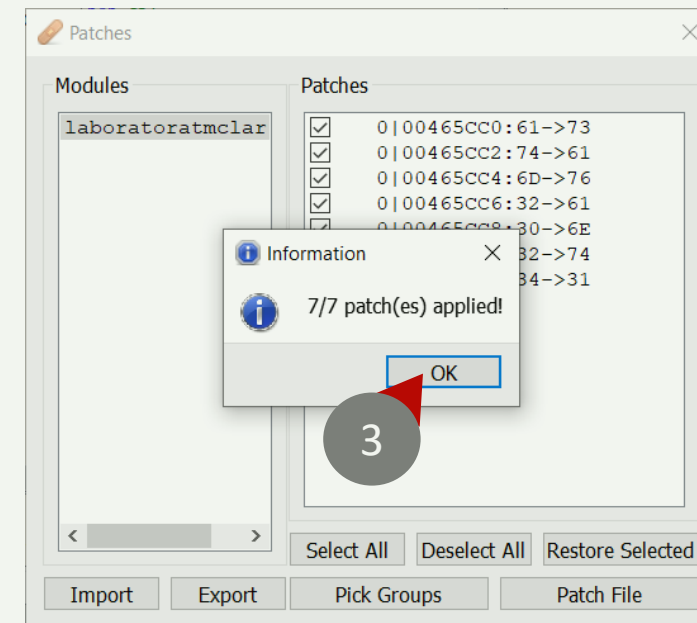
TESTĂM NOUL EXECUTABIL



Noua parola functioneaza !



Cate modificari?

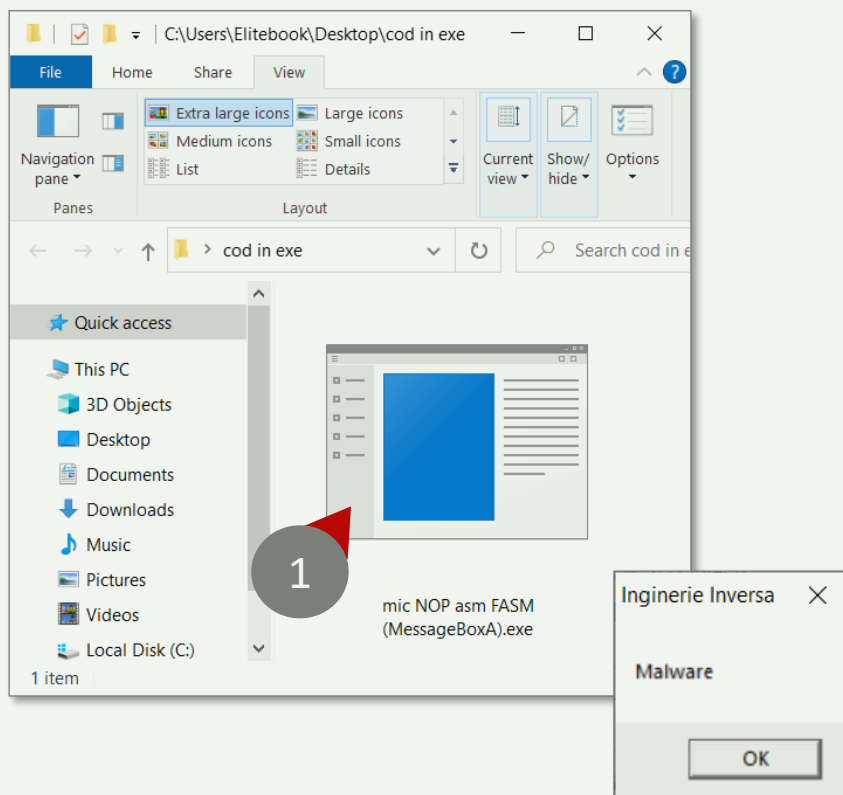


C.8.2 ADĂUGAREA DE **COD MASINĂ**



“PEȘTERA ÎN COD” (CODE CAVE)

SAU CUM SĂ ADĂUGAȚI COD MAȘINĂ IN EXECUTABILE



Cum să adăugați cod mașină în fișiere executabile PE?

Pentru a adăuga cod mașină:

- Trebuie să găsim o zonă din executabil care nu este folosită!
- Trebuie să eliminăm o parte din codul original fără a afecta „prea mult” funcționalitatea!

Compilarea unui executabil în FASM (din nou)

	<pre>format PE GUI 4.0 ; specifică formatul executabilului ca fiind PE (Portable Executable) pentru interfața grafică Windows, versiunea 4.0 entry start ; definește punctul de intrare al programului, unde execuția va începe</pre>
	<pre>include 'INCLUDE/win32a.inc' ; include fișierul de antet 'win32a.inc' care conține macro-uri și definiții pentru interfața cu API-ul Windows</pre>
.data	<pre>section '.data' data readable writeable ; începe o secțiune de date care poate fi citită și scrisă title db 'Inginerie Inversa',0 ; definește un șir de caractere pentru titlu, terminat cu null (0) pentru a fi folosit în mesaj message db 'Malware',0 ; definește un șir de caractere pentru mesaj, terminat cu null (0)</pre>
.text	<pre>section '.text' code readable executable ; începe secțiunea de cod, care poate fi citită și executată start: ; eticheta 'start', care este punctul de intrare al programului NOP ; introducem instrucțiuni NOP (No Operation), atatea câte dorim pentru exemplificare NOP NOP push 0 ; pune valoarea 0 pe stivă, reprezentând handle-ul pentru fereastra părinte (niciuna în acest caz) push title ; pune adresa șirului de titlu pe stivă push message ; pune adresa șirului de mesaj pe stivă push 0 ; pune valoarea 0 pe stivă, care reprezintă stilul cutiei de mesaj (MB_OK) call [MessageBoxA] ; apelează funcția MessageBoxA din user32.dll push 0 ; pune valoarea 0 pe stivă, argument pentru ExitProcess care indică statusul de ieșire call [ExitProcess] ; apelează funcția ExitProcess din kernel32.dll pentru a încheia programul</pre>
.idata	<pre>section '.idata' import data readable writeable ; începe secțiunea de date pentru importuri, care poate fi citită și scrisă library kernel32,'KERNEL32.DLL',\ ; specifică că vom folosi funcții din KERNEL32.DLL user32,'USER32.DLL' ; și din USER32.DLL import kernel32,\ ; începe definițiile de import pentru kernel32.dll ExitProcess,'ExitProcess' ; specifică că dorim să folosim funcția ExitProcess din acest DLL import user32,\ ; începe definițiile de import pentru user32.dll MessageBoxA,'MessageBoxA' ; specifică că dorim să folosim funcția MessageBoxA din acest DLL</pre>

Dezasamblam executabilul:

test.exe - PID: 15232 - Module: ntdll.dll - Thread: Main Thread 15824 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

77811EE3 EB 07 jmp ntdll.77811EE3

77811EE5 33C0 xor eax,esi

77811EE7 40 inc eax

77811EE8 C3 ret

77811EE9 8B65 E8 mov esp,ebp ss:[ebp-18]

77811EEC C745 FC FEFFFFFF mov dword ptr [ebp-4],FFFFFFFE

77811EF3 8B4D F0 mov ecx,dword ptr ss:[ebp-10]

77811EF6 64:890D 00000000 mov dword ptr [0],ecx

77811EFD 59 pop ecx

77811EFE 5F pop edi

77811EFF 5E pop esi

77811F00 5B pop ebx

77811F01 C9 leave

77811F02 C3 ret

77811F03 64:A1 30000000 mov eax,dword ptr [0:30]

77811F09 33C9 xor ecx,ecx

77811F0B 890D D4678877 mov dword ptr ds:[778867D4],ecx

77811F11 890D D8678877 mov dword ptr ds:[778867D8],ecx

77811F17 8B08 mov byte ptr ds:[eax],cl

77811F19 3848 02 cmp byte ptr ds:[eax+2],cl

77811F1C 74 05 je ntdll.77811F23

77811F1E E8 94FFFFFF call ntdll.77811EB7

77811F23 33C0 xor eax,eax

77811F25 C3 ret

77811F26 8BFF mov edi,edi

77811F28 55 push ebp

77811F29 8BEC mov ebp,esp

77811F2B 83E4 F8 and esp,FFFFFFF8

77811F2E 81EC 70010000 sub esp,170

77811F34 A1 70B38877 mov eax,dword ptr ds:[7788B370]

77811F39 33C4 xor eax,esp

77811F3B 898424 6C010000 mov dword ptr ss:[esp+16C],eax

77811F42 56 push esi

77811F43 8B35 FC918877 mov esi,dword ptr ds:[778891FC]

77811F49 57 push edi

77811F4A 6A 16 push 16

77811F4C 58 pop eax

77811F4D 66:894424 10 mov word ptr ss:[esp+10],ax

77811F52 8BF9 mov edi,ecx

77811F54 6A 18 push 18

77811F56 58 pop eax

77811F57 66:894424 12 mov word ptr ss:[esp+12],ax

77811F5C 8D4424 70 lea eax,dword ptr ss:[esp+70]

77811F60 894424 6C mov dword ptr ss:[esp+6C],eax

77811F64 33C0 xor eax,eax

77811F66 C74424 14 54477677 mov dword ptr ss:[esp+14],ntdll.77764754:77764754

77811F6E C74424 68 00000001 mov dword ptr ss:[esp+68],10000000

77811F76 66:894424 70 mov word ptr ss:[esp+70],ax

77811F7B 85F6 test esi,esi

77811F7D 74 29 je ntdll.77811FA8

Hide FPU

EAX 00000000

EBX 00000000

ECX BAA20000

EDX 00000000

EBP 000DFA44

ESP 000DFA18

ESI 777669FC "minkernel\ntdll\

EDI 77766A78 "LdrpInitializeProc

EIP 77811EE3 ntdll.77811EE3

EFLAGS 00000246

ZF 1 PF 1 AF 0

OF 0 SF 0 DF 0

CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)

LastStatus C0000034 (STATUS_OBJECT_NAME_NOT_FOUND)

GS 002B FS 0053

ES 002B DS 002B

CS 0023 SS 002B

ST(0) 00000000000000000000000000000000 x87r0 Empty

ST(1) 00000000000000000000000000000000 x87r1 Empty

ST(2) 00000000000000000000000000000000 x87r2 Empty

ST(3) 00000000000000000000000000000000 x87r3 Empty

ST(4) 00000000000000000000000000000000 x87r4 Empty

ST(5) 00000000000000000000000000000000 x87r5 Empty

ST(6) 00000000000000000000000000000000 x87r6 Empty

ST(7) 00000000000000000000000000000000 x87r7 Empty

x87Tagword FFFF

x87TW_0 3 (Empty) x87TW_1 3 (Empty)

x87TW_2 3 (Empty) x87TW_3 3 (Empty)

x87TW_4 3 (Empty) x87TW_5 3 (Empty)

x87TW_6 3 (Empty) x87TW_7 3 (Empty)

Default (stdcall) 5 Unlocked

1: [esp+4] 77766A78 ntdll.77766A78 "LdrpInitializeProc"

2: [esp+8] 777669FC ntdll.777669FC "minkernel\ntdll\

3: [esp+C] 00000000 00000000

4: [esp+10] 00000001 00000001

5: [esp+14] 000DFA18 000DFA18

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address Hex ASCII

77761000 16 00 18 00 F0 7D 76 77 14 00 16 00 50 7C 76 77}vw....P|vw

77761010 00 00 02 00 0C 5E 76 77 0E 00 10 00 C8 7F 76 77^vw....E.vw

77761020 0C 00 0E 00 B8 7F 76 77 08 00 0A 00 88 7B 76 77vw....{vw

77761030 06 00 08 00 98 7F 76 77 06 00 08 00 A8 7F 76 77vw....}vw

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused Show this address in memory map view. Equivalent command "memmapdump ad Time Wasted Debugging: 0:03:17:43



test.exe - PID: 15232 - Module: ntdll.dll - Thread: Main Thread 15824 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

Address	Size	Party	Info	Content
00010000	00001000	User	Info	
00020000	00001000	User		
00030000	00001000	User		
00040000	00001000	User		
00060000	00035000	User	Reserved	
00095000	00008000	User		
000A0000	0003A000	User	Reserved	
000DA000	00006000	User	stack (15824)	
000E0000	00004000	User		
000F0000	00002000	User		
00100000	00001000	User		
00110000	00001000	User		
00120000	00001000	User		
00130000	00001000	User		
00140000	00035000	User		
00175000	00008000	User		
00180000	00035000	User		
001B5000	00008000	User		
001F0000	00007000	User		
001F7000	00009000	User		
00200000	00092000	User		
00292000	00008000	User	TEB, TEB (15824), wow64 TEB (15824), TEB (18204), wow64 TEB (18204)	
0029D000	00163000	User	Reserved (00200000)	
00400000	00001000	User	test.exe	
00401000	00001000	User	".data"	
00402000	00001000	User	".text"	
00403000	00001000	User	".idata"	
00410000	0000C9000	User	\Device\HarddiskVolume3\windows\System32\locale.nls	
00540000	0000F000	User	Heap (ID 0)	
0054F000	00001000	User	Reserved	
00550000	000FD000	User	Reserved	
0064D000	00003000	User	Stack (18204)	
00650000	000FD000	User	Reserved	
0074D000	00003000	User	Reserved	
75570000	00001000	System	apphelp.dll	
75571000	00007D000	System	".text"	
755EE000	00002000	System	".data"	
755F0000	00003000	System	".idata"	
755F3000	00017000	System	".rsrsc"	
7560A000	00006000	System	".reloc"	
75810000	00001000	System	win32u.dll	
75811000	00006000	System	".text"	
75817000	0000E000	System	".data"	
75825000	00001000	System	".data"	
75826000	00001000	System	".rsrsc"	
75827000	00001000	System	".reloc"	
75D40000	00001000	System	kernelbase.dll	
75D41000	001DE000	System	".text"	
75F1F000	00006000	System	".data"	
75F23000	00006000	System	".idata"	
75F29000	00001000	System	".didat"	
75F2A000	00001000	System	".rsrsc"	
75F2B000	00001000	System	".reloc"	
75F28000	00031000	System		
766E0000	00001000	System	msvcrt.dll	
766E1000	0000E000	System	".text"	
7674F000	00003000	System	".data"	
76752000	00002000	System	".idata"	
76754000	00001000	System	".didat"	
76755000	00001000	System	".rsrsc"	
76756000	00005000	System	".reloc"	
76760000	00001000	System	user32.dll	
76761000	000A5000	System	".text"	
76806000	00004000	System	".data"	

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Default

Paused Show this address in memory map view. Equivalent command "memmapdump ad Time Wasted Debugging: 0:03:18:24

Suntem îndrumați la adresa punctului de intrare (real):

test.exe - PID: 15232 - Module: test.exe - Thread: Main Thread 15824 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

Hide FPU

EAX 00000000
EBX 00000000
ECX BAA20000
EDX 00000000
EBP 000DFA44
ESP 000DFA18
ESI 777669FC
EDI 77766A78
EIP 77811EE3 ntdll.77811EE3

EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C0000034 (STATUS_OBJECT_NAME_NOT_FOUND)
GS 002B FS 0053

00402000 90 nop
00402001 90 nop
00402002 90 nop
00402003 90 nop
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test.401000
0040200D 68 12104000 push test.401012
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBox>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
00402030 0000 add byte ptr ds:[eax],al
00402032 0000 add byte ptr ds:[eax],al
00402034 0000 add byte ptr ds:[eax],al
00402036 0000 add byte ptr ds:[eax],al
00402038 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al

Spațiu liber umplut cu instrucțiuni
ADD [EAX], AL (opcode 0x00 0x00)
sau pur și simplu zero (00).

Acest spațiu apare imediat după
finalul efectiv al codului (call
dword ptr ds:[ExitProcess]).

.text:00402000 test.exe:\$2000 #400 <optionalHeader.AddressOfEntryPoint>

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address Hex ASCII

77761000 16 00 18 00 F0 7D 76 77 14 00 16 00 50 7C 76 770}vw...Pvw
77761010 00 00 02 00 0C 5E 76 77 0E 00 10 00 C8 7F 76 77Avw...E.vw
77761020 0C 00 0E 00 B8 7F 76 77 08 00 0A 00 88 7B 76 77vw...{vw
77761030 06 00 08 00 98 7F 76 77 06 00 08 00 A8 7F 76 77vw...vw

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused Show this address in memory map view. Equivalent command "memmapdump ad Time Wasted Debugging: 0:03:18:46


```
Command Prompt
Microsoft Windows [Version 10.0.19045.5608]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Paul>cd desktop

C:\Users\Paul\Desktop>dumpbin /headers m.exe
Microsoft (R) COFF Binary File Dumper Version 6.00.8168
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.

Dump of file m.exe

PE signature found

File Type: EXECUTABLE IMAGE

FILE HEADER VALUES
 8664 machine (Unknown)
   5 number of sections
65F34149 time date stamp Thu Mar 14 20:26:17 2024
   0 file pointer to symbol table
   0 number of symbols
F0 size of optional header
22E characteristics
    Executable
    Line numbers stripped
    Symbols stripped
    Application can handle large (>2GB) addresses
    Debug information stripped
```

dumpbin /headers m.exe

Mai sus se prezintă utilizarea comenzii dumpbin /headers aplicată asupra unui fișier executabil (m.exe), prin care sunt afișate informații detaliate despre structura internă a fișierului PE (Portable Executable). Rezultatul include antetul fișierului (File Header), antetul opțional (Optional Header) și anteturile secțiunilor (.text, .rdata, .pdata, .xdata, .idata). Aceste informații arată tipul fișierului, arhitectura țintă, dimensiuni ale codului și datelor, punctul de intrare, precum și caracteristicile fiecărei secțiuni, fiind utile pentru analiza statică a executabilelor și pentru înțelegerea modului în care sunt organizate datele și instrucțiunile în memorie.

Microsoft Windows [Version 10.0.19045.5608]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Paul>cd desktop

C:\Users\Paul\Desktop>dumpbin /headers m.exe
Microsoft (R) COFF Binary File Dumper Version 6.00.8168
Copyright (C) Microsoft Corp 1992-1998. All rights reserved.

Dump of file m.exe

PE signature found

File Type: EXECUTABLE IMAGE

FILE HEADER VALUES
8664 machine (Unknown)
5 number of sections
65F34149 time date stamp Thu Mar 14 20:26:17 2024
0 file pointer to symbol table
0 number of symbols
F0 size of optional header
22E characteristics
Executable
Line numbers stripped
Symbols stripped
Application can handle large (>2GB) addresses
Debug information stripped

OPTIONAL HEADER VALUES
20B magic #
2.36 linker version
200 size of code
800 size of initialized data
0 size of uninitialized data
1000 RVA of entry point
1000 base of code

SECTION HEADER #1
.text name
60 virtual size
1000 virtual address
200 size of raw data
400 file pointer to raw data
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
60500020 flags
Code
RESERVED - UNKNOWN
RESERVED - UNKNOWN
Execute Read

SECTION HEADER #2
.rdata name
50 virtual size
2000 virtual address
200 size of raw data
600 file pointer to raw data
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
40500040 flags
Initialized Data
RESERVED - UNKNOWN
RESERVED - UNKNOWN
Read Only

SECTION HEADER #3
.pdata name
C virtual size
3000 virtual address
200 size of raw data
800 file pointer to raw data
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
40300040 flags
Initialized Data
RESERVED - UNKNOWN
RESERVED - UNKNOWN
Read Only

SECTION HEADER #4
.xdata name
8 virtual size
4000 virtual address
200 size of raw data
A00 file pointer to raw data
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
40300040 flags
Initialized Data
RESERVED - UNKNOWN
RESERVED - UNKNOWN
Read Only

SECTION HEADER #5
.idata name
68 virtual size
5000 virtual address
200 size of raw data
C00 file pointer to raw data
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
C0300040 flags
Initialized Data
RESERVED - UNKNOWN
RESERVED - UNKNOWN
Read Write

Summary

68 .idata
C .pdata
50 .rdata
60 .text
8 .xdata

De ce există spațiu liber după cod în .text?

- Formatul PE cere ca fiecare secțiune (.text, .data, .rdata etc.) să fie aliniată la o dimensiune multiplă de *SectionAlignment* (RAM - implicit 4096 bytes) și *FileAlignment* (DISC - implicit 512 bytes) din headerul PE.
- De exemplu, chiar dacă .text conține doar 0x1A bytes de cod, secțiunea poate fi alocată pe 0x200 sau 0x1000 bytes în fișierul PE sau în memorie.



Dacă .text conține 4097 bytes de cod, ce dimensiune va avea secțiunea .text în fișierul PE?

- **În fișierul PE (on disk)**, dimensiunea secțiunii este rotunjită la multiplu de **FileAlignment** (implicit 512 bytes). Deci 4097 este rotunjit la 4608 bytes (9×512).
- **În RAM (on load)**, dimensiunea secțiunii este rotunjită la multiplu de **SectionAlignment** (implicit 4096 bytes = 1 pagină de memorie). Deci 4097 este rotunjit la 8192 bytes (2×4096).

Prin urmare:

- În fișier: **4608 bytes**
- În RAM: **8192 bytes**



Facem un spațiu umplut cu instrucțiuni nop

mic NOP asm FASM (MessageBoxA).exe - PID: 17384 - Module: mic nop asm fasm (messageboxa).exe - Thread: ...

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 90 nop
00402003 90 nop
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push mic nop asm fasm (messageboxa).401
00402009 68 12104000 push mic nop asm fasm (messageboxa).401
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
00402030 0000 add byte ptr ds:[eax],al
00402032 0000 add byte ptr ds:[eax],al
00402034 0000 add byte ptr ds:[eax],al
00402036 0000 add byte ptr ds:[eax],al
00402038 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al

Hide FPU

EAX 00000000
EBX 00000000
ECX 36B70000
EDX 00000000
EBP 000DFA44
ESP 000DFA18
ESI 777669FC
EDI 77766A78

"minkernel\\ntdll\\ldr
"LdrpInitializeProc

EIP 77811EE3 ntdll.77811EE3

EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C0000034 (STATUS_OBJECT_NAME_N

GS 002B FS 0053
ES 002B DS 002B
CS 0023 SS 002B

Binary Edit Ctrl+E
Copy F
Fill...
Fill with NOPs Ctrl+9
Copy Shift+C

Size
Expression: 00000018
Bytes: 18000000
Signed: 24
Unsigned: 24
ASCII:

OK Cancel

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused mic nop asm fasm (messageboxa)

Time Wasted Debugging: 0:02:38:59

Acum am definit o regiune de interes în zona liberă a executabilului:

mic NOP asm FASM (MessageBoxA).exe - PID: 17384 - Module: mic nop asm fasm (messageboxa).exe - Thread: ...

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
00402030 90 nop
00402031 90 nop
00402032 90 nop
00402033 90 nop
00402034 90 nop
00402035 90 nop
00402036 90 nop
00402037 90 nop
00402038 90 nop
00402039 90 nop
0040203A 90 nop
0040203B 90 nop
0040203C 90 nop
0040203D 90 nop
0040203E 90 nop
0040203F 90 nop
00402040 90 nop
00402041 90 nop
00402042 90 nop
00402043 90 nop
00402044 90 nop
00402045 90 nop
00402046 90 nop
00402047 90 nop
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al

Hide FPU

EAX 00000000
EBX 00000000
ECX 36B70000
EDX 00000000
EBP 000DFA44
ESP 000DFA18
ESI 777669FC
EDI 77766A78

"minkernel\\ntdll\\ldr
"LdrpInitializeProc

EIP 77811EE3 ntdll.77811EE3

EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C0000034 (STATUS_OBJECT_NAM

GS 002B FS 0053
ES 002B DS 002B
CS 0023 SS 002B

ST(0) 00000000000000000000000000000000 x87r0 Empty
ST(1) 00000000000000000000000000000000 x87r1 Empty
ST(2) 00000000000000000000000000000000 x87r2 Empty
ST(3) 00000000000000000000000000000000 x87r3 Empty
ST(4) 00000000000000000000000000000000 x87r4 Empty
ST(5) 00000000000000000000000000000000 x87r5 Empty
ST(6) 00000000000000000000000000000000 x87r6 Empty

Default (stdcall) 5 Unlod

1: [esp+4] 77766A78 ntdll.77766A78 "Ld
2: [esp+8] 777669FC ntdll.777669FC "mi
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 000DFA18 000DFA18

.text:00402030 mic nop asm fasm (messageboxa).exe:\$2030 #430

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address Hex ASCII

00401000 49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73 Inginerie Invers
00401010 61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 a.Malware...
00401020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401040 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

000DFA18 9FF3AC2E ntdll."LdrpInitialize
000DFA20 777669FC ntdll."minkernel\\ntd
000DFA24 00000000
000DFA28 00000001
000DFA2C 000DFA18
000DFA30 7780C420
000DFA34 000DFC9C
000DFA38 777DAE60
000DFA3C E878874A
000DFA40 00000000
000DFA44 000DFCAC
000DFA48 7780C431
000DFA4C 9FF3AC6C
000DFA50 0034E000
000DFA54 00000000
000DFA58 00351000
000DFA5C 00920090

return to ntdll.RtlCap
Pointer to SEH_Record
ntdll.wcstombs+70
return to ntdll.RtlCap

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused mic nop asm fasm (messageboxa).exe: 00402030 -> 00402047 (0x00000018) byt Time Wasted Debugging: 0:02:40:54

Selectați regiunea pentru o vizualizare mai bună:

mic NOP asm FASM (MessageBoxA).exe - PID: 17384 - Module: mic nop asm fasm (messageboxa).exe - Thread: ...
— □ ✕

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

Address	Disassembly	Comment
00402019	90	nop
0040201A	FF15 80304000	call dword ptr ds:[<MessageBoxA>]
00402020	6A 00	push 0
00402022	FF15 60304000	call dword ptr ds:[<ExitProcess>]
00402028	0000	add byte ptr ds:[eax],al
0040202A	0000	add byte ptr ds:[eax],al
0040202C	0000	add byte ptr ds:[eax],al
0040202E	0000	add byte ptr ds:[eax],al
00402030	90	nop
00402031	90	nop
00402032	90	nop
00402033	90	nop
00402034	90	nop
00402035	90	nop
00402036	90	nop
00402037	90	nop
00402038	90	nop
00402039	90	nop
0040203A	90	nop
0040203B	90	nop
0040203C	90	nop
0040203D	90	nop
0040203E	90	nop
0040203F	90	nop
00402040	90	nop
00402041	90	nop
00402042	90	nop
00402043	90	nop
00402044	90	nop
00402045	90	nop
00402046	90	nop
00402047	90	nop
00402048	0000	add byte ptr ds:[eax],al
0040204A	0000	add byte ptr ds:[eax],al
0040204C	0000	add byte ptr ds:[eax],al
0040204E	0000	add byte ptr ds:[eax],al
00402050	0000	add byte ptr ds:[eax],al
00402052	0000	add byte ptr ds:[eax],al
00402054	0000	add byte ptr ds:[eax],al
00402056	0000	add byte ptr ds:[eax],al

Hide CPU
⌵

EAX	00000000
EBX	00000000
ECX	36870000
EDX	00000000
EBP	000DFA44
ESP	000DFA18
ESI	777669FC "minkernel\\ntdll\\LdrpInitializeProc
EDI	77766A78 ntdll.77811EE3
EIP	77811EE3 ntdll.77811EE3
EFLAGS	00000246
ZF	1 PF 1 AF 0
OF	0 SF 0 DF 0
CF	0 TF 0 IF 1
LastError	00000000 (ERROR_SUCCESS)
LastStatus	C0000034 (STATUS_OBJECT_NAM
GS 002B	FS 0053
ES 002B	DS 002B
CS 0023	SS 002B
ST(0)	000000000000000000000000 x87r0 Empty
ST(1)	000000000000000000000000 x87r1 Empty
ST(2)	000000000000000000000000 x87r2 Empty
ST(3)	000000000000000000000000 x87r3 Empty
ST(4)	000000000000000000000000 x87r4 Empty
ST(5)	000000000000000000000000 x87r5 Empty
ST(6)	000000000000000000000000 x87r6 Empty

Default (stdcall)
5 Unlocked

1: [esp+4]	77766A78 ntdll.77766A78 "Ld
2: [esp+8]	777669FC ntdll.777669FC "mi
3: [esp+C]	00000000 00000000
4: [esp+10]	00000001 00000001
5: [esp+14]	000DFA18 000DFA18

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5
⌵

Address	Hex	ASCII
00401000	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Ingénierie Invers
00401010	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00	a.Malware.....
00401020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010C0	00 00 00 00 00 00 00 0	

Comentăm punctele de intrare și ieșire din zona delimitată:

[illegible]

Dorim sa ajungem la adresa 00402030 și să revenim la adresa 00402046.

Este copiată adresa punctului de intrare în regiunea definită:

mic NOP asm FASM (MessageBoxA).exe - PID: 17384 - Module: mic nop asm fasm (messageboxa).exe - Thread: ...

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402019 90 nop
0040201A FF15 80304000 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
00402030 90 nop

Binary
Copy
Restore selection
Breakpoint
Follow in Dump
Follow in Memory Map
Graph
Help on mnemonic
Show mnemonic brief
Highlighting mode
Edit columns...
Label
Comment
Toggle Bookmark
Trace coverage
Analysis
Assemble
Patches
Set EIP Here
Create New Thread Here
Go to
Search for
Find references to

Selection
Ctrl+C
Selection to File
Selection (No Bytes)
Selection to File (No Bytes)
Address
Alt+Ins
RVA
File Offset
Header VA
Disassembly

00000000 (ERROR_SUCCESS)
C0000034 (STATUS_OBJECT_NAME_EXISTS)

FS 0053
DS 002B
SS 002B

0000000000000000 x87r0 Empty
0000000000000000 x87r1 Empty
0000000000000000 x87r2 Empty
0000000000000000 x87r3 Empty
0000000000000000 x87r4 Empty
0000000000000000 x87r5 Empty
0000000000000000 x87r6 Empty

Default (stdcall) 5 Unload

1: [esp+4] 77766A78 ntdll.77766A78 "LdrpInitializeProcess
2: [esp+8] 777669FC ntdll.777669FC "minkernel\ntdll.LdrpInitializeProc
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 000DFA18 000DFA18

000DFA18 9FF3AC2E ntdll.LdrpInitialize
000DFA1C 77766A78 ntdll."minkernel\ntd
000DFA20 00000000
000DFA22 00000001
000DFA2C 000DFA18
000DFA30 7780C420 return to ntdll.RtlCa
000DFA34 000DFC9C Pointer to SEH_Record
000DFA38 777DAE60 ntdll.wcstombs+70
000DFA3C E878874A
000DFA40 00000000
000DFA44 000DFCAC
000DFA48 7780C431 return to ntdll.RtlCa
000DFA4C 9FF3AAC6
000DFA50 0034E000
000DFA54 00000000
000DFA58 00351000
000DFA5C 00920090

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:02:51:08

O instrucțiune JMP este inserată spre punctul de intrare în regiune:

mic NOP asm FASM (MessageBoxA).exe - PID: 17384 - Module: mic nop asm fasm (messageboxa).exe - Thread: ...

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 90 nop
00402003 90 nop
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push mi
0040200D 68 12104000 push 0
00402012 90 nop
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dw
00402020 6A 00 push 0
00402022 FF15 60304000 call dw
00402028 0000 add byt
0040202A 0000 add byt
0040202C 0000 add byt
0040202E 0000 add byt
00402030 90 nop
00402031 90 nop
00402032 90 nop
00402033 90 nop
00402034 90 nop
00402035 90 nop
00402036 90 nop
00402037 90 nop
00402038 90 nop
00402039 90 nop
0040203A 90 nop
0040203B 90 nop
0040203C 90 nop
0040203D 90 nop
0040203E 90 nop
0040203F 90 nop
00402040 90 nop

Binary
Copy
Breakpoint
Follow in Dump
Follow in Memory Map
Graph
Help on mnemonic
Show mnemonic brief
Highlighting mode
Edit columns...
Label
Comment
Toggle Bookmark
Trace coverage
Analysis
Assemble
Patches
Set EIP Here
Create New Thread Here
Go to
Search for
Find references to

Space
Ctrl+P
Ctrl+*

00000000
00000000
36870000
00000000
000DFA44
000DFA18
777669FC
77766A78
77811EE3
ntdll.77811EE3
00000246
PF 1 AF 0
SF 0 DF 0
TF 0 IF 1
Error 00000000 (ERROR_SUCCESS)
Status C0000034 (STATUS_OBJECT_NAME_EXISTS)

2B FS 0053
2B DS 002B
23 SS 002B

0000000000000000 x87r0 Empty
0000000000000000 x87r1 Empty
0000000000000000 x87r2 Empty
0000000000000000 x87r3 Empty
0000000000000000 x87r4 Empty
0000000000000000 x87r5 Empty
0000000000000000 x87r6 Empty

Default (stdcall) 5 Unload

1: [esp+4] 77766A78 ntdll.77766A78 "LdrpInitializeProcess
2: [esp+8] 777669FC ntdll.777669FC "minkernel\ntdll.LdrpInitializeProc
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 000DFA18 000DFA18

000DFA18 9FF3AC2E ntdll.LdrpInitialize
000DFA1C 77766A78 ntdll."minkernel\ntd
000DFA20 00000000
000DFA22 00000001
000DFA2C 000DFA18
000DFA30 7780C420 return to ntdll.RtlCa
000DFA34 000DFC9C Pointer to SEH_Record
000DFA38 777DAE60 ntdll.wcstombs+70
000DFA3C E878874A
000DFA40 00000000
000DFA44 000DFCAC
000DFA48 7780C431 return to ntdll.RtlCa
000DFA4C 9FF3AAC6
000DFA50 0034E000
000DFA54 00000000
000DFA58 00351000
000DFA5C 00920090

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:02:52:02

Introduceți instrucțiunea JMP + adresa de unde începe regiunea:

mic NOP asm FASM (MessageBoxA).exe - PID: 17384 - Module: mic nop asm fasm (messageboxa).exe - Thread: ...

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 90 nop
00402003 90 nop
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push mic nop asm fasm (messageboxa).401(40:
0040200D 68 12104000 push mic nop asm fasm (messageboxa).401(40:
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
00402030 90 nop
00402031 90 nop
00402032 90 nop
00402033 90 nop
00402034 90 nop
00402035 90 nop
00402036 90 nop
00402037 90 nop
00402038 90 nop
00402039 90 nop
0040203A 90 nop
0040203B 90 nop
0040203C 90 nop
0040203D 90 nop
0040203E 90 nop
0040203F 90 nop
00402040 90 nop

Assemble at 00402002

jmp 00402030

☐ Keep Size ☒ Fill with NOP's ☒ XEDParse ☐ asmjit OK Cancel

Instruction encoded successfully!
Bytes: EB2C

Default (stdcall) 5 Unload

1: [esp+4] 77766A78 ntdll.77766A78 "Ld
2: [esp+8] 777669FC ntdll.777669FC "mi
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 000DFA18 000DFA18

.text:00402002 mic nop asm fasm (messageboxa).exe:\$2002 #402

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
00401000	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie Invers
00401010	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00	a.Malware.....
00401020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010C0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010D0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:02:54:54

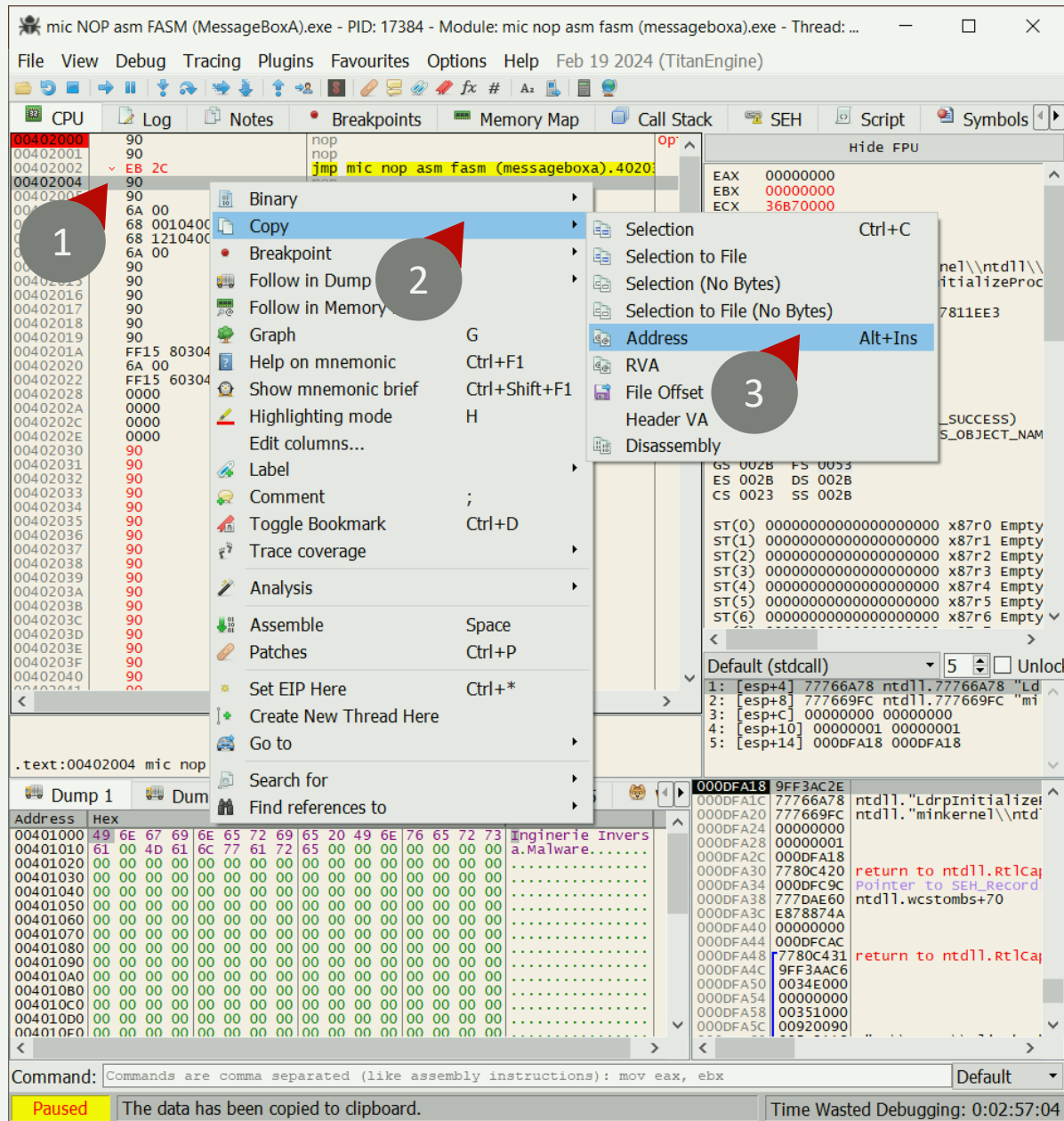
X32dbg permite vizualizarea salturilor cu ajutorul săgeților:

CPU Log Notes Breakpoints Memory Map Call Stack

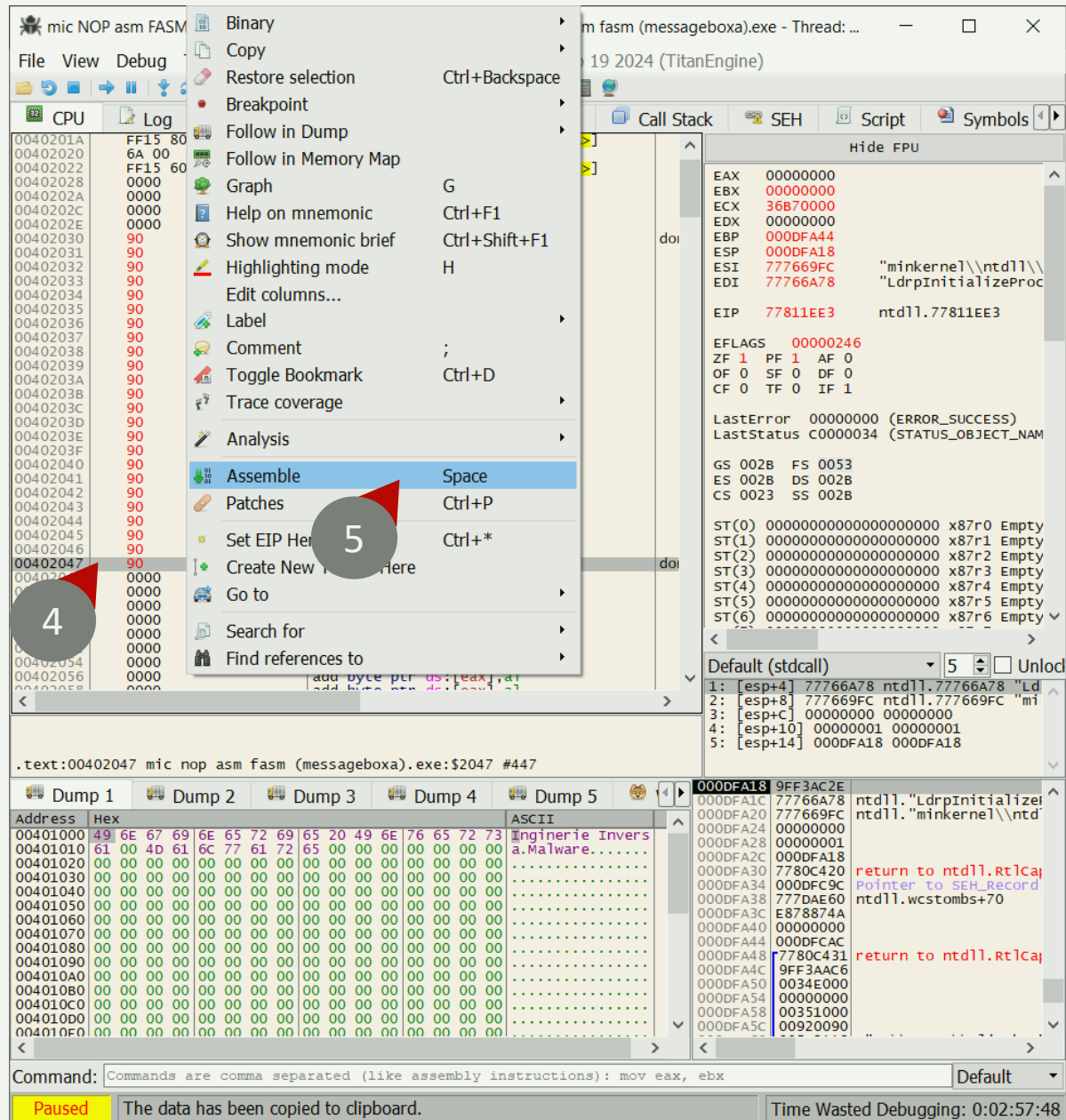
00402000 90 nop
00402001 90 nop
00402002 EB 2C jmp mic nop asm fasm (messageboxa).4020
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push mic nop asm fasm (messageboxa).401(40:
0040200D 68 12104000 push mic nop asm fasm (messageboxa).401(40:
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
00402030 90 nop
00402031 90 nop
00402032 90 nop
00402033 90 nop
00402034 90 nop
00402035 90 nop
00402036 90 nop
00402037 90 nop
00402038 90 nop
00402039 90 nop
0040203A 90 nop
0040203B 90 nop
0040203C 90 nop
0040203D 90 nop
0040203E 90 nop
0040203F 90 nop
00402040 90 nop

Saritura de la adresa 00402002, la zona imediat după adresa codului ! (00402030)

Adresa de după saltul inițial este copiată:



La punctul de ieșire al regiunii, se cere o inserție de instrucțiuni:



În punctul de ieșire din regiune, introduceți JMP + adresa copiată:

mic NOP asm FASM (MessageBoxA).exe - PID: 17384 - Module: mic nop asm fasm (messageboxa).exe - Thread: ...

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

Assemble at 00402047

jmp 00402004

☐ Keep Size ☒ Fill with NOP's ☒ XEDParse ☐ asmjit

OK Cancel

Inst encoded successfully!

Bytes: EBBB

Default (stdcall) 5 Unload

1: [esp+4] 77766A78 ntdll.77766A78 "Ld

2: [esp+8] 777669FC ntdll.777669FC "mi

3: [esp+C] 00000000 00000000

4: [esp+10] 00000001 00000001

5: [esp+14] 000DFA18 000DFA18

.text:00402047 mic nop asm fasm (messageboxa).exe:2047 #447

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
00401000	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie Invers
00401010	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00	a.Malware.....
00401020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010C0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010D0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010E0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:02:58:43

Instrucțiunea JMP + adresa copiată a fost scrisă!

CPU Log Notes Breakpoints Memory Map Call Stack

Address	Hex	Instruction
0040201A	FF15 80304000	call dword ptr ds:[<MessageBoxA>]
00402020	6A 00	push 0
00402022	FF15 60304000	call dword ptr ds:[<ExitProcess>]
00402028	0000	add byte ptr ds:[eax],al
0040202A	0000	add byte ptr ds:[eax],al
0040202C	0000	add byte ptr ds:[eax],al
0040202E	0000	add byte ptr ds:[eax],al
00402030	90	nop
00402031	90	nop
00402032	90	nop
00402033	90	nop
00402034	90	nop
00402035	90	nop
00402036	90	nop
00402037	90	nop
00402038	90	nop
00402039	90	nop
0040203A	90	nop
0040203B	90	nop
0040203C	90	nop
0040203D	90	nop
0040203E	90	nop
0040203F	90	nop
00402040	90	nop
00402041	90	nop
00402042	90	nop
00402043	90	nop
00402044	90	nop
00402045	90	nop
00402046	90	nop
00402047	EB BB	jmp mic nop asm fasm (messageboxa).exe:2047 #447
00402048	90	nop
00402049	0000	add byte ptr ds:[eax],al
0040204A	0000	add byte ptr ds:[eax],al
0040204B	0000	add byte ptr ds:[eax],al
0040204C	0000	add byte ptr ds:[eax],al
0040204D	0000	add byte ptr ds:[eax],al
0040204E	0000	add byte ptr ds:[eax],al
0040204F	0000	add byte ptr ds:[eax],al

Saritura de la adresa 00402047, la zona imediat după adresa punctului de intrare! (00402004)

“PEȘTERA ÎN COD” (CODE CAVE)

ACUM, ÎN LOC DE INSTRUCȚIUNI **NOP**, PUTEM SCRIE COD

Avem un „buzunar” gol care poate fi folosit pentru a executa cod arbitrar (care se potrivește cu dimensiunea alocată de noi).

```
00402000 90 nop
00402001 90 nop
00402002 EB 2C jmp mic nop asm fasm (messageboxa).4020
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 001 push mic nop asm fasm (messageboxa).40140:
0040200D 68 12104 push mic nop asm fasm (messageboxa).40140:
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
00402030 90 nop
00402031 90 nop
00402032 90 nop
00402033 90 nop
00402034 90 nop
00402035 90 nop
00402036 90 nop
00402037 90 nop
00402038 90 nop
00402039 90 nop
0040203A 90 nop
0040203B 90 nop
0040203C 90 nop
0040203D 90 nop
0040203E 90 nop
0040203F 90 nop
00402040 90 nop
```

```
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
00402030 90 nop
00402031 90 nop
00402032 90 nop
00402033 90 nop
00402034 90 nop
00402035 90 nop
00402036 90 nop
00402037 90 nop
00402038 90 nop
00402039 90 nop
0040203A 90 nop
0040203B 90 nop
0040203C 90 nop
0040203D 90 nop
0040203E 90 nop
0040203F 90 nop
00402040 90 nop
00402041 90 nop
00402042 90 nop
00402043 90 nop
00402044 90 nop
00402045 90 nop
00402046 90 nop
00402047 EB BB jmp mic nop asm fasm (messageboxa).4020
00402048 90 nop
00402049 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204B 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204D 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
0040204F 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
```

C.8.3

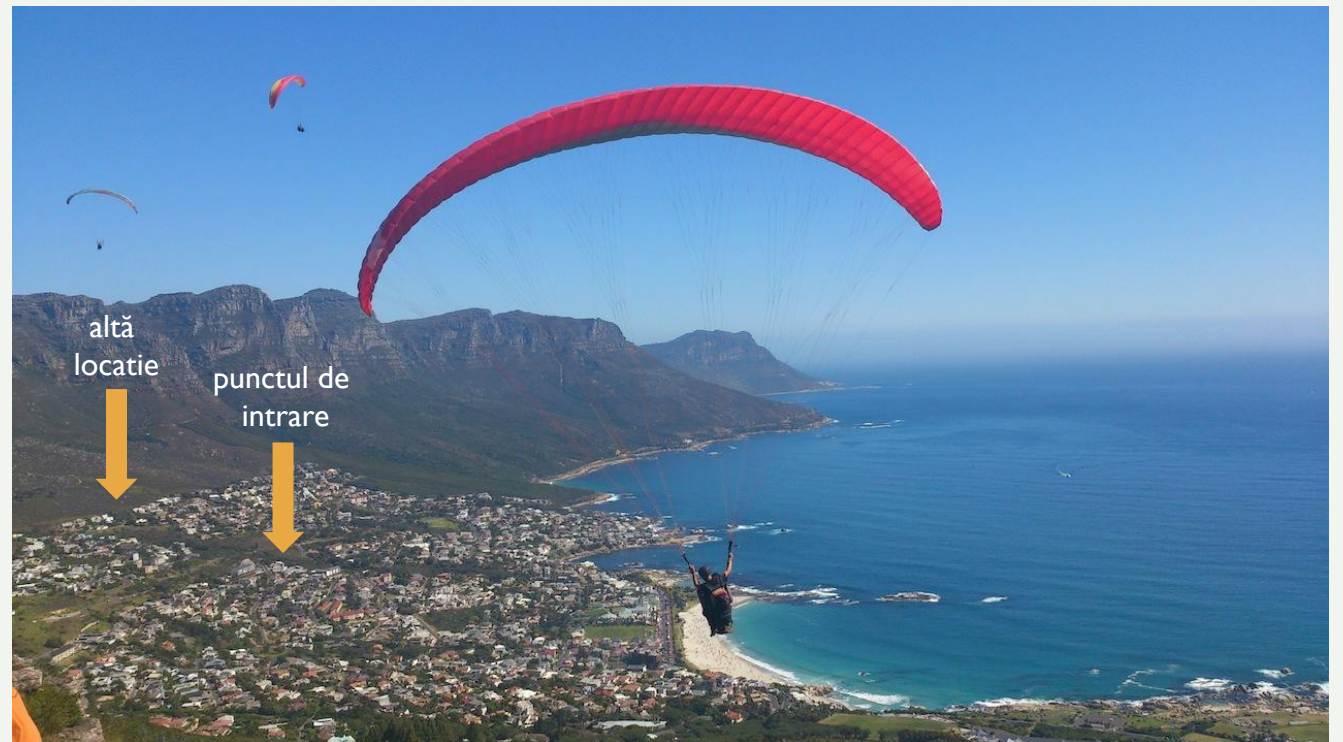
MODIFICARE ENTRY POINT ȘI INSERȚIE DE COD MAȘINĂ



MODIFICARE ENTRY POINT

PUNCTUL DE INTRARE CĂTRE ALT SEGMENT

1. Schimbați punctul de intrare la un alt segment
2. Rulați segmentul în regiunea selectată
3. Salt la adresa de sub punctul de intrare



X32dbg afișează punctul de inițializare (nu punctul de intrare):

test.exe - PID: 3676 - Module: ntdll.dll - Thread: Main Thread 21220 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

77811EE3 EB 07 jmp ntdll.77811EE3
77811EE5 33C0 xor eax, eax
77811EE7 40 inc eax
77811EE8 C3 ret
77811EE9 8B65 E8 mov esp, dword ptr [ebp-18]
77811EEC C745 FC FEFFFFFF mov dword ptr [ebp-4], FFFFFFFF
77811EF3 8B4D F0 mov ecx, dword ptr [ebp-10]
77811EF6 64:890D 00000000 mov dword ptr [eax], ecx
77811EFD 59 pop ecx
77811EFE 5F pop edi
77811EFF 5E pop esi
77811F00 5B pop ebx
77811F01 C9 leave
77811F02 C3 ret
77811F03 64:A1 30000000 mov eax, dword ptr [eax:30]
77811F09 33C9 xor ecx, ecx
77811F0B 890D D4678877 mov dword ptr ds:[778867D4], ecx
77811F11 890D D8678877 mov dword ptr ds:[778867D8], ecx
77811F17 8808 mov byte ptr ds:[eax], cl
77811F19 3848 02 cmp byte ptr ds:[eax+2], cl
77811F1C 74 05 je ntdll.77811F23
77811F1E E8 94FFFFFF call ntdll.77811EB7
77811F23 33C0 xor eax, eax
77811F25 C3 ret
77811F26 8BFF mov edi, edi
77811F28 55 push ebp
77811F29 8BEC mov ebp, esp
77811F2B 83E4 F8 and esp, FFFFFFF8
77811F2E 81EC 70010000 sub esp, 170
77811F34 A1 70B38877 mov eax, dword ptr ds:[7788B370]
77811F39 33C4 xor eax, esp
77811F3B 898424 6C010000 mov dword ptr ss:[esp+16C], eax
77811F42 56 push esi
77811F43 8B35 FC918877 mov esi, dword ptr ds:[778891FC]
77811F49 57 push edi
77811F4A 6A 16 push 16
77811F4C 58 pop eax
77811F4D 66:894424 10 mov word ptr ss:[esp+10], ax
77811F52 8BF9 mov edi, ecx
77811F54 6A 18 push 18
77811F56 58 pop eax
77811F57 66:894424 12 mov word ptr ss:[esp+12], ax
77811F5C 8D4424 70 lea eax, dword ptr ss:[esp+70]
77811F60 894424 6C mov dword ptr ss:[esp+6C], eax

ntdll.77811EE3
.text:77811EE3 ntdll.dll:\$B1EE3 #B1E23

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
77761000	16 00 18 00 F0 7D 76 77 14 00 16 00 50 7C 76 77	...0}vw...P vw
77761010	00 00 02 00 0C 5E 76 77 0E 00 10 00 C8 7F 76 77	...^vw...E.vw
77761020	0C 00 0E 00 B8 7F 76 77 08 00 0A 00 88 7B 76 77	...{vw...vw
77761030	06 00 08 00 98 7F 76 77 06 00 08 00 A8 7F 76 77	...vw...vw
77761040	06 00 08 00 A0 7F 76 77 06 00 08 00 B0 7F 76 77	...vw...vw
77761050	1C 00 1E 00 84 7C 76 77 20 00 22 00 40 82 76 77	... vw...@.vw
77761060	84 00 86 00 B8 81 76 77 50 6C 79 77 D0 4A 86 77	...vwplywDJ.w
77761070	80 B4 78 77 10 49 86 77 50 20 79 77 E0 69 79 77	...xw.I.wP ywaiy
77761080	90 49 86 77 50 4A 86 77 C0 57 79 77 D0 4A 86 77	...I.wPJ.wAwywDJ.w
77761090	E0 25 79 77 E0 69 79 77 E0 49 86 77 50 4A 86 77	...aywaiywi.wPJ.w
777610A0	F0 3F 76 77 F0 4A 86 77 00 00 00 00 00 00 00 00	...+1vw...

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused System breakpoint reached! Time Wasted Debugging: 0:03:29:52

Selectați fila „Memory map” și accesați secțiunea .text:

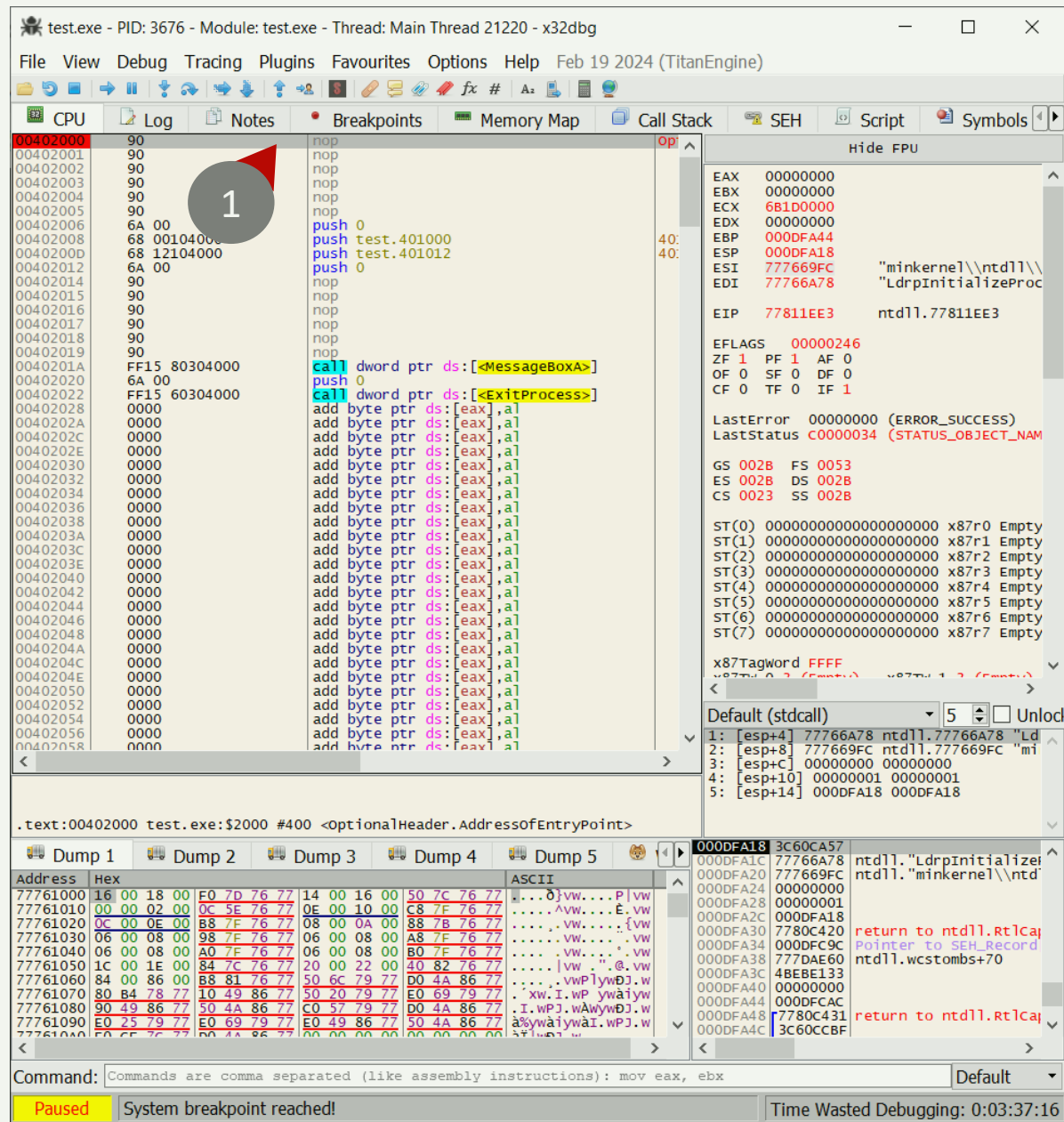
test.exe - PID: 3676 - Module: ntdll.dll - Thread: Main Thread 21220 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

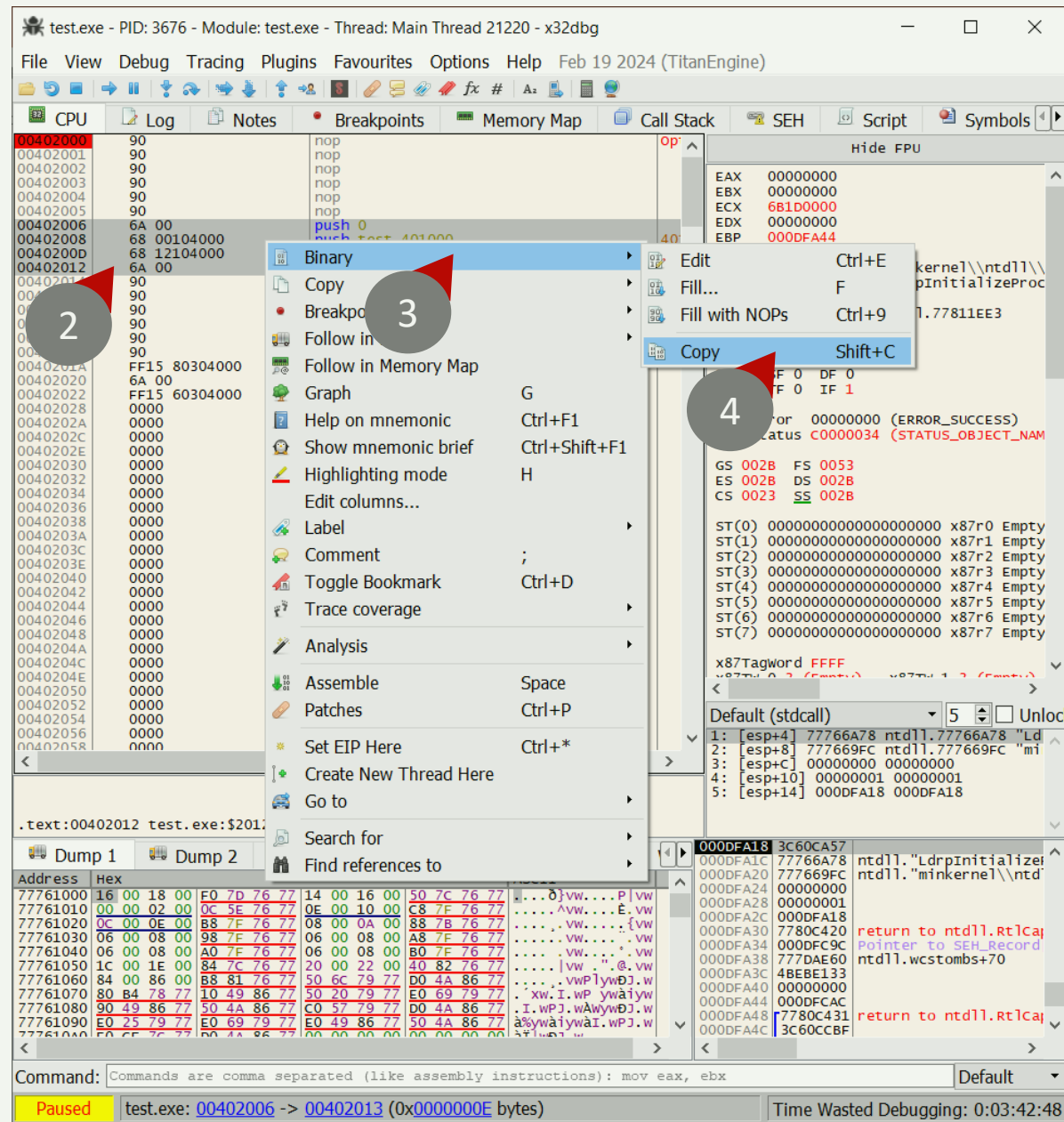
CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

Address	Size	Party	Info	Content
00010000	00001000	User		
00020000	00001000	User		
00030000	00001000	User		
00040000	00001000	User		
00060000	00003000	User		
00095000	00008000	User		
000AA000	00003A00	User		
000DA000	00006000	User		
000E0000	00004000	User		
000F0000	00002000	User		
00100000	00001000	User		
00110000	00001000	User		
00120000	00001000	User		
00130000	00001000	User		
00140000	00003500	User		
00175000	00008000	User		
00180000	00003500	User		
001B5000	00008000	User		
001D0000	00007000	User		
001D7000	00009000	User		
00200000	00007000	User		
0027F000	0000E000	User		
0028D000	00173000	User		
00400000	00001000	User		
00401000	00001000	User		
00402000	00001000	User		
00403000	00001000	User		
00410000	000C9000	User		
004E0000	00035000	User		
00515000	0000B000	User		
005A0000	00006000	User		
005A6000	0000A000	User		
005B0000	000FD000	User		
006AD000	00003000	User		
006B0000	000FD000	User		
007AD000	00003000	User		
007B0000	000FD000	User		
008AD000	00003000	User		
75810000	00001000	System	win32u.dll	
75811000	00006000	System	kernelbase.dll	
75817000	0000E000	System	kernelbase.dll	
75825000	00001000	System	kernelbase.dll	
75826000	00001000	System	kernelbase.dll	
75827000	00001000	System	kernelbase.dll	
75D40000	00001000	System	kernelbase.dll	
75D41000	001DE000	System	kernelbase.dll	
75F1F000	00004000	System	kernelbase.dll	
75F23000	00006000	System	kernelbase.dll	
75F29000	00001000	System	kernelbase.dll	
75F2A000	00001000	System	kernelbase.dll	
75F2B000	00001000	System	kernelbase.dll	
75F2C000	00001000	System	kernelbase.dll	
75F2D000	00001000	System	kernelbase.dll	
75F2E000	00001000	System	kernelbase.dll	
75F2F000	00001000	System	kernelbase.dll	
75F30000	00001000	System	kernelbase.dll	
75F31000	00001000	System	kernelbase.dll	
75F32000	00001000	System	kernelbase.dll	
75F33000	00001000	System	kernelbase.dll	
75F34000	00001000	System	kernelbase.dll	
75F35000	00001000	System	kernelbase.dll	
75F36000	00001000	System	kernelbase.dll	
75F37000	00001000	System	kernelbase.dll	
75F38000	00001000	System	kernelbase.dll	
75F39000	00001000	System	kernelbase.dll	
75F3A000	00001000	System	kernelbase.dll	
75F3B000	00001000	System	kernelbase.dll	
75F3C000	00001000	System	kernelbase.dll	
75F3D000	00001000	System	kernelbase.dll	
75F3E000	00001000	System	kernelbase.dll	
75F3F000	00001000	System	kernelbase.dll	
75F40000	00001000	System	kernelbase.dll	
75F41000	00001000	System	kernelbase.dll	
75F42000	00001000	System	kernelbase.dll	
75F43000	00001000	System	kernelbase.dll	
75F44000	00001000	System	kernelbase.dll	
75F45000	00001000	System	kernelbase.dll	
75F46000	00001000	System	kernelbase.dll	
75F47000	00001000	System	kernelbase.dll	
75F48000	00001000	System	kernelbase.dll	
75F49000	00001000	System	kernelbase.dll	
75F4A000	00001000	System	kernelbase.dll	
75F4B000	00001000	System	kernelbase.dll	
75F4C000	00001000	System	kernelbase.dll	
75F4D000	00001000	System	kernelbase.dll	
75F4E000	00001000	System	kernelbase.dll	
75F4F000	00001000	System	kernelbase.dll	
75F50000	00001000	System	kernelbase.dll	
75F51000	00001000	System	kernelbase.dll	
75F52000	00001000	System	kernelbase.dll	
75F53000	00001000	System	kernelbase.dll	
75F54000	00001000	System	kernelbase.dll	
75F55000	00001000	System	kernelbase.dll	
75F56000	00001000	System	kernelbase.dll	
75F57000	00001000	System	kernelbase.dll	
75F58000	00001000	System	kernelbase.dll	
75F59000	00001000	System	kernelbase.dll	
75F5A000	00001000	System	kernelbase.dll	
75F5B000	00001000	System	kernelbase.dll	
75F5C000	00001000	System	kernelbase.dll	
75F5D000	00001000	System	kernelbase.dll	
75F5E000	00001000	System	kernelbase.dll	
75F5F000	00001000	System	kernelbase.dll	
75F60000	00001000	System	kernelbase.dll	
75F61000	00001000	System	kernelbase.dll	
75F62000	00001000	System	kernelbase.dll	
75F63000	00001000	System	kernelbase.dll	
75F64000	00001000	System	kernelbase.dll	
75F65000	00001000	System	kernelbase.dll	
75F66000	00001000	System	kernelbase.dll	
75F67000	00001000	System	kernelbase.dll	
75F68000	00001000	System	kernelbase.dll	
75F69000	00001000	System	kernelbase.dll	
75F6A000	00001000	System	kernelbase.dll	
75F6B000	00001000	System	kernelbase.dll	
75F6C000	00001000	System	kernelbase.dll	
75F6D000	00001000	System	kernelbase.dll	
75F6E000	00001000	System	kernelbase.dll	
75F6F000	00001000	System	kernelbase.dll	
75F70000	00001000	System	kernelbase.dll	
75F71000	00001000	System	kernelbase.dll	
75F72000	00001000	System	kernelbase.dll	
75F73000	00001000	System	kernelbase.dll	
75F74000	00001000	System	kernelbase.dll	
75F75000	00001000	System	kernelbase.dll	
75F76000	00001000	System	kernelbase.dll	
75F77000	00001000	System	kernelbase.dll	
75F78000	00001000	System	kernelbase.dll	
75F79000	00001000	System	kernelbase.dll	
75F7A000	00001000	System	kernelbase.dll	
75F7B000	00001000	System	kernelbase.dll	
75F7C000	00001000	System	kernelbase.dll	
75F7D000	00001000	System	kernelbase.dll	
75F7E000	00001000	System	kernelbase.dll	
75F7F000	00001000	System	kernelbase.dll	
75F80000	00001000	System	kernelbase.dll	
75F81000	00001000	System	kernelbase.dll	
75F82000	00001000	System	kernelbase.dll	
75F83000	00001000	System	kernelbase.dll	
75F84000	00001000	System	kernelbase.dll	
75F85000	00001000	System	kernelbase.dll	
75F86000	00001000	System	kernelbase.dll	
75F87000	00001000	System	kernelbase.dll	
75F88000	00001000	System	kernelbase.dll	
75F89000	00001000	System	kernelbase.dll	
75F8A000	00001000	System	kernelbase.dll	
75F8B000	00001000	System	kernelbase.dll	
75F8C000	00001000	System	kernelbase.dll	
75F8D000	00001000	System	kernelbase.dll	
75F8E000	00001000	System	kernelbase.dll	
75F8F000	00001000	System	kernelbase.dll	
75F90000	00001000	System	kernelbase.dll	
75F91000	00001000	System	kernelbase.dll	
75F92000	00001000	System	kernelbase.dll	
75F93000	00001000	System	kernelbase.dll	
75F94000	00001000	System	kernelbase.dll	
75F95000	00001000	System	kernelbase.dll	
75F96000	00001000	System	kernelbase.dll	
75F97000	00001000	System	kernelbase.dll	
75F98000	00001000	System	kernelbase.dll	
75F99000	00001000	System	kernelbase.dll	
75F9A000	00001000	System	kernelbase.dll	
75F9B000	00001000	System	kernelbase.dll	
75F9C000	00001000	System	kernelbase.dll	
75F9D000	00001000	System	kernelbase.dll	
75F9E000	00001000	System	kernelbase.dll	
75F9F000	00001000	System	kernelbase.dll	
75FA0000	00001000	System	kernelbase.dll	
75FA1000	00001000	System	kernelbase.dll	
75FA2000	00001000	System	kernelbase.dll	
75FA3000	00001000	System	kernelbase.dll	
75FA4000	00001000	System	kernelbase.dll	
75FA5000	00001000	System	kernelbase.dll	
75FA6000	00001000	System	kernelbase.dll	
75FA7000	00001000	System	kernelbase.dll	
75FA8000	00001000	System	kernelbase.dll	
75FA9000	00001000	System	kernelbase.dll	
75FAA000	00001000	System	kernelbase.dll	
75FAB000	00001000	System	kernelbase.dll	
75FAC000	00001000	System	kernelbase.dll	
75FAD000	00001000	System	kernelbase.dll	
75FAE000	00001000	System	kernelbase.dll	
75FAF000	00001000	System	kernelbase.dll	
75FB0000	00001000	System	kernelbase.dll	
75FB1000	00001000	System	kernelbase.dll	
75FB2000	00001000	System	kernelbase.dll	
75FB3000	00001000	System	kernelbase.dll	
75FB4000	00001000	System	kernelbase.dll	
75FB5000	00001000	System	kernelbase.dll	
75FB6000	00001000	System	kernelbase.dll	
75FB7000	00001000	System	kernelbase.dll	
75FB8000	00001000	System	kernelbase.dll	
75FB9000	00001000	System	kernelbase.dll	
75FBA000	00001000	System	kernelbase.dll	
75FBB000	00001000	System	kernelbase.dll	
75FBC000	00001000	System	kernelbase.dll	
75FBD000	00001000	System	kernelbase.dll	
75FBE000	00001000	System	kernelbase.dll	
75FBF000	00001000	System	kernelbase.dll	
75FC0000	00001000	System	kernelbase.dll	
75FC1000	00001000	System	kernelbase.dll	
75FC2000	00001000	System	kernelbase.dll	
75FC3000	00001000	System	kernelbase.dll	
75FC4000	00001000	System	kernelbase.dll	
75FC5000	00001000	System	kernelbase.dll	
75FC6000	00001000	System	kernelbase.dll	
75FC7000	00001000	System	kernelbase.dll	
75FC8000	00001000	System	kernelbase.dll	
75FC9000	00001000	System	kernelbase.dll	
75FCA000	00001000	System	kernelbase.dll	
75FCB000	00001000	System	kernelbase.dll	
75FCC000	00001000	System	kernelbase.dll	
75FCD000	00001000	System	kernelbase.dll	

După accesarea secțiunii .text, x23dbg afișează punctul de intrare:



Selectați partea de cod, pe care apoi o copiați:



Clonăm cu „paste” codul într-o zonă liberă nedefinită:

test.exe - PID: 3676 - Module: test.exe - Thread: Main Thread 21220 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 90 nop
00402003 90 nop
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test.401000
0040200D 68 12104000 push test.401012
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 0000

Hide FPU

EAX 00000000
EBX 00000000
ECX 6B1D0000
EDX 00000000
EBP 000DFA44
ESP 000DFA18
ESI 777669FC "minkernel\\ntdll\\
EDI 77766A78 "LdrpInitializeProc
EIP 77811EE3 ntdll.77811EE3
EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1
LastError 00000000 (ERROR_SUCCESS)
LastStatus C0000034 (STATUS_OBJECT_NAME_NOT_FOUND)

Ctrl+E
F
Ctrl+9
Shift+C
Shift+V
Ctrl+Shift+V

Binary
Copy
Breakpoint
Follow in Dump
Follow in Memory Map
Graph
Help on mnemonic
Show mnemonic brief
Highlighting mode
Edit columns...
Label
Comment
Toggle Bookmark
Trace coverage
Analysis
Assemble
Patches
Set EIP Here
Create New Thread Here
Go to
Search for
Find references to

Default (stdcall) 5 Unload
1: [esp+4] 77766A78 ntdll.77766A78 "LdrpInitializeProc
2: [esp+8] 777669FC ntdll.777669FC "minkernel\\ntdll\\
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 000DFA18 000DFA18

Dump 5
ASCII
000DFA18 3C60CA57 ntdll."LdrpInitializeProc
000DFA1C 77766A78 ntdll."minkernel\\ntdll\\
000DFA20 777669FC
000DFA24 00000000
000DFA28 00000001
000DFA2C 000DFA18
000DFA30 7780C420
000DFA34 000DFC9C
000DFA38 777DAE60
000DFA3C 4BEBE133
000DFA40 00000000
000DFA44 000DFC9C
000DFA48 7780C431
000DFA4C 3C60CCBF

Command: instructions): mov eax, ebx

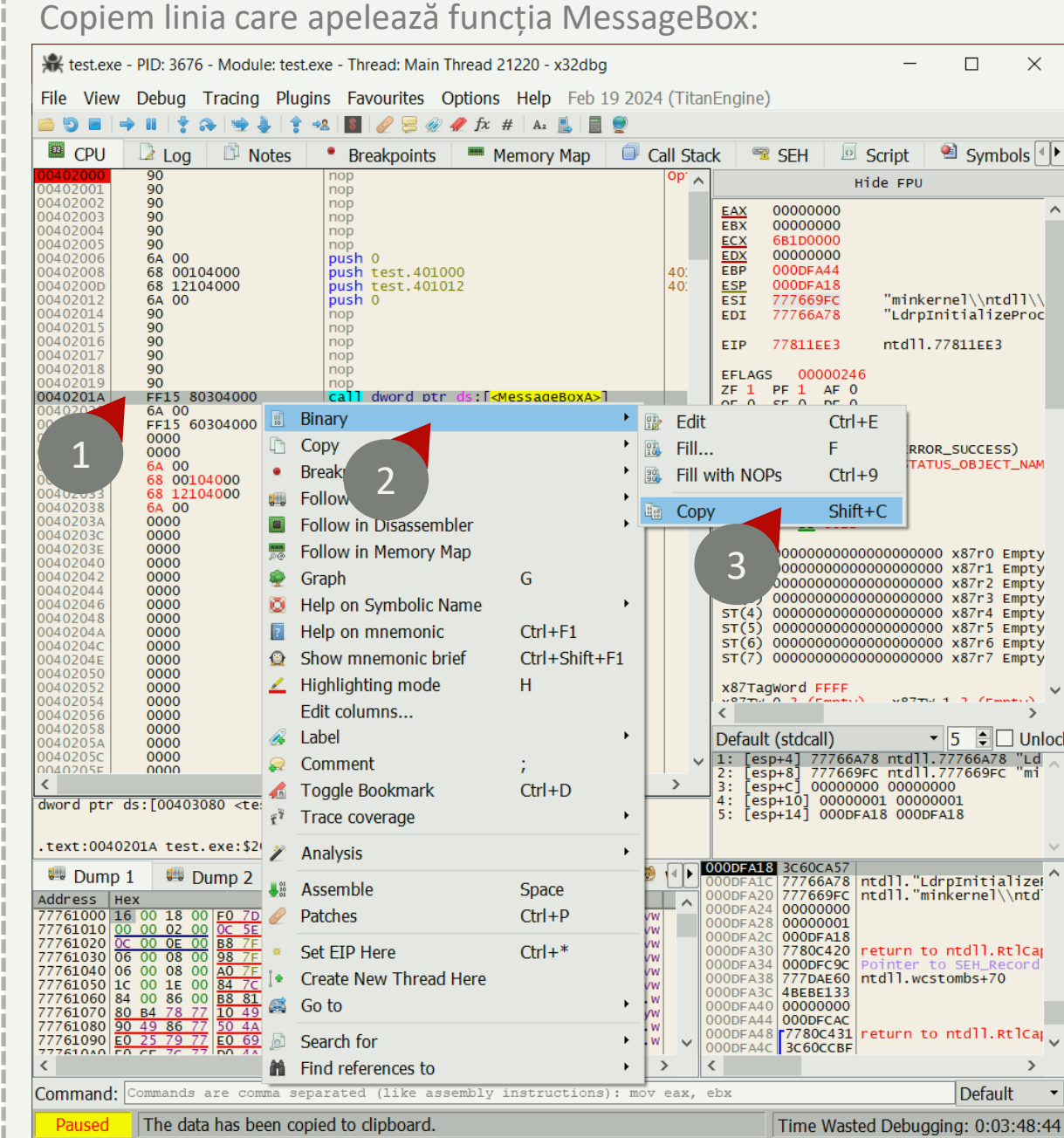
Paused The data has been copied to clipboard. Time Wasted Debugging: 0:03:44:45

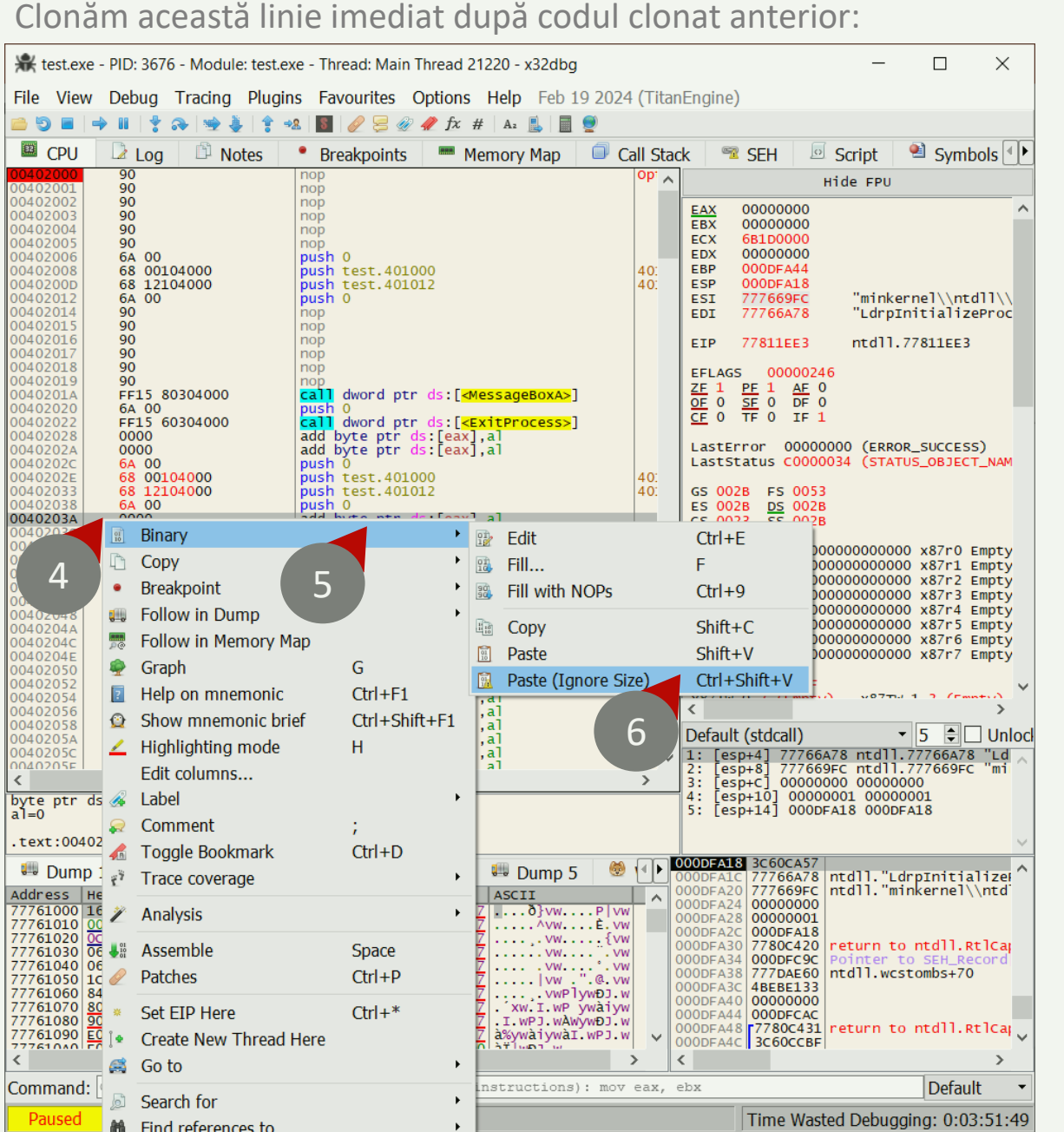
Codul clonat este afișat la adresa selectată:

CPU Log Notes Breakpoints Memory Map Call Stack

00402000 90 nop
00402001 90 nop
00402002 90 nop
00402003 90 nop
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test.401000
0040200D 68 12104000 push test.401012
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 6A 00 push 0
0040202E 68 00104000 push test.401000
00402033 68 12104000 push test.401012
00402038 6A 00 push 0
0040203A 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al

Copiați codul de la adresa 00402006 - 00402012 și
inserați codul la adresa 0040202C - 00402038





Avem cod clonat care nu poate fi accesat (ExitProcess):

Address	Hex	Disassembly
00402000	90	nop
00402001	90	nop
00402002	90	nop
00402003	90	nop
00402004	90	nop
00402005	90	nop
00402006	6A 00	push 0
00402008	68 00104000	push test.401000
0040200D	68 12104000	push test.401012
00402012	6A 00	push 0
00402014	90	nop
00402015	90	nop
00402016	90	nop
00402017	90	nop
00402018	90	nop
00402019	90	nop
0040201A	FF15 80304000	call dword ptr ds:[<MessageBoxA>]
00402020	6A 00	push 0
00402022	FF15 60304000	call dword ptr ds:[<ExitProcess>]
00402028	0000	add byte ptr ds:[eax],al
0040202A	0000	add byte ptr ds:[eax],al
0040202C	6A 00	push 0
0040202E	68 00104000	push test.401000
00402033	68 12104000	push test.401012
00402038	6A 00	push 0
0040203A	FF15 80304000	call dword ptr ds:[<MessageBoxA>]
00402040	0000	add byte ptr ds:[eax],al
00402042	0000	add byte ptr ds:[eax],al
00402044	0000	add byte ptr ds:[eax],al
00402046	0000	add byte ptr ds:[eax],al
00402048	0000	add byte ptr ds:[eax],al
0040204A	0000	add byte ptr ds:[eax],al
0040204C	0000	add byte ptr ds:[eax],al
0040204E	0000	add byte ptr ds:[eax],al
00402050	0000	add byte ptr ds:[eax],al
00402052	0000	add byte ptr ds:[eax],al
00402054	0000	add byte ptr ds:[eax],al
00402056	0000	add byte ptr ds:[eax],al
00402058	0000	add byte ptr ds:[eax],al
0040205A	0000	add byte ptr ds:[eax],al
0040205C	0000	add byte ptr ds:[eax],al
0040205E	0000	add byte ptr ds:[eax],al
00402060	0000	add byte ptr ds:[eax],al
00402062	0000	add byte ptr ds:[eax],al

Copiați linia de cod de la adresa 0040201A și
inserați linia de cod la adresa 0040203A.

Copiem adresa de unde începe codul clonat:

test.exe - PID: 3676 - Module: test.exe - Thread: Main Thread 21220 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 90 jmp test.402040
00402003 90 nop
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test.401000
0040200D 68 12104000 push test.401012
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 6A 00 push 0
0040202E 68 00104000 push test.401000
00402033 68 12104000 push test.401012
00402038 6A 00 push 0
0040203A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402040 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al

Binary
Copy
Restore
Breakpoint
Follow in Memory Map
Follow in Disassembly
Graph
Help on mnemonic
Show mnemonic brief
Highlighting mode
Edit columns...
Label
Comment
Toggle Bookmark
Trace coverage
Analysis
Assemble
Patches
Set EIP Here
Create New Thread Here
Go to
Search for
Find references to

Selection
Selection to File
Selection (No Bytes)
Selection to File (No Bytes)
Address
RVA
File
Header
Disassembly

Default (stdcall)
1: [esp+4] 77766A78 ntdll.77766A78 "LdrpInitializeProc
2: [esp+8] 777669FC ntdll.777669FC "mi
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 000DFA18 000DFA18

000DFA18 3C60CA57 ntdll."LdrpInitialize
000DFA1C 77766A78 ntdll."minkernel\\ntd
000DFA20 777669FC
000DFA24 00000000
000DFA28 00000001
000DFA2C 000DFA18
000DFA30 7780C420
000DFA34 000DFC9C
000DFA38 777DAE60
000DFA3C 4B5BE133
000DFA40 00000000
000DFA44 000DFC4C
000DFA48 7780C431 return to ntdll.RtlCa
000DFA4C 3C60CCBF

Command: Commands are co
Paused The data has been copied to clipboard. Time Wasted Debugging: 0:04:08:19

Solicităm o introducere cu instrucțiuni:

test.exe - PID: 3676 - Module: test.exe - Thread: Main Thread 21220 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 90 nop
00402003 90 nop
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test.401000
00402009 68 12104000 push test.401012
0040200A 6A 00 push 0
0040200B 90 nop
0040200C 90 nop
0040200D 90 nop
0040200E 90 nop
0040200F 90 nop
00402010 90 nop
00402011 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402012 6A 00 push 0
00402013 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402014 0000 add byte ptr ds:[eax],al
00402015 0000 add byte ptr ds:[eax],al
00402016 6A 00 push 0
00402017 68 00104000 push test.401000
00402018 68 12104000 push test.401012
00402019 6A 00 push 0
0040201A 0000 add byte ptr ds:[eax],al
0040201B 0000 add byte ptr ds:[eax],al
0040201C 0000 add byte ptr ds:[eax],al
0040201D 0000 add byte ptr ds:[eax],al
0040201E 0000 add byte ptr ds:[eax],al
0040201F 0000 add byte ptr ds:[eax],al
00402020 0000 add byte ptr ds:[eax],al
00402021 0000 add byte ptr ds:[eax],al
00402022 0000 add byte ptr ds:[eax],al
00402023 0000 add byte ptr ds:[eax],al
00402024 0000 add byte ptr ds:[eax],al
00402025 0000 add byte ptr ds:[eax],al
00402026 0000 add byte ptr ds:[eax],al
00402027 0000 add byte ptr ds:[eax],al
00402028 0000 add byte ptr ds:[eax],al
00402029 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202B 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202D 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
0040202F 0000 add byte ptr ds:[eax],al
00402030 0000 add byte ptr ds:[eax],al
00402031 0000 add byte ptr ds:[eax],al
00402032 0000 add byte ptr ds:[eax],al
00402033 0000 add byte ptr ds:[eax],al
00402034 0000 add byte ptr ds:[eax],al
00402035 0000 add byte ptr ds:[eax],al
00402036 0000 add byte ptr ds:[eax],al
00402037 0000 add byte ptr ds:[eax],al
00402038 0000 add byte ptr ds:[eax],al
00402039 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203B 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203D 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
0040203F 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402041 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402043 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402045 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402047 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
00402049 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204B 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204D 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
0040204F 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402051 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402053 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402055 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402057 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
00402059 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205B 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205D 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
0040205F 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402061 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al

Hide FPU

EAX 00000000
EBX 00000000
ECX 681D0000
EDX 00000000
EBP 000DFA44
ESP 000DFA18
ESI 777669FC
EDI 77766A78

EIP 77811EE3 ntdll.77811EE3

EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C0000034 (STATUS_OBJECT_NAME_NOT_FOUND)

Assemble at 00402002

nop

Keep Size ☒ Fill with NOP's ☒ XEDParse ☐ asmjit OK Cancel

Instruction encoded successfully! Bytes: 90

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:04:02:51

Apare fereastra unde instrucțiunea nop este înlocuită cu jmp:

test.exe - PID: 3676 - Module: test.exe - Thread: Main Thread 21220 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 90 nop
00402003 90 nop
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test.401000
00402009 68 12104000 push test.401012
0040200A 6A 00 push 0
0040200B 90 nop
0040200C 90 nop
0040200D 90 nop
0040200E 90 nop
0040200F 90 nop
00402010 90 nop
00402011 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402012 6A 00 push 0
00402013 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402014 0000 add byte ptr ds:[eax],al
00402015 0000 add byte ptr ds:[eax],al
00402016 6A 00 push 0
00402017 68 00104000 push test.401000
00402018 68 12104000 push test.401012
00402019 6A 00 push 0
0040201A 0000 add byte ptr ds:[eax],al
0040201B 0000 add byte ptr ds:[eax],al
0040201C 0000 add byte ptr ds:[eax],al
0040201D 0000 add byte ptr ds:[eax],al
0040201E 0000 add byte ptr ds:[eax],al
0040201F 0000 add byte ptr ds:[eax],al
00402020 0000 add byte ptr ds:[eax],al
00402021 0000 add byte ptr ds:[eax],al
00402022 0000 add byte ptr ds:[eax],al
00402023 0000 add byte ptr ds:[eax],al
00402024 0000 add byte ptr ds:[eax],al
00402025 0000 add byte ptr ds:[eax],al
00402026 0000 add byte ptr ds:[eax],al
00402027 0000 add byte ptr ds:[eax],al
00402028 0000 add byte ptr ds:[eax],al
00402029 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202B 0000 add byte ptr ds:[eax],al
0040202C 0000 add byte ptr ds:[eax],al
0040202D 0000 add byte ptr ds:[eax],al
0040202E 0000 add byte ptr ds:[eax],al
0040202F 0000 add byte ptr ds:[eax],al
00402030 0000 add byte ptr ds:[eax],al
00402031 0000 add byte ptr ds:[eax],al
00402032 0000 add byte ptr ds:[eax],al
00402033 0000 add byte ptr ds:[eax],al
00402034 0000 add byte ptr ds:[eax],al
00402035 0000 add byte ptr ds:[eax],al
00402036 0000 add byte ptr ds:[eax],al
00402037 0000 add byte ptr ds:[eax],al
00402038 0000 add byte ptr ds:[eax],al
00402039 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203B 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203D 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
0040203F 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402041 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402043 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402045 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402047 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
00402049 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204B 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204D 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
0040204F 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402051 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402053 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402055 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402057 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
00402059 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205B 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205D 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
0040205F 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402061 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al

Hide FPU

EAX 00000000
EBX 00000000
ECX 681D0000
EDX 00000000
EBP 000DFA44
ESP 000DFA18
ESI 777669FC
EDI 77766A78

EIP 77811EE3 ntdll.77811EE3

EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C0000034 (STATUS_OBJECT_NAME_NOT_FOUND)

Assemble at 00402002

nop

Keep Size ☐ Fill with NOP's ☒ XEDParse ☐ asmjit OK Cancel

Instruction encoded successfully! Bytes: 90

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:04:03:49

Introducem un salt necondiționat la adresa copiată anterior:

The screenshot shows a debugger window with the assembly view. A red circle with the number 1 points to a dialog box titled "Assemble at 00402002". The dialog box contains the instruction `jmp 0x0040202C` and the option "Fill with NOP's" is checked. A green message "Instruction encoded successfully! Bytes: EB28" is displayed. A red circle with the number 2 points to the assembly view where the instruction `jmp test.40202C` is being entered. The assembly view shows the following instructions:

```
00402000: nop
00402001: nop
00402002: jmp test.40202C
00402004: nop
00402005: nop
00402006: push 0
00402008: push test.401000
0040200A: push test.401012
0040200C: push 0
0040200E: nop
0040200F: nop
00402010: nop
00402011: nop
00402012: nop
00402013: nop
00402014: call dword ptr ds:[<MessageBoxA>]
00402016: push 0
00402018: call dword ptr ds:[<ExitProcess>]
0040201A: add byte ptr ds:[eax],al
0040201C: add byte ptr ds:[eax],al
0040201E: push 0
00402020: 68 00104000
00402022: 68 12104000
00402024: 6A 00
00402026: FF15 80304000
00402028: 0000
0040202A: 0000
0040202C: 6A 00
0040202E: 68 00104000
00402030: 68 12104000
00402032: 6A 00
00402034: FF15 80304000
00402036: 0000
00402038: 0000
0040203A: 0000
0040203C: 0000
0040203E: 0000
00402040: 0000
00402042: 0000
00402044: 0000
00402046: 0000
00402048: 0000
0040204A: 0000
0040204C: 0000
0040204E: 0000
00402050: 0000
00402052: 0000
00402054: 0000
00402056: 0000
00402058: 0000
0040205A: 0000
0040205C: 0000
0040205E: 0000
00402060: 0000
00402062: 0000
00402064: 0000
```

Clonăm această linie imediat după codul clonat anterior:

The screenshot shows a debugger window with the assembly view. A red circle with the number 3 points to the instruction `jmp test.40202C` at address 00402002. The assembly view shows the following instructions:

```
00402000: nop
00402001: nop
00402002: jmp test.40202C
00402004: nop
00402005: nop
00402006: push 0
00402008: push test.401000
0040200A: push test.401012
0040200C: push 0
0040200E: nop
0040200F: nop
00402010: nop
00402011: nop
00402012: nop
00402013: nop
00402014: call dword ptr ds:[<MessageBoxA>]
00402016: push 0
00402018: call dword ptr ds:[<ExitProcess>]
0040201A: add byte ptr ds:[eax],al
0040201C: add byte ptr ds:[eax],al
0040201E: push 0
00402020: 68 00104000
00402022: 68 12104000
00402024: 6A 00
00402026: FF15 80304000
00402028: 0000
0040202A: 0000
0040202C: 6A 00
0040202E: 68 00104000
00402030: 68 12104000
00402032: 6A 00
00402034: FF15 80304000
00402036: 0000
00402038: 0000
0040203A: 0000
0040203C: 0000
0040203E: 0000
00402040: 0000
00402042: 0000
00402044: 0000
00402046: 0000
00402048: 0000
0040204A: 0000
0040204C: 0000
0040204E: 0000
00402050: 0000
00402052: 0000
00402054: 0000
00402056: 0000
00402058: 0000
0040205A: 0000
0040205C: 0000
0040205E: 0000
00402060: 0000
00402062: 0000
00402064: 0000
```

Facem un salt de la adresa 00402002 la adresa 0040202C.

Adresa după saltul inițial la codul nostru este copiată:

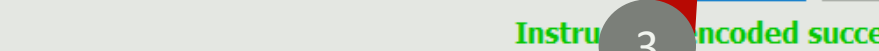
The screenshot shows the x32dbg interface with the CPU window displaying assembly code. A red arrow labeled '1' points to the instruction `jmp test.40202c`. Another red arrow labeled '2' points to the right-click context menu. A third red arrow labeled '3' points to the 'Address' option in the menu, which has the keyboard shortcut 'Alt+Ins' displayed next to it. The 'Command' window at the bottom shows the command `mov eax, ebx`.

După codul clonat, se solicită o introducere de instrucțiuni:

The screenshot shows the x32dbg interface with the CPU window displaying assembly code. A red arrow labeled '4' points to the instruction `jmp test.40202c`. Another red arrow labeled '5' points to the 'Assemble' option in the right-click context menu. The 'Command' window at the bottom shows the command `mov eax, ebx`.

Codul redundant este înlocuit cu un salt la adresa copiată:

The screenshot displays the Immunity Debugger interface. The main window shows assembly code for a module named 'test.exe'. The code includes instructions like 'nop', 'jmp test.40202C', 'push 0', 'push test.401000', 'push test.401012', 'push 0', 'call dword ptr ds:[<MessageBoxA>', 'push 0', 'call dword ptr ds:[<ExitProcess>', 'add byte ptr ds:[eax], al', and 'push 0'. The registers window on the right shows EAX=00000000, EBX=00000000, ECX=681D0000, EDX=00000000, EBP=000DFA44, ESP=000DFA18, ESI=777669FC, EDI=77766A78, and EIP=77811EE3. The command window at the bottom shows the command 'Commands are comma separated (like assembly instructions): mov eax, ebx'. The status bar at the bottom indicates 'Paused' and 'The data has been copied to clipboard.'



Assemble at 00402040

jmp 00402004

☐ Keep Size ☒ Fill with NOP's ☒ XEDParse ☐ asmjit

OK Cancel

Instruction encoded successfully!
Bytes: EBC2

CPU	Log	Notes	Breakpoints	Memory Map	Call Stack
00402000	90		nop		op
00402001	90		nop		
00402002	EB 28		jmp test.40202c		
00402004	90		nop		
00402005	90		nop		
00402006	6A 00		push 0		
00402008	68 00104000		push test.401000		40:
0040200D	68 12104000		push test.401012		40:
00402012	6A 00		push 0		
00402014	90		nop		
00402015	90		nop		
00402016	90		nop		
00402017	90		nop		
00402018	90		nop		
00402019	90		nop		
0040201A	FF15 80304000		call dword ptr ds:[<MessageBoxA>]		
00402020	6A 00		push 0		
00402022	FF15 60304000		call dword ptr ds:[<ExitProcess>]		
00402028	0000		add byte ptr ds:[eax],al		
0040202A	0000		add byte ptr ds:[eax],al		
0040202C	6A 00		push 0		
0040202E	68 00104000		push test.401000		40:
00402033	68 12104000		push test.401012		40:
00402038	6A 00		push 0		
0040203A	FF15 80304000		call dword ptr ds:[<MessageBoxA>]		
00402040	EB C2		jmp test.402004		
00402042	0000		add byte ptr ds:[eax],al		
00402044	0000		add byte ptr ds:[eax],al		
00402046	0000		add byte ptr ds:[eax],al		
00402048	0000		add byte ptr ds:[eax],al		
0040204A	0000		add byte ptr ds:[eax],al		
0040204C	0000		add byte ptr ds:[eax],al		
0040204E	0000		add byte ptr ds:[eax],al		
00402050	0000		add byte ptr ds:[eax],al		
00402052	0000		add byte ptr ds:[eax],al		
00402054	0000		add byte ptr ds:[eax],al		
00402056	0000		add byte ptr ds:[eax],al		
00402058	0000		add byte ptr ds:[eax],al		
0040205A	0000		add byte ptr ds:[eax],al		
0040205C	0000		add byte ptr ds:[eax],al		
0040205E	0000		add byte ptr ds:[eax],al		
00402060	0000		add byte ptr ds:[eax],al		
00402062	0000		add byte ptr ds:[eax],al		
00402064	0000		add byte ptr ds:[eax],al		

Noua versiune a executabilului este salvată:

test.exe - PID: 3676 - Module: test.exe - Thread: Main Thread 21220 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Alt+C
Log Window Alt+L
Notes
Breakpoints Alt+B
Memory Map Alt+M
Call Stack Alt+K
SEH Chain
Script Alt+S
Symbol Info Ctrl+Alt+S
Modules Alt+E
Source Ctrl+Shift+S
References Alt+R
Threads Alt+T
Handles
Graph Alt+G
Trace
Patch file... Ctrl+P
Comment Ctrl+Alt+C
Labels Ctrl+Alt+L
Bookmarks Ctrl+Alt+B
Functions Ctrl+Alt+F
Variables
Command Ctrl+Return
Previous Tab Alt+Left
Next Tab Alt+Right
Previous View Ctrl+Shift+Tab
Next View Ctrl+Tab
Hide Tab Ctrl+W

Address Hex
77761000 16 00 18 00 F0 7D 76 77 14 00 16 00 50 7C 76 77
77761010 00 00 02 00 0C 5E 76 77 0E 00 10 00 C8 7F 76 77
77761020 0C 00 0E 00 B8 7F 76 77 08 00 0A 00 88 7B 76 77
77761030 06 00 08 00 98 7F 76 77 06 00 08 00 A8 7F 76 77
77761040 06 00 08 00 A0 7F 76 77 06 00 08 00 B0 7F 76 77
77761050 1C 00 1E 00 84 7C 76 77 20 00 22 00 40 82 76 77
77761060 84 00 86 00 B8 81 76 77 50 6C 79 77 D0 4A 86 77
77761070 80 B4 78 77 10 49 86 77 50 20 79 77 E0 69 79 77
77761080 90 49 86 77 50 4A 86 77 C0 57 79 77 D0 4A 86 77
77761090 E0 25 79 77 E0 69 79 77 E0 49 86 77 50 4A 86 77
777610A0 E0 6F 7C 77 E0 4A 86 77 00 00 80 00 00 00 00 00

Command: Commands are comma separated (like assembly instructions): mov eax, ebx
Paused Open the patch dialog. Time Wasted Debugging: 0:04:21:36

Patches

Modules
test.exe

Patches
0|00402002:90->EB
0|00402003:90->28
1|0040202C:00->6A
1|0040202E:00->68
1|00402030:00->10
0->40
0->68
0->12
0->10
0->40
0->6A
0->FF
40203B:00->15
40203C:00->80
1|0040203D:00->30
1|0040203F:00->10

Information
18/18 patch(es) applied!
OK

Select All Deselect All Restore Selected
Import Export Pick Groups Patch File

Save file

This PC > Desktop > cod in exe

Organize New folder

Name Date modified Type Size
test.exe 3/15/2024 1:44 AM Application 2

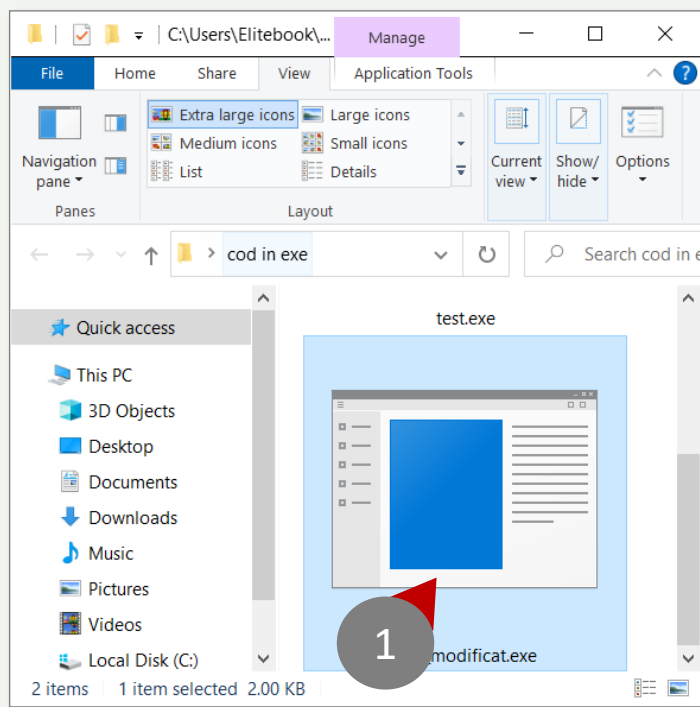
File name: test_modificat.exe
All files (*.*)

Save Cancel

C.8.4

ADAUGAREA DE STRUCTURI DE DATE IN EXECUTABILE





1. Schimbați punctul de intrare la un alt segment
2. Rulați segmentul în regiunea selectată
3. Salt la adresa de sub punctul de intrare

Selectați adresa și urmăriți eticheta în *dump* la adresa specificată:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

Hide FPU

EAX 00000000
EBX 00000000
ECX B4C80000
EDX 00000000
EBP 000DFA44
ESP 000DFA18
ESI 777669FC "minkernel\\ntdll\\
EDI 77766A78 "LdrpInitializeProc

EIP 77811EE3 ntdll.77811EE3

EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
LastStatus C0000034 (STATUS_OBJECT_NAM

GS 002B FS 0053
ES 002B DS 002B
CS 0023 SS 002B

ST(0) 000000000000000000 x87r0 Empty
ST(1) 00000000000000000000 x87r1 Empty
ST(2) 00000000000000000000 x87r2 Empty
ST(3) 00000000000000000000 x87r3 Empty
ST(4) 00000000000000000000 x87r4 Empty
ST(5) 00000000000000000000 x87r5 Empty
ST(6) 00000000000000000000 x87r6 Empty
ST(7) 00000000000000000000 x87r7 Empty

x87Tagword FFFF
x87C0 0.2 (Empty) x87C1 1.2 (Empty)

Default (stdcall) 5 Unlocked

1: [esp+4] 77766A78 ntdll.77766A78 "Ld
2: [esp+8] 777669FC ntdll.777669FC "mi
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 000DFA18 000DFA18

Dump 4 Dump 5

000DFA18 6EAC2CD1 ntdll."LdrpInitialize
000DFA1C 77766A78 ntdll."minkernel\\ntd
000DFA20 777669FC
000DFA24 00000000
000DFA28 00000001
000DFA2C 000DFA18
000DFA30 7780C420
000DFA34 000DFC9C
000DFA38 777DAE60
000DFA3C 192707B5
000DFA40 00000000
000DFA44 000DFC4C
000DFA48 7780C431
000DFA4C 6EAC2A39

return to ntdll.RtlCa
Pointer to SEH_Record
ntdll.wcstombs+70
return to ntdll.RtlCa

assembly instructions): mov eax, ebx

Time Wasted Debugging: 0:04:32:45

Ajungem la adresa 0040100 în *dump* și observăm șirul:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 EB 28 jmp test_modificat.40202C
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test_modificat.401000
00402009 68 12104000 push test_modificat.401012
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 6A 00 push 0
0040202E 68 00104000 push test_modificat.401000
00402030 68 12104000 push test_modificat.401012
00402032 6A 00 push 0
00402034 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402036 EB C2 jmp test_modificat.402004
00402038 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al

00401000 "Inginerie Inversa"

.text:0040202E test_modificat.exe:\$202E #42E

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
00401000	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie Inversa
00401010	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00	a.Malware.....
00401020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused test_modificat.exe: 00401000 -> 00401000 (0x00000001 bytes) Time Wasted Debugging: 0:04:34:40

Selectați șirul și copiați datele:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 EB 28 jmp test_modificat.40202C
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test_modificat.401000
00402009 68 12104000 push test_modificat.401012
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 6A 00 push 0
0040202E 68 00104000 push test_modificat.401000
00402030 68 12104000 push test_modificat.401012
00402032 6A 00 push 0
00402034 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402036 EB C2 jmp test_modificat.402004
00402038 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al

00401000 "Inginerie Inversa"

.text:0040202E test_modificat.exe:\$202E #42E

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
00401000	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie Inversa
00401010	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00	a.Malware.....
00401020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused test_modificat.exe: 00401000 -> 00401018 (0x00000019 bytes) Time Wasted Debugging: 0:04:35:42

Găsim zona liberă și dublăm datele folosind comanda paste:

The screenshot shows the Immunity Debugger interface. The CPU window displays assembly code for 'test_modificat.exe'. A context menu is open over the memory dump, with the following options numbered:

- 1: Hex
- 2: Binary
- 3: Paste (Ignore Size)
- 4: Dump 1

The memory dump shows a sequence of bytes, with the ASCII column displaying 'Inginerie Invers a.Malware.....'.

Avem date clonate la diferite adrese:

The screenshot shows the memory dump window with multiple dumps. The data is organized into columns for Address, Hex, and ASCII. The ASCII column displays 'Inginerie Invers a.Malware.....'.

Address	Hex	ASCII
00401000	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie Invers
00401010	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00	a.Malware.....
00401020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401040	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie Invers
00401050	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00	a.Malware.....
00401060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Clonăm datele pentru a diversifica argumentele pentru prima și a doua casetă de mesaj.



Editați șirul în *dump* la noua adresă unde se află clona:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 EB 28 jmp test_modificat.40202C
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test_modificat.401000
0040200D 68 12104000 push test_modificat.401012
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 6A 00 push 0
0040202E 68 00104000 push test_modificat.401000
00402033 68 12104000 push test_modificat.401012
00402038 6A 00 push 0
0040203A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402042 EB C2 jmp test_modificat.40204C
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al

00401000 "Inginerie Inversa"

.text:0040202E test_modificat.exe:\$02E #42E

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
00401000	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie
00401010	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00	a.Malware.
00401020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401040	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie
00401050	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00	a.Malware.
00401060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused test_modificat.exe: 00401040 -> 00401059 (0x0000001A bytes) Time Wasted Debugging: 0:04:46:19

Schimbați șirul, dar mențineți dimensiunea pentru moment:

Edit data at 00401040

Hex String Copy data

ASCII

Inginerie Inversa Malware

UNICODE:

UTF-8

Codepage...

Hex:

49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73
61 00 4D 61 6C 77 61 72 65 00

Keep Size

OK Cancel

Edit data at 00401040

Hex String Copy data

ASCII

Inginerie Malware MODware

UNICODE:

UTF-8

Codepage...

Hex:

49 6E 67 69 6E 65 72 69 65 20 4D 61 6C 77 61 72
65 00 4D 4F 44 77 61 72 65

Keep Size

OK Cancel

6

Ambele șiruri din clonă sunt editate:

Dump 1	Dump 2	Dump 3	Dump 4	Dump 5	
Address	Hex			ASCII	
00401000	49 6E 67 69	6E 65 72 69	65 20 49 6E	76 65 72 73	Inginerie Invers
00401010	61 00 4D 61	6C 77 61 72	65 00 00 00	00 00 00 00	a.Malware.....
00401020	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	
00401030	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	
00401040	49 6E 67 69	6E 65 72 69	65 20 49 6E	76 65 72 73	Inginerie Invers
00401050	61 00 4D 61	6C 77 61 72	65 00 00 00	00 00 00 00	a.Malware.....
00401060	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	
00401070	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	
00401080	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	
00401090	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	
004010A0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	

- “Inginerie Inversa” devine “Inginerie Malware”
- “Malware” devine “MODware”

Dump 1				Dump 2				Dump 3				Dump 4				Dump 5					
Address		Hex								ASCII											
00401000		49	6E	67	69	6E	65	72	69	65	20	49	6E	76	65	72	73	Inginerie Invers	^		
00401010		61	00	4D	61	6C	77	61	72	65	00	00	00	00	00	00	00	a.Malware.....			
00401020		00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00				
00401030		00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00				
00401040		49	6E	67	69	6E	65	72	69	65	20	4D	61	6C	77	61	72	Inginerie Malwar			
00401050		65	00	4D	4F	44	77	61	72	65	00	00	00	00	00	00	00	e.MODware.....			
00401060		00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00				
00401070		00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00				
00401080		00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00				
00401090		00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00				
004010A0		00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00				

Putem scrie un șir mai mare atâta timp cât
deplasăm adresa pentru cea de-a doua etichetă!

Copiați adresa la care se află șirul din clonă:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

Hide FPU

EAX 00000000
EBX 00000000
ECX 84C80000
EDX 00000000
EBP 000DFA44
ESP 000DFA18
ESI 777669FC "minkernel\\ntdll\\
EDI 77766A78 "LdrpInitializeProc

EIP 77811EE3 ntdll.77811EE3

EFLAGS 00000246
ZF 1 PF 1 AF 0
OF 0 SF 0 DF 0
CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)
Status C0000034 (STATUS_OBJECT_NAME

002B FS 0053
002B DS 002B
0023 SS 002B

(0) 00000000000000000000000000000000 x87r0 Empty
ST(1) 00000000000000000000000000000000 x87r1 Empty
ST(2) 00000000000000000000000000000000 x87r2 Empty
ST(3) 00000000000000000000000000000000 x87r3 Empty
ST(4) 00000000000000000000000000000000 x87r4 Empty
ST(5) 00000000000000000000000000000000 x87r5 Empty
ST(6) 00000000000000000000000000000000 x87r6 Empty
ST(7) 00000000000000000000000000000000 x87r7 Empty

x87Tagword FFFF
x87CN 0.3 (empty) x87PM 1.3 (empty)

Default (stdcall) 5 Unlod

1: [esp+4] 77766A78 ntdll.77766A78 "Ld
2: [esp+8] 777669FC ntdll.777669FC "mi
3: [esp+C] 00000000 00000000
4: [esp+10] 00000001 00000001
5: [esp+14] 000DFA18 000DFA18

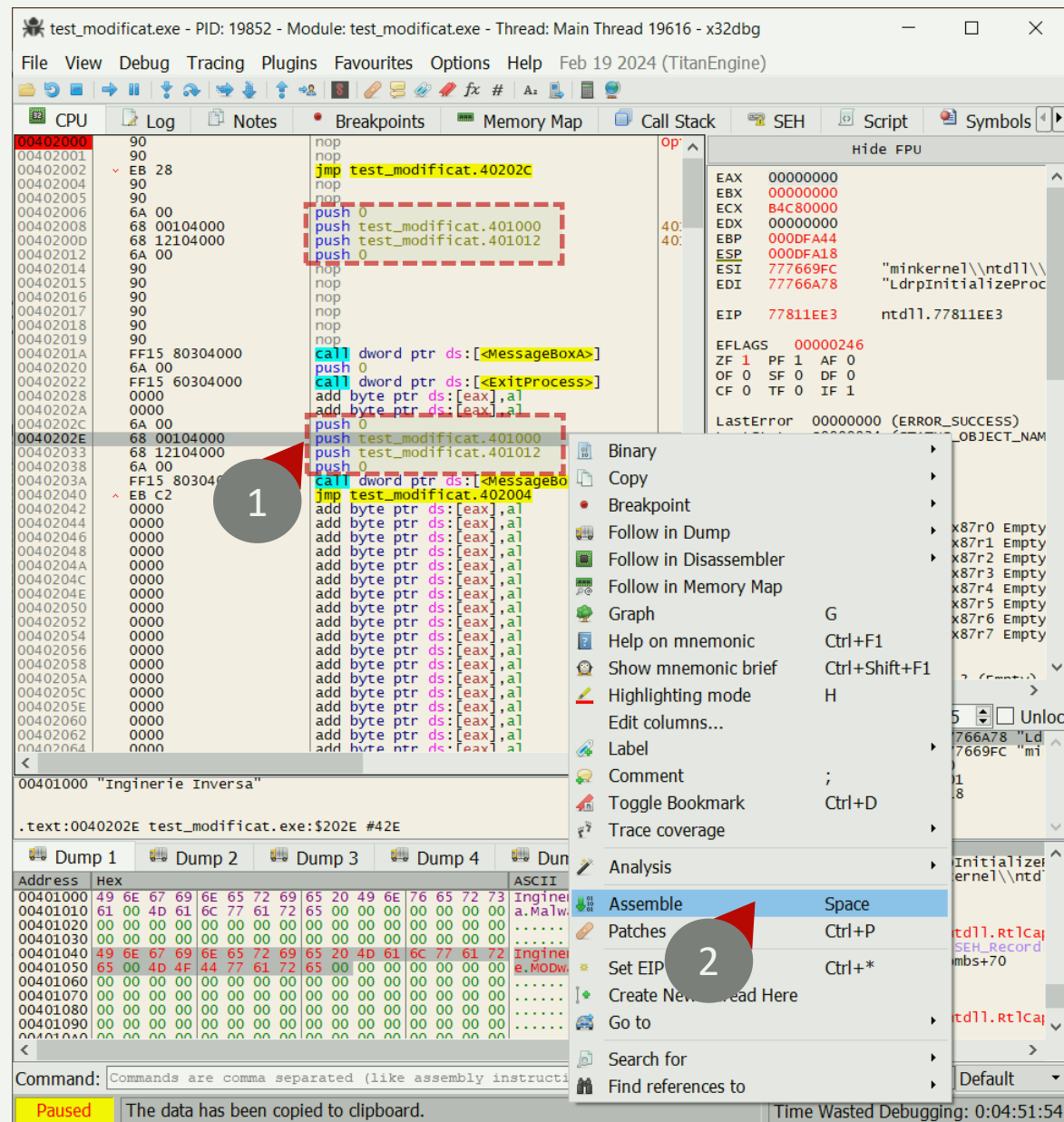
000DFA18 6EAC2CD1 ntdll."LdrpInitialize
000DFA1C 77766A78 ntdll."minkernel\\ntd
000DFA20 777669FC
000DFA24 00000000
000DFA28 00000001
000DFA2C 000DFA18
000DFA30 7780C420
000DFA34 000DFC9C
000DFA38 777DAE60
000DFA3C 19270785
000DFA40 00000000
000DFA44 000DFCAC
000DFA48 7780C431
000DFA4C 6EAC2A39

return to ntdll.RtlCap
Pointer to SEH_record
ntdll.wcstombs+70
return to ntdll.RtlCap

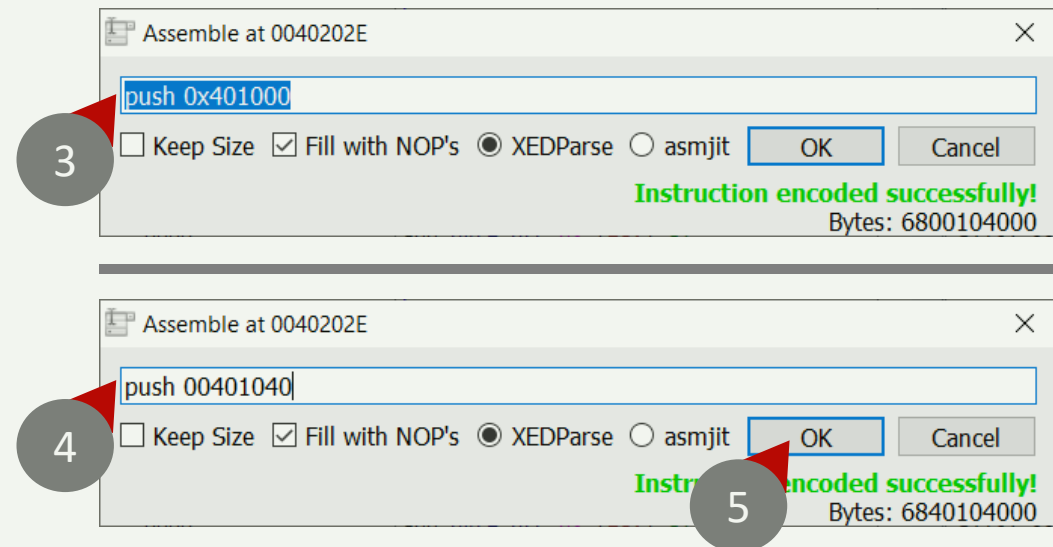
Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused test_modificat.exe: 00401040 -> 00401059 (0x0000001A bytes) Time Wasted Debugging: 0:04:50:10

Schimbăm adresa etichetei originale cu adresa clonei modificate:



Adresa originală este înlocuită cu cea nouă a clonei:



Observați că noua adresă la al doilea argument este schimbată:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 EB 28 jmp test_modificat.40202C
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test_modificat.401000
00402009 68 12104000 push test_modificat.401012
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 6A 00 push 0
0040202E 68 40104000 push test_modificat.401040
00402033 68 12104000 push test_modificat.401012
00402038 6A 00 push 0
0040203A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402040 EB C2 jmp test_modificat.402044
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al

00401040 "Inginerie Malware"

.text:0040202E test_modificat.exe:\$020E #42E

Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
00401000	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie Invers
00401010	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00	a.Malware.....
00401020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401040	49 6E 67 69 6E 65 72 69 65 20 4D 61 6C 77 61 72	Inginerie Malwar
00401050	65 00 4D 4F 44 77 61 72 65 00 00 00 00 00 00 00	e.Modware.....
00401060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:04:53:52

Peticim executabilul:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Window Notes Breakpoints Memory Map Call Stack SEH Chain Script Symbol Info Modules Source References Threads Handles Graph Trace Patch file... Comment Labels Bookmarks Functions Variables Command Previous Tab Next Tab Previous View Next View Hide Tab

00402000 90 nop
00402001 90 nop
00402002 EB 28 jmp test_modificat.40202C
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test_modificat.401000
00402009 68 12104000 push test_modificat.401012
00402012 6A 00 push 0
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402020 6A 00 push 0
00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402028 0000 add byte ptr ds:[eax],al
0040202A 0000 add byte ptr ds:[eax],al
0040202C 6A 00 push 0
0040202E 68 40104000 push test_modificat.401040
00402033 68 12104000 push test_modificat.401012
00402038 6A 00 push 0
0040203A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402040 EB C2 jmp test_modificat.402044
00402042 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al

00401040 "Inginerie Malware"

.text:0040202E test_modificat.exe:\$020E #42E

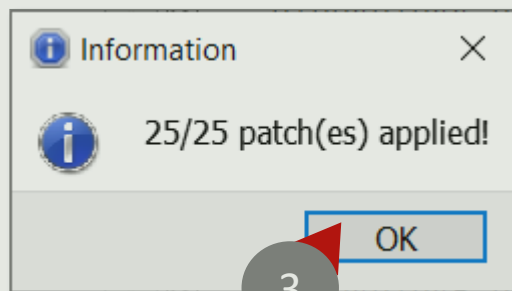
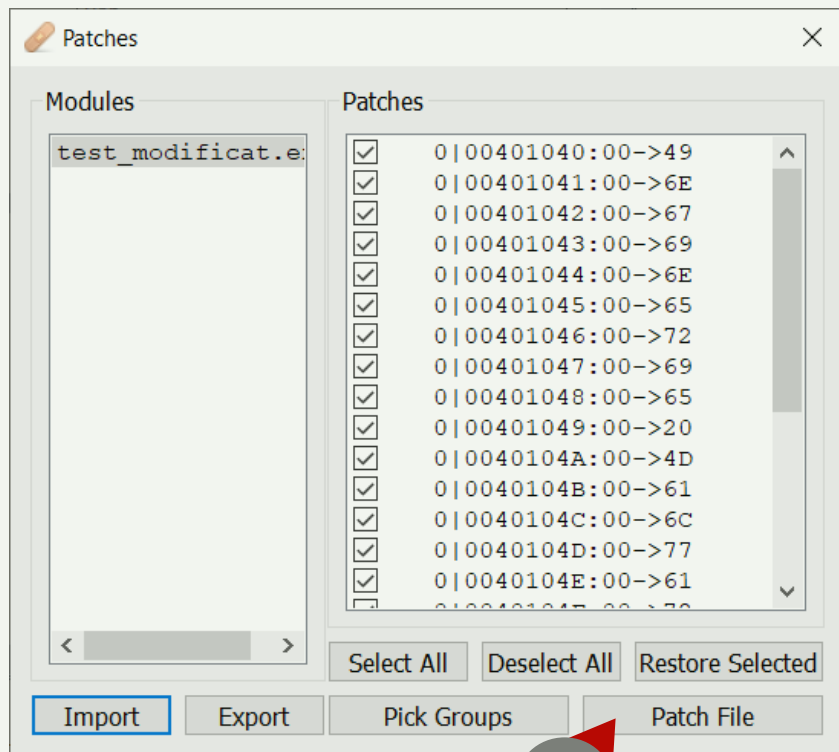
Dump 1 Dump 2 Dump 3 Dump 4 Dump 5

Address	Hex	ASCII
00401000	49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73	Inginerie Invers
00401010	61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00	a.Malware.....
00401020	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401040	49 6E 67 69 6E 65 72 69 65 20 4D 61 6C 77 61 72	Inginerie Malwar
00401050	65 00 4D 4F 44 77 61 72 65 00 00 00 00 00 00 00	e.Modware.....
00401060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401070	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401080	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00401090	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
004010A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

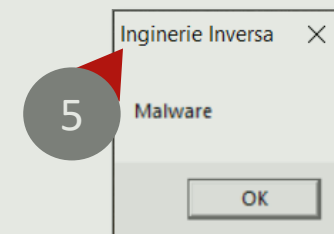
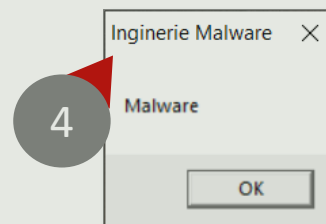
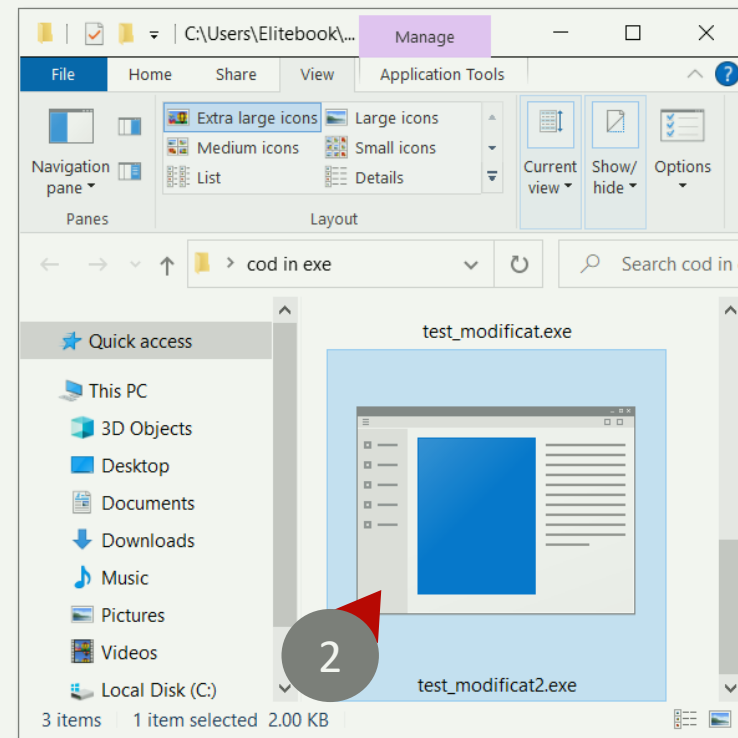
Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused Open the patch dialog. Time Wasted Debugging: 0:04:54:18

Modificările sunt prezentate orientativ:



Noua versiune poate fi testată:



C.8.5

MODIFICĂRI DE ADRESE CĂTRE ALTE ARGUMENTE



Copiați adresa pentru direcționarea către cel de-al treilea argument:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 EB 28 jmp test_modificat.40202C
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test_modificat.401000
00402009 68 12104000 push test_modificat.401012
00402010 6A 00 push 0
00402011 90 nop
00402012 90 nop
00402013 90 nop
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
0040201B 6A 00 push 0
0040201C FF15 60304000 call dword ptr ds:[<ExitProcess>]
0040201D 0000 add byte ptr ds:[eax],al
0040201E 0000 add byte ptr ds:[eax],al
0040201F 6A 00 push 0
00402020 68 40104000 push test_modificat.401040
00402021 68 12104000 push test_modificat.401012
00402022 6A 00 push 0
00402023 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402024 6A 00 push 0
00402025 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402026 0000 add byte ptr ds:[eax],al
00402027 0000 add byte ptr ds:[eax],al
00402028 6A 00 push 0
00402029 68 40104000 push test_modificat.401040
0040202A 68 12104000 push test_modificat.401012
0040202B 6A 00 push 0
0040202C FF15 80304000 call dword ptr ds:[<MessageBoxA>]
0040202D 6A 00 push 0
0040202E FF15 60304000 call dword ptr ds:[<ExitProcess>]
0040202F 0000 add byte ptr ds:[eax],al
00402030 0000 add byte ptr ds:[eax],al
00402031 6A 00 push 0
00402032 68 40104000 push test_modificat.401040
00402033 68 12104000 push test_modificat.401012
00402034 6A 00 push 0
00402035 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402036 6A 00 push 0
00402037 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402038 0000 add byte ptr ds:[eax],al
00402039 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203B 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203D 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
0040203F 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402041 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402043 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402045 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402047 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
00402049 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204B 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204D 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
0040204F 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402051 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402053 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402055 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402057 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
00402059 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205B 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205D 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
0040205F 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402061 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402063 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al

Binary
Copy
Restore selection
Follow in Disassembler
Follow in Memory
Label Current Address
Watch DWORD
Modify Value
Breakpoint
Find Pattern...
Find References
Sync with expression
Allocate Memory
Go to
Hex
Text
Integer
Float
Address
Disassembly

Address Hex
00401000 49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73
00401010 61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00
00401020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401040 49 6E 67 69 6E 65 72 69 65 20 4D 61 6C 77 61 72
00401050 65 00 4D 4F 44 77 61 72 65 00 00 00 00 00 00
00401060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:05:02:44

Schimbarea adresei etichetei:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols

00402000 90 nop
00402001 90 nop
00402002 EB 28 jmp test_modificat.40202C
00402004 90 nop
00402005 90 nop
00402006 6A 00 push 0
00402008 68 00104000 push test_modificat.401000
00402009 68 12104000 push test_modificat.401012
00402010 6A 00 push 0
00402011 90 nop
00402012 90 nop
00402013 90 nop
00402014 90 nop
00402015 90 nop
00402016 90 nop
00402017 90 nop
00402018 90 nop
00402019 90 nop
0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]
0040201B 6A 00 push 0
0040201C FF15 60304000 call dword ptr ds:[<ExitProcess>]
0040201D 0000 add byte ptr ds:[eax],al
0040201E 0000 add byte ptr ds:[eax],al
0040201F 6A 00 push 0
00402020 68 40104000 push test_modificat.401040
00402021 68 12104000 push test_modificat.401012
00402022 6A 00 push 0
00402023 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402024 6A 00 push 0
00402025 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402026 0000 add byte ptr ds:[eax],al
00402027 0000 add byte ptr ds:[eax],al
00402028 6A 00 push 0
00402029 68 40104000 push test_modificat.401040
0040202A 68 12104000 push test_modificat.401012
0040202B 6A 00 push 0
0040202C FF15 80304000 call dword ptr ds:[<MessageBoxA>]
0040202D 6A 00 push 0
0040202E FF15 60304000 call dword ptr ds:[<ExitProcess>]
0040202F 0000 add byte ptr ds:[eax],al
00402030 0000 add byte ptr ds:[eax],al
00402031 6A 00 push 0
00402032 68 40104000 push test_modificat.401040
00402033 68 12104000 push test_modificat.401012
00402034 6A 00 push 0
00402035 FF15 80304000 call dword ptr ds:[<MessageBoxA>]
00402036 6A 00 push 0
00402037 FF15 60304000 call dword ptr ds:[<ExitProcess>]
00402038 0000 add byte ptr ds:[eax],al
00402039 0000 add byte ptr ds:[eax],al
0040203A 0000 add byte ptr ds:[eax],al
0040203B 0000 add byte ptr ds:[eax],al
0040203C 0000 add byte ptr ds:[eax],al
0040203D 0000 add byte ptr ds:[eax],al
0040203E 0000 add byte ptr ds:[eax],al
0040203F 0000 add byte ptr ds:[eax],al
00402040 0000 add byte ptr ds:[eax],al
00402041 0000 add byte ptr ds:[eax],al
00402042 0000 add byte ptr ds:[eax],al
00402043 0000 add byte ptr ds:[eax],al
00402044 0000 add byte ptr ds:[eax],al
00402045 0000 add byte ptr ds:[eax],al
00402046 0000 add byte ptr ds:[eax],al
00402047 0000 add byte ptr ds:[eax],al
00402048 0000 add byte ptr ds:[eax],al
00402049 0000 add byte ptr ds:[eax],al
0040204A 0000 add byte ptr ds:[eax],al
0040204B 0000 add byte ptr ds:[eax],al
0040204C 0000 add byte ptr ds:[eax],al
0040204D 0000 add byte ptr ds:[eax],al
0040204E 0000 add byte ptr ds:[eax],al
0040204F 0000 add byte ptr ds:[eax],al
00402050 0000 add byte ptr ds:[eax],al
00402051 0000 add byte ptr ds:[eax],al
00402052 0000 add byte ptr ds:[eax],al
00402053 0000 add byte ptr ds:[eax],al
00402054 0000 add byte ptr ds:[eax],al
00402055 0000 add byte ptr ds:[eax],al
00402056 0000 add byte ptr ds:[eax],al
00402057 0000 add byte ptr ds:[eax],al
00402058 0000 add byte ptr ds:[eax],al
00402059 0000 add byte ptr ds:[eax],al
0040205A 0000 add byte ptr ds:[eax],al
0040205B 0000 add byte ptr ds:[eax],al
0040205C 0000 add byte ptr ds:[eax],al
0040205D 0000 add byte ptr ds:[eax],al
0040205E 0000 add byte ptr ds:[eax],al
0040205F 0000 add byte ptr ds:[eax],al
00402060 0000 add byte ptr ds:[eax],al
00402061 0000 add byte ptr ds:[eax],al
00402062 0000 add byte ptr ds:[eax],al
00402063 0000 add byte ptr ds:[eax],al
00402064 0000 add byte ptr ds:[eax],al

Binary
Copy
Restore selection
Follow in Disassembler
Follow in Memory
Label Current Address
Watch DWORD
Modify Value
Breakpoint
Find Pattern...
Find References
Sync with expression
Allocate Memory
Go to
Hex
Text
Integer
Float
Address
Disassembly

Address Hex
00401000 49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73
00401010 61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 00
00401020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401040 49 6E 67 69 6E 65 72 69 65 20 4D 61 6C 77 61 72
00401050 65 00 4D 4F 44 77 61 72 65 00 00 00 00 00 00
00401060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused The data has been copied to clipboard. Time Wasted Debugging: 0:07:55

Schimbarea adresei originale:

Assemble at 00402033

push 0x401012

1

☐ Keep size

☒ Fill with NOP's

☒ XEDParse

☐ asmjit

OK

Cancel

Instruction encoded successfully!

Bytes: 6812104000

Cu adresa nou din clona:

Assemble at 00402033

push 00401052

2

☐ Keep size

☒ Fill with NOP's

☒ XEDParse

☐ asmjit

OK

Cancel

Instruction encoded successfully!

Bytes: 6852104000



Adresa a fost schimbată:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU

Log

Notes

Breakpoints

Memory Map

Call Stack

SEH

Script

Symbols

Hide FPU

EAX 00000000

EBX 00000000

ECX 84C80000

EDX 00000000

EBP 000DFA44

ESP 000DFA18

ESI 777669FC

EDI 77766A78

EIP 77811EE3

EFLAGS 00000246

ZF 1 PF 1 AF 0

OF 0 SF 0 DF 0

CF 0 TF 0 IF 1

LastError 00000000 (ERROR_SUCCESS)

LastStatus C0000034 (STATUS_OBJECT_NAME_NOT_FOUND)

GS 002B FS 0053

ES 002B DS 002B

CS 002B SS 002B

ST(0) 0000000000000000 x87r0 Empty

ST(1) 0000000000000000 x87r1 Empty

ST(2) 0000000000000000 x87r2 Empty

ST(3) 0000000000000000 x87r3 Empty

ST(4) 0000000000000000 x87r4 Empty

ST(5) 0000000000000000 x87r5 Empty

ST(6) 0000000000000000 x87r6 Empty

ST(7) 0000000000000000 x87r7 Empty

x87Tagword FFFF

x87CN 0.3 (Empty)

x87PM 1.3 (Empty)

Default (stdcall)

5

Unlod

1: [esp+4] 77766A78 ntdll.77766A78 "LdrpInitializeProcess"

2: [esp+8] 777669FC ntdll.777669FC "minkernel\\ntdll\\LdrpInitializeProcess"

3: [esp+C] 00000000 00000000

4: [esp+10] 00000001 00000001

5: [esp+14] 000DFA18 000DFA18

00402000 90 nop

00402001 90 nop

00402002 EB 28 jmp test_modificat.40202C

00402004 90 nop

00402005 90 nop

00402006 6A 00 push 0

00402008 68 00104000 push test_modificat.401000

0040200D 68 12104000 push test_modificat.401012

00402012 6A 00 push 0

00402014 90 nop

00402015 90 nop

00402016 90 nop

00402017 90 nop

00402018 90 nop

00402019 90 nop

0040201A FF15 80304000 call dword ptr ds:[<MessageBoxA>]

00402020 6A 00 push 0

00402022 FF15 60304000 call dword ptr ds:[<ExitProcess>]

00402028 0000 add byte ptr ds:[eax],al

0040202A 0000 add byte ptr ds:[eax],al

0040202C 6A 00 push 0

0040202E 68 40104000 push test_modificat.401040

00402033 68 52104000 push test_modificat.401052

00402038 6A 00 push 0

0040203A FF15 80304000 call dword ptr ds:[<MessageBoxA>]

00402040 EB C2 jmp test_modificat.404004

00402042 0000 add byte ptr ds:[eax],al

00402044 0000 add byte ptr ds:[eax],al

00402046 0000 add byte ptr ds:[eax],al

00402048 0000 add byte ptr ds:[eax],al

0040204A 0000 add byte ptr ds:[eax],al

0040204C 0000 add byte ptr ds:[eax],al

0040204E 0000 add byte ptr ds:[eax],al

00402050 0000 add byte ptr ds:[eax],al

00402052 0000 add byte ptr ds:[eax],al

00402054 0000 add byte ptr ds:[eax],al

00402056 0000 add byte ptr ds:[eax],al

00402058 0000 add byte ptr ds:[eax],al

0040205A 0000 add byte ptr ds:[eax],al

0040205C 0000 add byte ptr ds:[eax],al

0040205E 0000 add byte ptr ds:[eax],al

00402060 0000 add byte ptr ds:[eax],al

00402062 0000 add byte ptr ds:[eax],al

00402064 0000 add byte ptr ds:[eax],al

00401052 "MODware"

.text:00402033 test_modificat.exe:\$2033 #433

Dump 1

Dump 2

Dump 3

Dump 4

Dump 5

000DFA18 6EAC2CD1 ntdll."LdrpInitializeProcess"

000DFA1C 77766A78 ntdll."minkernel\\ntdll\\LdrpInitializeProcess"

000DFA20 777669FC

000DFA24 00000000

000DFA28 00000001

000DFA2C 000DFA18

000DFA30 7780C420

000DFA34 000DFC9C

000DFA38 777DAE60

000DFA3C 19270785

000DFA40 00000000

000DFA44 000DFCAC

000DFA48 7780C431

000DFA4C 6EAC2A39

return to ntdll.RtlCall

Pointer to SEH_record

ntdll.wcstombs+70

return to ntdll.RtlCall

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused

The data has been copied to clipboard.

Time Wasted Debugging: 0:05:11:17

Se trece la peticirea executabilului:

test_modificat.exe - PID: 19852 - Module: test_modificat.exe - Thread: Main Thread 19616 - x32dbg

File View Debug Tracing Plugins Favourites Options Help Feb 19 2024 (TitanEngine)

CPU Alt+C
Log Window Alt+L
Notes
Breakpoints Alt+B
Memory Map Alt+M
Call Stack Alt+K
SEH Chain
Script Alt+S
Symbol Info Ctrl+Alt+S
Modules Alt+E
Source Ctrl+Shift+S
References Alt+R
Threads Alt+T
Handles
Graph Alt+G
Trace
Patch file... Ctrl+P
Comment Ctrl+Alt+C
Label Ctrl+Alt+L
Bookmark Ctrl+Alt+B
Functions Ctrl+Alt+F
Variables
Command Ctrl+Return
Previous Tab Alt+Left
Next Tab Alt+Right
Previous View Ctrl+Shift+Tab
Next View Ctrl+Tab
Hide Tab Ctrl+W

Address Hex ASCII
00401000 49 6E 67 69 6E 65 72 69 65 20 49 6E 76 65 72 73 Inginerie Invers
00401010 61 00 4D 61 6C 77 61 72 65 00 00 00 00 00 00 a.Malware.....
00401020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401040 49 6E 67 69 6E 65 72 69 65 20 4D 61 6C 77 61 72 Inginerie Malwar
00401050 65 00 4D 4F 44 77 61 72 65 00 00 00 00 00 00 e.Modware.....
00401060 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401070 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401080 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00401090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
004010A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Command: Commands are comma separated (like assembly instructions): mov eax, ebx
Paused Open the patch dialog. Time Wasted Debugging: 0:05:15:26

Noua versiune este scrisă pe disc și poate fi testată:

Patches

Modules
test_modificat.e

Patches
0|00401040:00->49
0|00401041:00->6E
0|00401042:00->67
0|00401043:00->69
0|00401044:00->6E
0|00401045:00->65
0|00401046:00->72
0|00401047:00->69
0|00401048:00->65
0|00401049:00->20
0|0040104A:00->4D
0|0040104B:00->61
0|0040104C:00->6C
0|0040104D:00->77
0|0040104E:00->61
0|0040104F:00->70

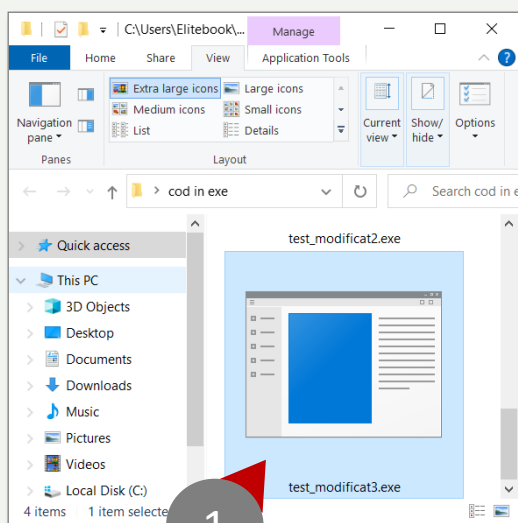
Select All Deselect All Restore Selected

Import Export Pick Groups Patch File

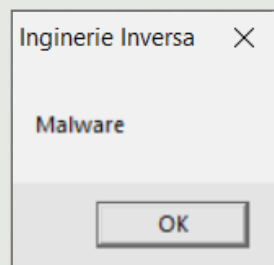
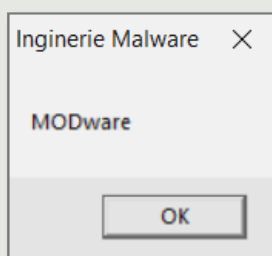
ÎN CAZUL CODULUI MALIȚIOS

CE PUTEM REALIZA CU ACESTE TIPURI DE SCHIMBĂRI?

Câteva activități de ghidare:



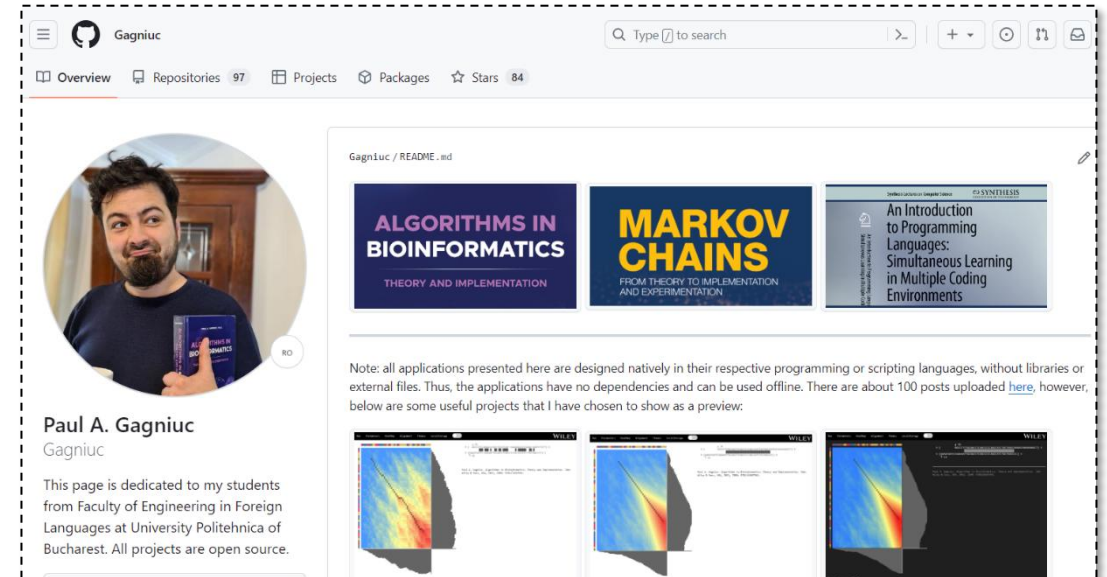
1. Putem vedea legături între serverele de comandă și control (C&C).
2. Putem dilua infecția mașinilor din rețelele mari creând variante ale aceluiași malware (ex. poate exista o luptă în procesul de infecție; unele aplicații malware răspund la C&C original, iar altele la un honeypot).
3. De cele mai multe ori, există un marker pentru infecție (o serie de octeți). Odată ce o mașină este infectată cu malware-ul corectat, malware-ul original nu va infecta acea mașină (este similar cu un **vaccin biologic**).
4. Abaterea de la secțiunea **.text** poate duce la confuzia unui virus dacă se așteaptă la denumirea clasică **.text**



BIBLIOGRAFIE / RESURSE

- Paul A. Gagniuc. *Antivirus Engines: From Methods to Innovations, Design, and Applications*. Cambridge, MA: Elsevier Syngress, 2024. pp. 1-656.
- Paul A. Gagniuc. *An Introduction to Programming Languages: Simultaneous Learning in Multiple Coding Environments. Synthesis Lectures on Computer Science*. Springer International Publishing, 2023, pp. 1-280.
- Paul A. Gagniuc. *Coding Examples from Simple to Complex - Applications in MATLAB*, Springer, 2024, pp. 1-255.
- Paul A. Gagniuc. *Coding Examples from Simple to Complex - Applications in Python*, Springer, 2024, pp. 1-245.
- Paul A. Gagniuc. *Coding Examples from Simple to Complex - Applications in Javascript*, Springer, 2024, pp. 1-240.
- Paul A. Gagniuc. *Markov chains: from theory to implementation and experimentation*. Hoboken, NJ, John Wiley & Sons, USA, 2017, ISBN: 978-1-119-38755-8.

<https://github.com/gagniuc>



Gagniuc

Overview Repositories 97 Projects Packages Stars 84

Gagniuc / README.md

ALGORITHMS IN BIOINFORMATICS
THEORY AND IMPLEMENTATION

MARKOV CHAINS
FROM THEORY TO IMPLEMENTATION AND EXPERIMENTATION

An Introduction to Programming Languages: Simultaneous Learning in Multiple Coding Environments

Note: all applications presented here are designed natively in their respective programming or scripting languages, without libraries or external files. Thus, the applications have no dependencies and can be used offline. There are about 100 posts uploaded [here](#), however, below are some useful projects that I have chosen to show as a preview.

Paul A. Gagniuc
Gagniuc

This page is dedicated to my students from Faculty of Engineering in Foreign Languages at University Politehnica of Bucharest. All projects are open source.